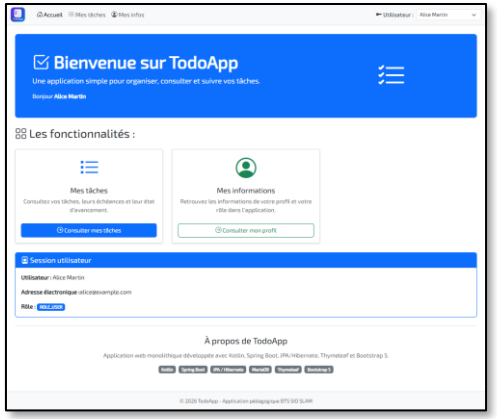
Développer une Application Monolithique avec Spring Boot

Application ToDoApp



PRÉSENTATION & OBJECTIFS

PRÉSENTATION & OBJECTIFS

- **Objectif principal :**
  - Durant ce TP vous allez apprendre à développer *"from scratch"* (de zéro) une application monolithique avec le framework **Spring Boot**.

- **Définition :**
  - Une application **monolithique** est une application informatique dont les différentes fonctionnalités sont regroupées au sein d'un même programme, développé, déployé et exécuté comme une **seule unité**.

  - **Remarque :**
    - La principale alternative à l'architecture **monolithique** est l'architecture **distribuée**. Dans une architecture distribuée, une application Web peut être décomposée en plusieurs applications : Un serveur API REST met à disposition les données; Les réponses (au format JSON) de l'API REST sont alors affichées soit dans une application mobile; une application de bureau ou dans une application frontend.

PRÉSENTATION & OBJECTIFS

- **À retenir :**

  - Une architecture monolithique n'implique pas une absence de structuration du code. Elle permet notamment de séparer les responsabilités grâce au modèle d'achitecture **MVC**, tout en conservant une application unique.

  - Dans notre TP **TodoApp**, le **M**odèle des données, les **V**ues et les **C**ontrôleurs seront ainsi regroupés au sein de la même application **Spring Boot TodoApp**.

PILE (STACK) TECHNOLOGIQE

PILE (STACK) TECHNOLOGIQUE

- **Langage & frameworks :**
  - Notre application sera développée avec le langage **Kotlin** et structurée par le frameworks **Spring Boot**.





PILE (STACK) TECHNOLOGIQUE

- **Gestion des données :**
  - Nous utiliserons **MariaDB** comme SGBD et l'ORM : **JPA** + **HYBERNATE**







PILE (STACK) TECHNOLOGIQUE

- **A propos des ORM :**

  - Un **ORM** (**O**bject-**R**elational **M**apping), ou **M**apping **O**bjet-**R**elationnel, est une technique de programmation qui permet d'établir une correspondance entre les objets d'une application et les tables d'une base de données relationnelle.

  - Il permet de manipuler les données sous forme d'objets, sans avoir à écrire explicitement les requêtes **SQL** pour les opérations courantes (Création, Lecture, Modification et Suppression) désignée **CRUD** (**C**reate; **R**ead; **U**pdate et **D**elete).

PILE (STACK) TECHNOLOGIQUE

- **A propos de JPA :**



  - **JPA** (**J**akarta **P**ersistence API) est une spécification Java qui définit un ensemble de règles et d'interfaces permettant de gérer la persistance des objets dans une base de données relationnelle.

  - **<u>Remarque :</u>**
    - Historiquement le "**J**" de **JPA** signifiait **J**ava, mais à la suite du rachat de Sun (créateur de Java) par Oracle, les droits de la propriété intellectuelle ont imposé ce changement.

PILE (STACK) TECHNOLOGIQUE

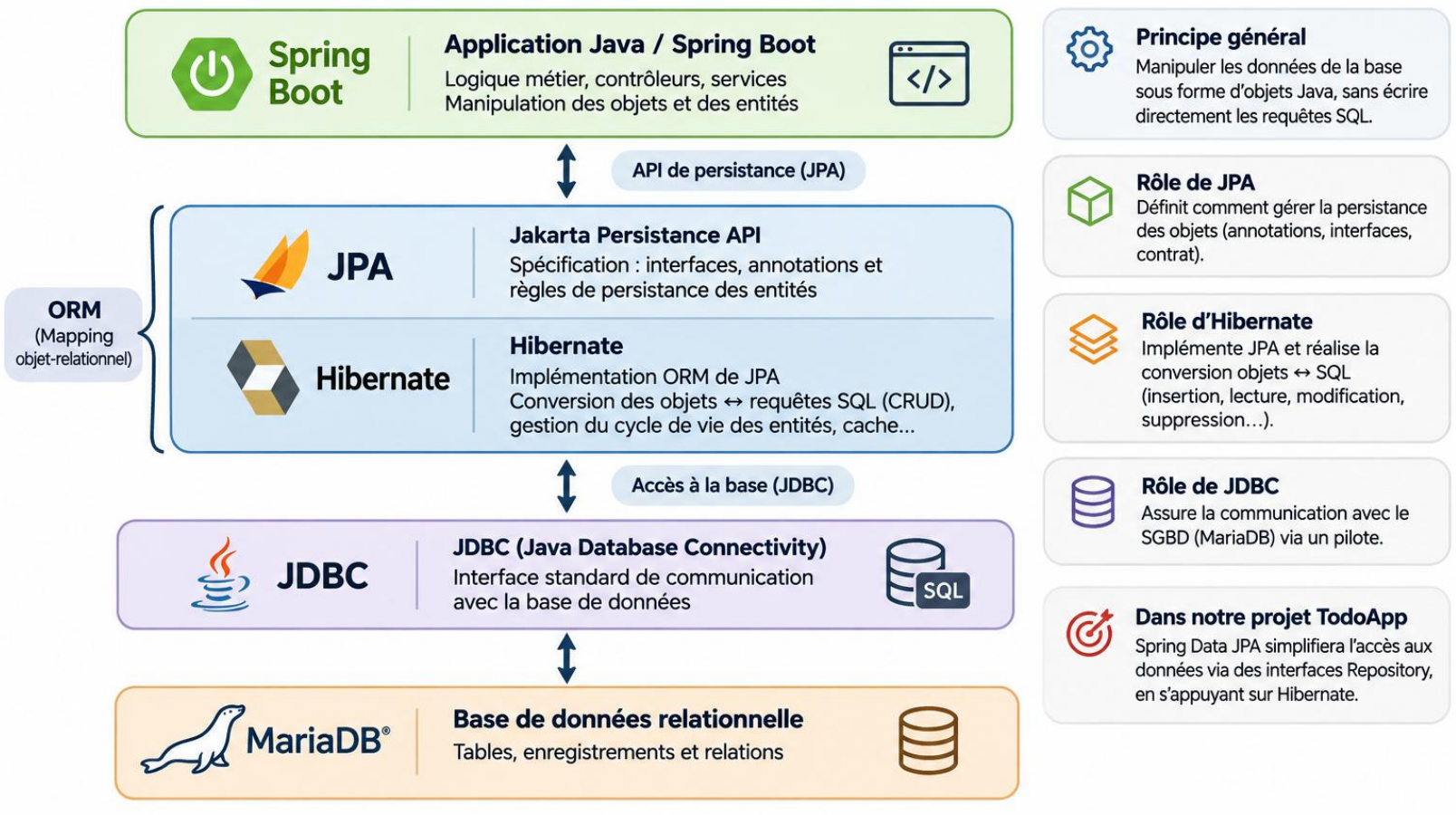
- **A propos d'Hibernate :**



  - Hibernate est un framework ORM qui implémente cette spécification. Il assure concrètement la correspondance entre les objets Java et les tables de la base de données, ainsi que la génération et l'exécution des requêtes SQL nécessaires.

PILE (STACK) TECHNOLOGIQUE

- **Schéma général du mécanisme de persistance des données :**



PILE (STACK) TECHNOLOGIQUE

- **Fonctionnement sur 3 principes :**
  - **Définition des entités :**
    - les classes **Java/Kotlin** sont associées aux tables de la base de données à l'aide d'annotations **JPA** comme `@Entity`, `@Id` ou `@Column`.
  - **Manipulation des objets :**
    - l'application utilise des objets **Java/Kotlin** pour créer, rechercher, modifier ou supprimer des données (CRUD).
  - **Persistance des données :**
    - **Hibernate** traduit les opérations sur les objets en requêtes **SQL**, exécutées sur la base de données via **JDBC**.

PILE (STACK) TECHNOLOGIQUE

- **Templating :**
  - Nous utiliserons **Thymeleaf** comme moteur de template.



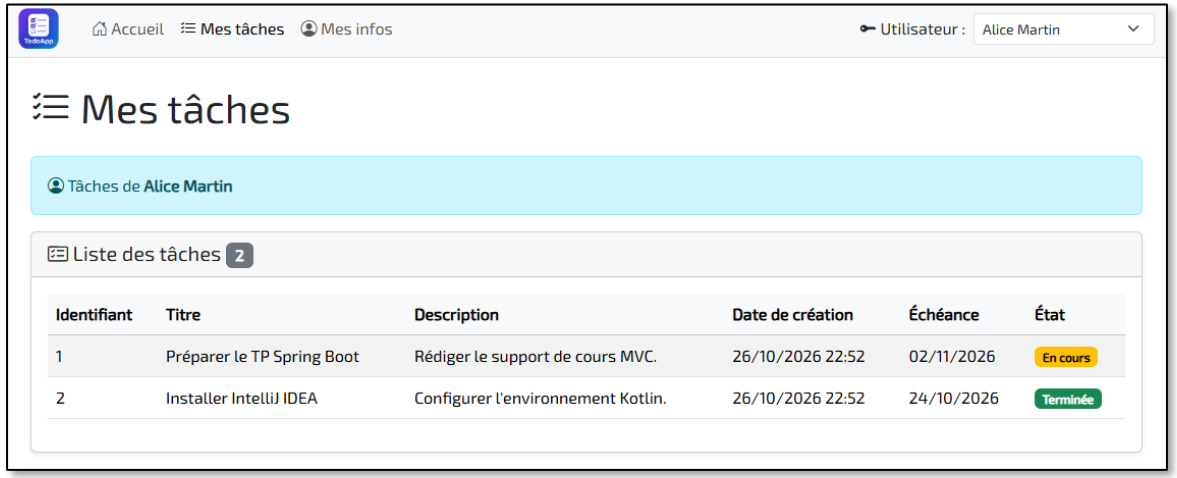
  - Nous utiliserons également **Bootstrap** pour élaborer les éléments **CSS** de nos vues.



PRÉSENTATION DE TODOAPP

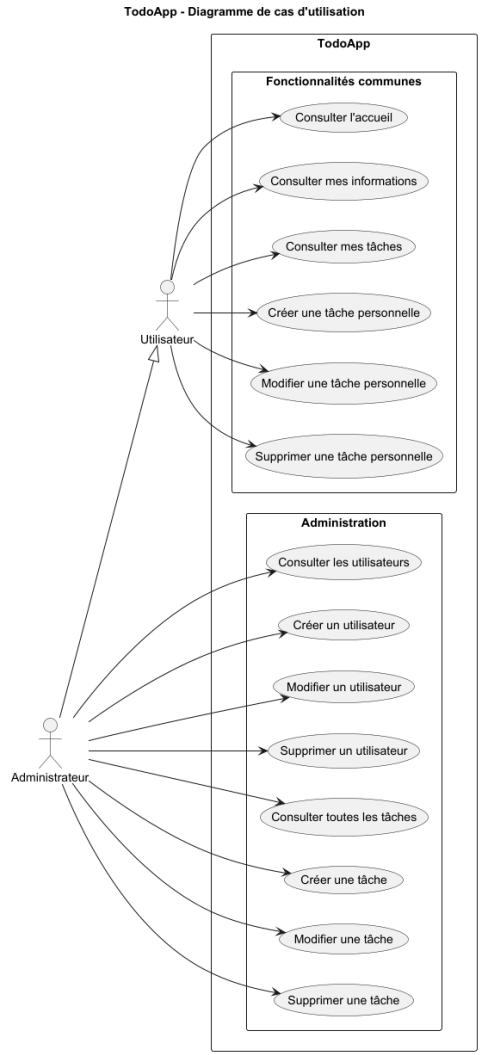
Présentation de TodoApp

- **Généralités :**
  - **TodoApp** est une application web de création de listes de tâches.
  - L'utilisateur pour associer à chaque tâche :
    - Un titre
    - Une description
    - Une date de début
    - Une date de fin
    - Un statut : *A faire, Terminée, En cours, etc.*



PRÉSENTATION DE TODOAPP

- **Liste des cas d'utilisation de TodoApp :**
  - Ci-contre, le diagramme des cas d'utilisation de l'application.
  - Nous distinguerons 2 types rôles :
    - **Le rôle d'utilisateur :**
      - Personne identifiée et qui pourra : *consulter; créer; modifier ou supprimer* ses tâches.
      - Elle pourra également *consulter* ses informations
    - **Le rôle d'administrateur :**
      - Hérite des cas d'utilisation d'un utilisateur standard, mais accède à un **backoffice** lui permettant :
        - Sur les utilisateurs : *Consulter*; *Créer*; *Modifier* et *Supprimer*.
        - Sur les tâches : *Consulter*; *Créer*; *Modifier* et *Supprimer*.

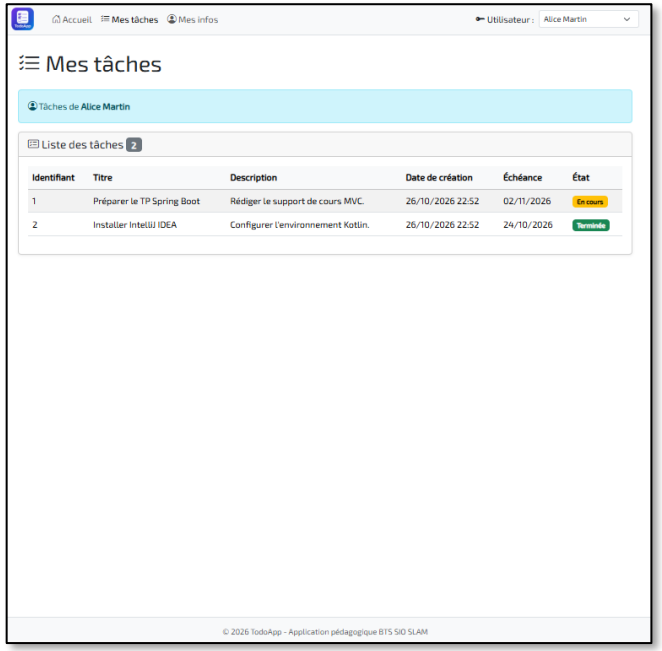


PRÉSENTATION DE TODOAPP

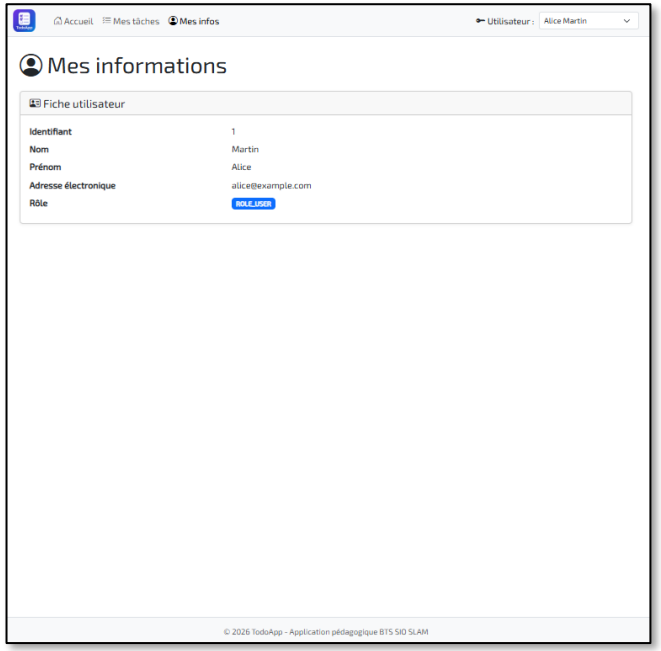
- **Interface proposée :**
  - Un **utilisateur** disposera de 3 pages pour exécuter ses cas d'utilisation.



Une page d'accueil



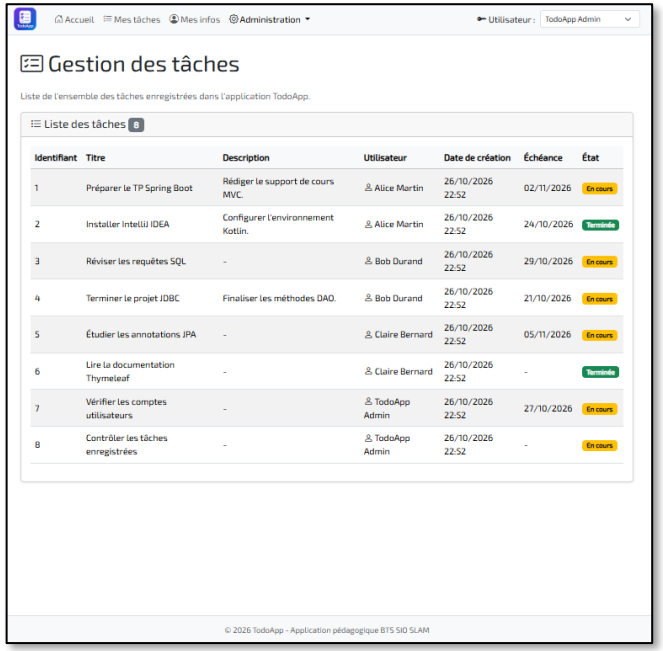
Une page pour les tâches



Une page pour les informations

Présentation de TodoApp

- **Interface proposée :**
  - Un **administrateur** dispose de 2 pages en backoffice.



Une page de gestion des tâches

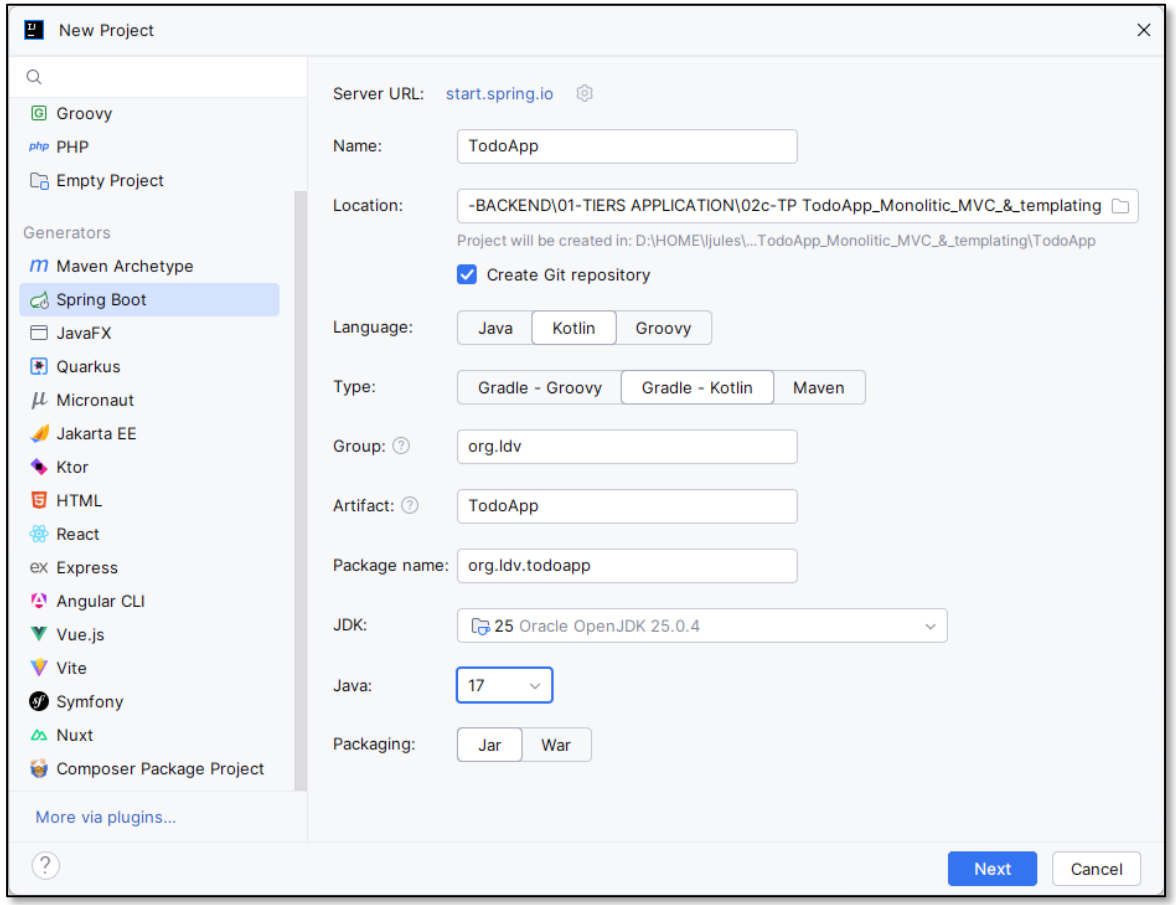


Une page de gestion des utilisateurs

INITIALISATION DU PROJET

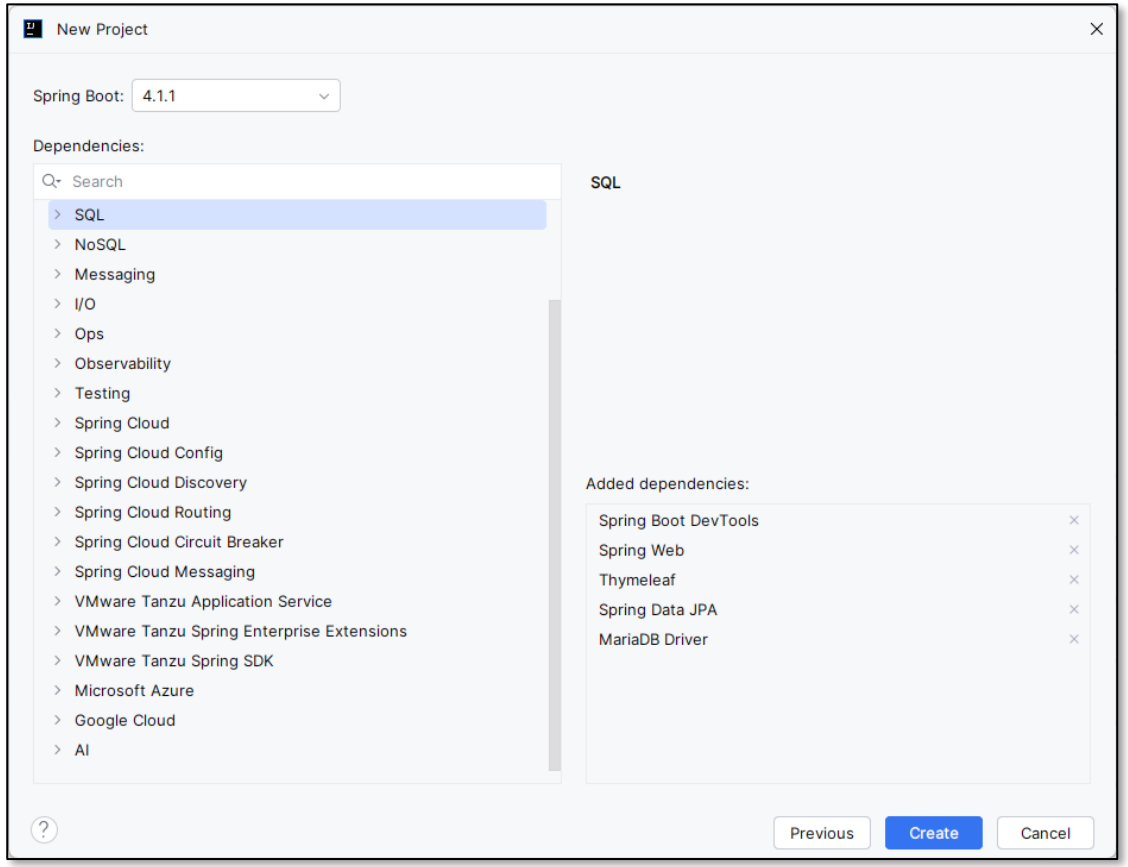
INITIALISATION DU PROJET

- **Lancer un nouveau projet sur IntelliJ avec les options suivantes :**
  - Name : `TodoApp`
  - Location : `Dossier de votre choix`
  - Cocher : `Create Get repository`
  - Language : `Kotlin`
  - Type : `Gradle – Kotlin`
  - Group : `org.ldv`
  - Artifact : `ToDoApp`
  - Package name : `org.ldv.todoapp`
  - JDK : `25`
  - Java : `17`
  - Packaging : `Jar`



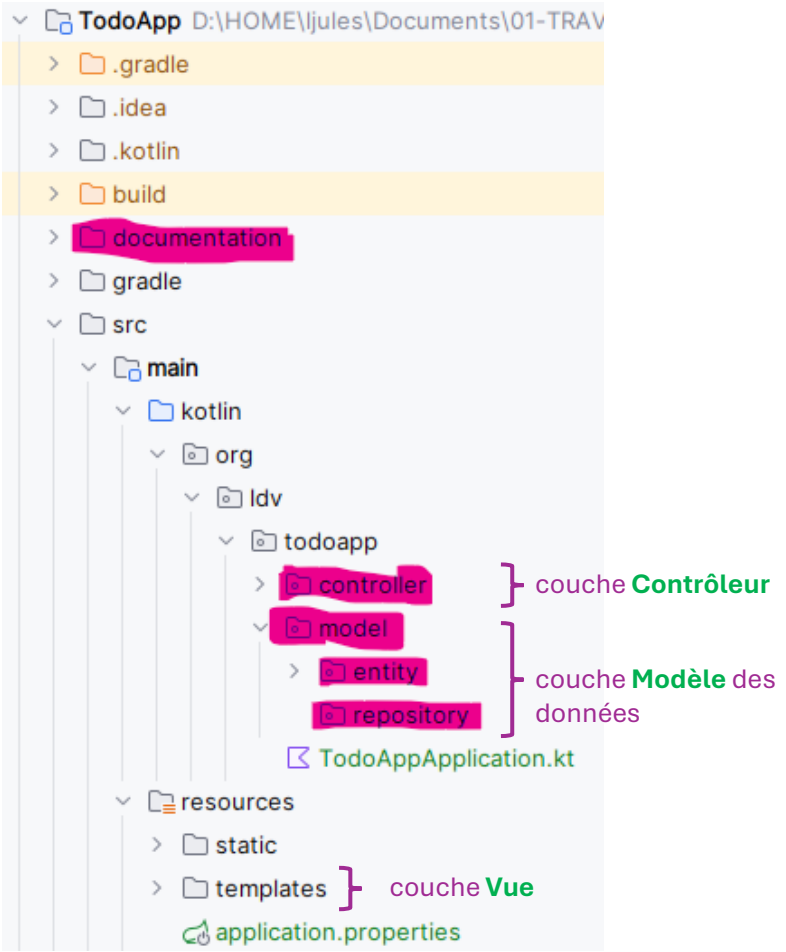
INITIALISATION DU PROJET

- **Choix d'options suivants :**
  - Choisir la version **`4.1.2`** de **Spring Boot**
  - Ajouter les dépendances suivantes :
    - **Developer Tools :**
      - `Spring Boot DevTools`
    - **Web :**
      - `Spring Web`
    - **Template Engines :**
      - `Thymleaf`
    - **SQL :**
      - `Spring Data JPA`
      - `MariaDB Driver`



INITIALISATION DU PROJET

- **Création de l'arborescence du projet :**
  - Construire les dossiers et packages
    - Pour accueillir la documentation créer un dossier `documentation` à la racine `TodoApp`.
    - Pour la couche **Modèle** des données :
      - Dans le package `todoapp` rajouter le package `model`
      - Dans le package model rajouter :
        - le package `entity`
        - le package `repository`
    - Pour la couche **Contrôleur**, rajouter le package `controller` dans le package `todoapp`.
    - Pour la couche **Vue** nous rajouterons plus tard des dossiers et fichiers dans le dossier `./src/ressources/template`



INITIALISATION DU PROJET

- **Ajout de la documentation :**
  - En amont une réflexion a été menée et a conduit la production de 3 diagrammes que nous vous fournissons sous la forme de fichiers **PlantUML** (`.plm`) :

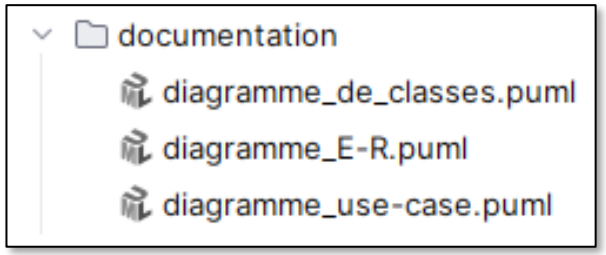
    - `diagramme_use-case.puml :` Diagramme des cas d'utilisation qui a été exposé au début de ce document pour présenter les cas d'utilisation de l'application.

    - `diagramme_de_classes.puml :` Diagramme de classes qui nous sera utile pour construire le modèle des données de notre application.

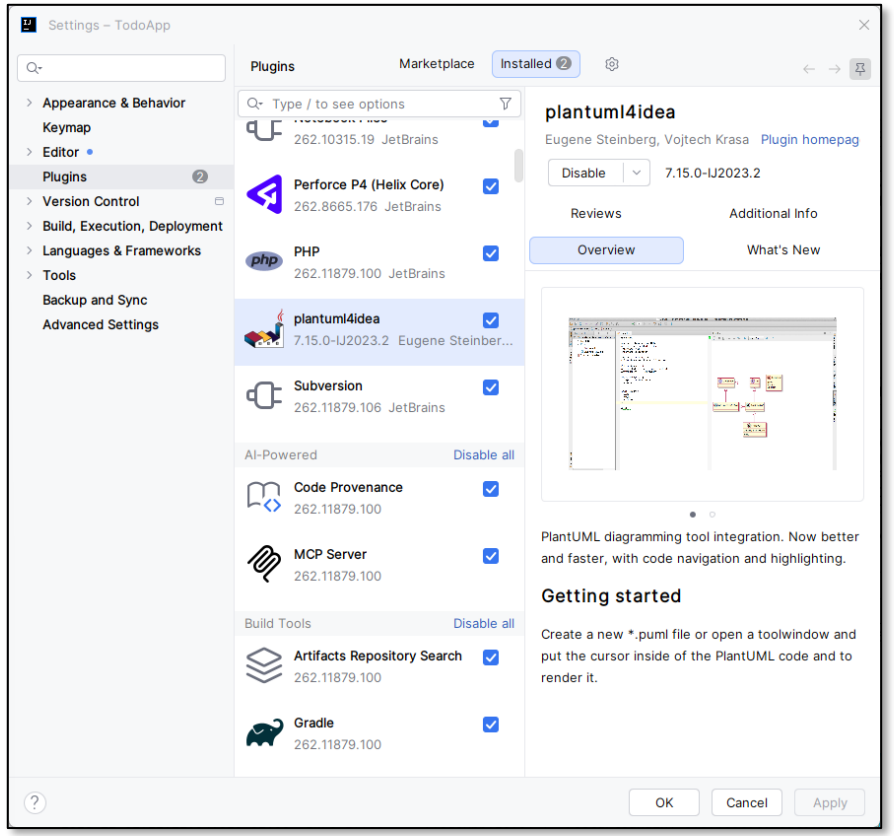
    - `diagramme_E-R.puml :` Diagramme Entité-Relation qui nous sera utilise pour valider le schéma relation de notre base de données.

INITIALISATION DU PROJET

- **Ajout de la documentation :**
  - Copier ou déplacer les 3 fichiers dans le dossier documentation :

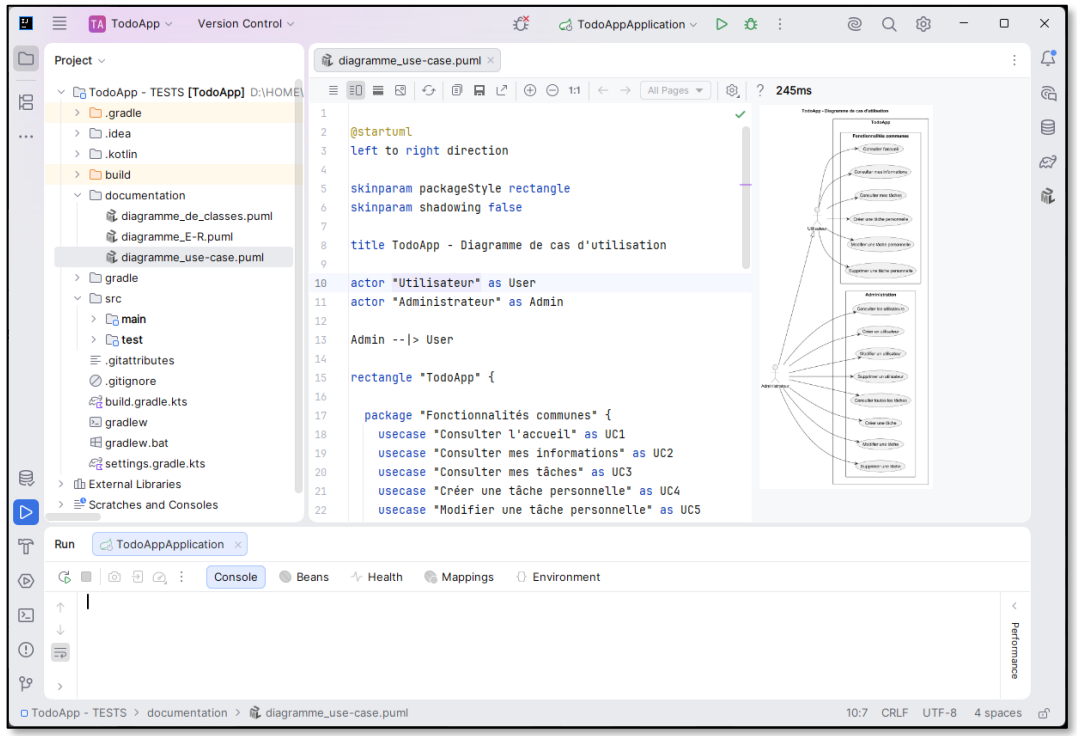


- **Ajout du plugin PlantUML :**
  - Pour pouvoir visualiser nos diagrammes IntelliJ doit disposer du plugin **plantuml4idea**.
  - Pour vérifier cela, aller dans le menu : **`File/Plugins…`**
  - Puis vérifier la présence du plugin, sinon le rajouter en allant dans la rubrique **Marketplace**.



INITIALISATION DU PROJET

- **Consultation de la documentation :**
  - Maintenant vous pouvez ouvrir et surtout consulter le rendu des diagrammes directement dans IntelliJ.



AJOUT DES BIBLIOTHÈQUES & ASSETS

AJOUT DES BIBLIOTHÈQUES & ASSETS

- **Stratégie des bibliothèques & assets frontend**
  - Pour la partie purement backend nous nous appuyons sur **Gradle** pour l'automatisation de la compilation mais également pour la gestion des dépendances : *pilote MariaDB*; *TomCat*; *JPA*; *Hibernate*; *Thymleaf*, etc.

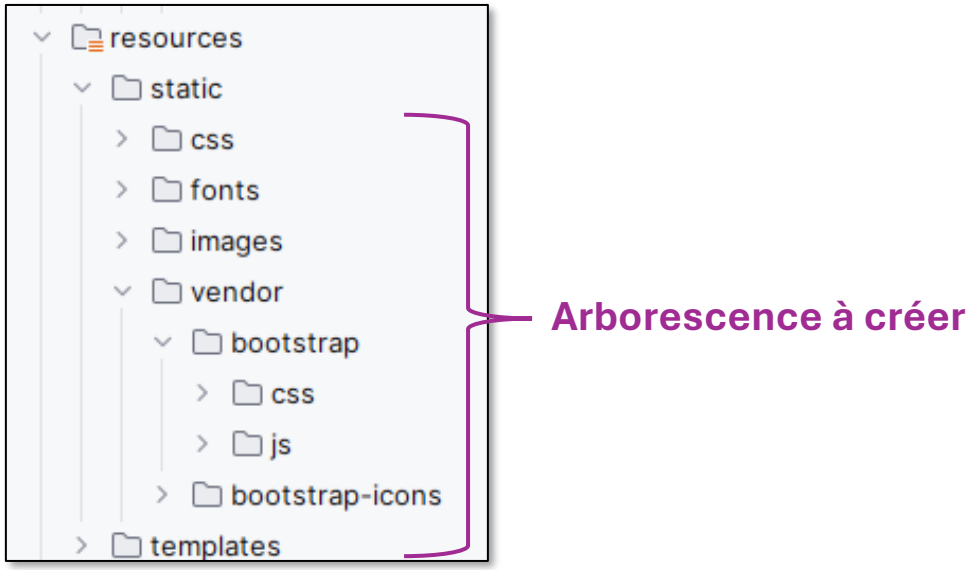


  - Cependant pour la partie frontend nous avons besoin d'autres bibliothèques que nous allons importer manuellement comme notamment **Bootstrap**.

    - **<u>Remarques :</u>**
      - Pour **Bootstrap** nous pourrions utiliser également **Gradle** ou aussi **npm** (nous utiliserons **npm** pour les projets **Angular**).

AJOUT DES BIBLIOTHÈQUES & ASSETS

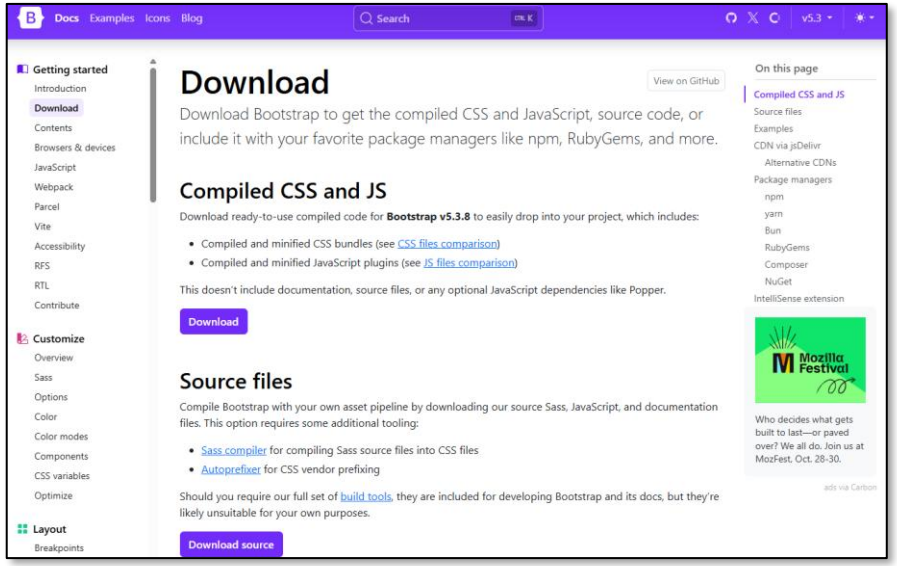
- **Structure du sous-dossier static :**
  - **Spring Boot** a créer le dossier `./src/ressources/static` durant l'initialisation du projet.
  - Il est configuré par défaut sur ce dossier pour que les chemins relatifs partent de ce dossier pour récupérer les ressources statiques. C'est-à-dire vos fichiers CSS, vos images, fonts, etc.
  - **A FAIRE :** Structurer ce dossier en créant l'arborescence suivante :



AJOUT DES BIBLIOTHÈQUES & ASSETS

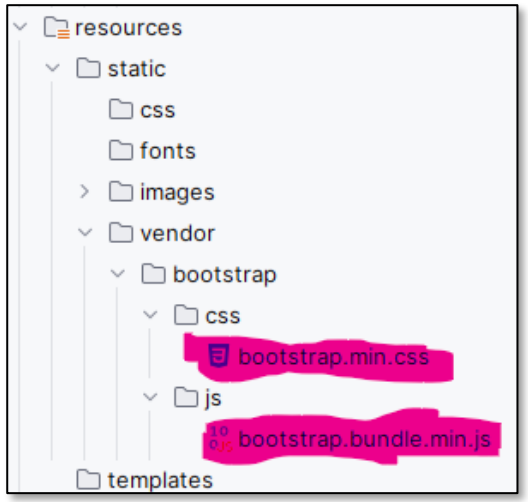
- **Récupération de Bootstrap :**
  - Pour notre projet nous utiliserons la dernière version de **Bootstrap**, soit la version **5.3**.
  - Rendez vous sur la page de téléchargement :
    - https://getbootstrap.com/docs/5.3/getting-started/download/

  - Cliquer le bouton ***Download*** pour récupérer l'archive :
    - **`bootstrap-5.3.8-dist.zip`**



AJOUT DES BIBLIOTHÈQUES & ASSETS

- **Extraction des fichiers utiles :**
  - Soit avec un clic droit sur le fichier **`bootstrap-5.3.8-dist.zip`** ou avec la commande : **`unzip bootstrap-5.3.8-dist.zip`**
  - Dans le dossier **`bootstrap-5.3.8-dist`** 2 fichiers nous intéressent :
    - **bootstrap-5.3.8-dist/css/ bootstrap.min.css**
    - **bootstrap-5.3.8-dist/js/ bootstrap.bundle.min.**js
  - Copier les deux fichiers précédents dans le dossier de votre projet :
    - dans `/vendor/bootstrap/css copier bootstrap.min.css`
    - dans `/vendor/bootstrap/js copier bootstrap.bundle.min.js`



AJOUT DES BIBLIOTHÈQUES & ASSETS

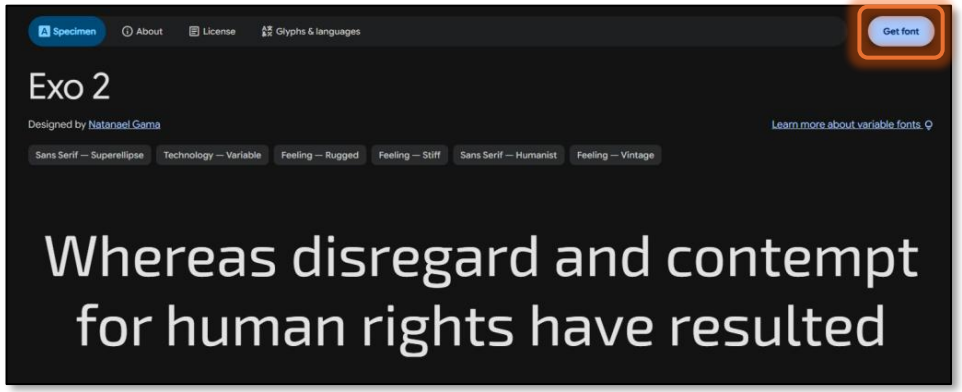
- **Ajout d'une police (font) :**
  - Nous vous proposons d'intégrer une police du site : https://fonts.google.com/ à votre projet.

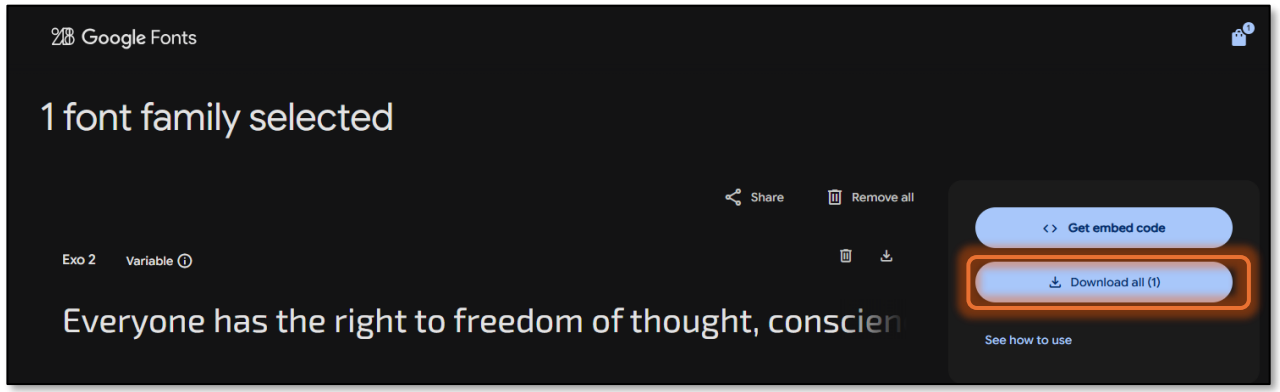
  - Vous êtes libre de prendre la police de votre choix, mais pour illustrer la démarche on va s'appuyer la police **exo2** qui est disponible sur la page : https://fonts.google.com/specimen/Exo+2?categoryFilters=Sans+Serif:%2FSans%2FSuperellipse



AJOUT DES BIBLIOTHÈQUES & ASSETS

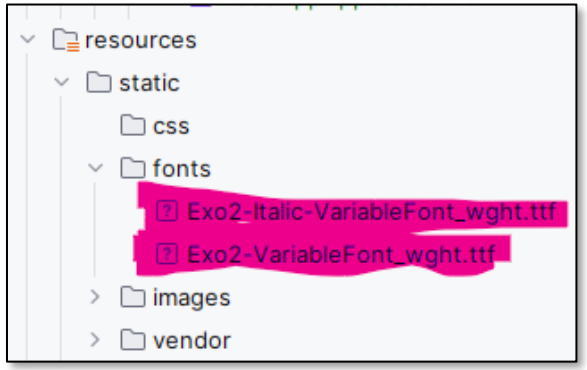
- **Ajout d'une police (font) :**
  - Télécharger la police en cliquant sur **Get Font** puis sur **Download all**





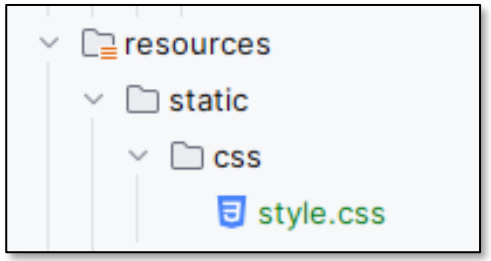
AJOUT DES BIBLIOTHÈQUES & ASSETS

- **Ajout d'une police (font) :**
  - Dans notre exemple nous avons téléchargé le fichier : **`Exo_2.zip`**
    - Dézipper ce fichier soit par un clic droit dessus, soit avec la commande :
      - **`unzip ./Exo_2.zip -d ./Exo_2`**
    - Dans le fichier d'extraction vous trouverez des fichiers **`.ttf`**, dans le cas de la police **Exo 2** nous en avons 2 :
      - `Exo2-Italic-VariableFont_wght.ttf`
      - `Exo2-VariableFont_wght.ttf`
  - Copier vos fichiers **`.ttf`** dans le dossier :
    - **`./resources/static/fonts`**



AJOUT DES BIBLIOTHÈQUES & ASSETS

- **Ajout d'une police (font) :**
  - Les polices sont maintenant disponibles en termes de fichiers, mais nous la déclarer comme police par défaut au niveau du style **CSS**.
  - Pour ce faire, créer un nouveau fichier : `/css/styles.css`



  - Puis compléter votre fichier `style.css` comme ci-dessous :

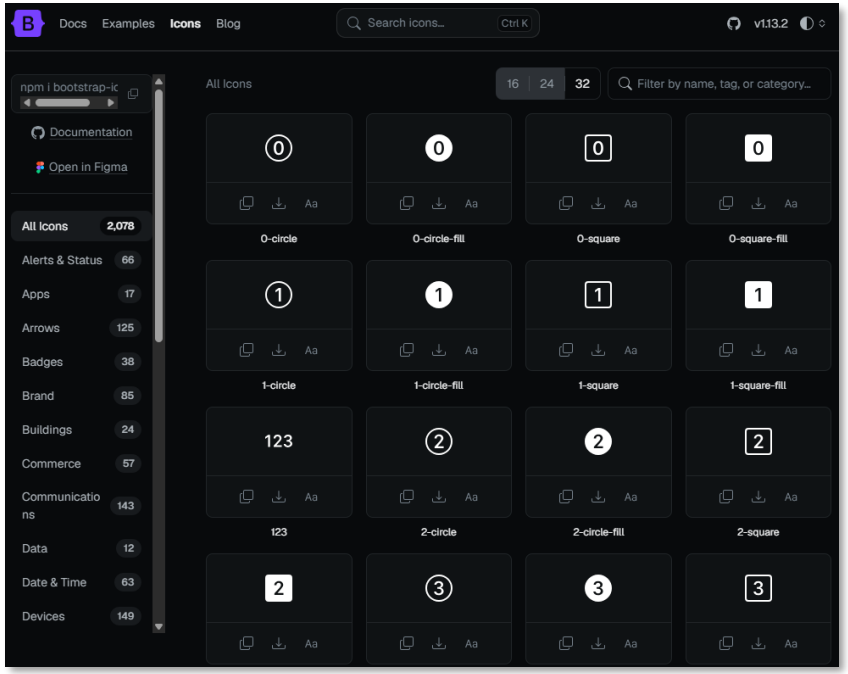
style.css

```css
@font-face {
    font-family: "MaPolice";
    src: url("../fonts/Exo2-VariableFont_wght.ttf") format("truetype");
    font-weight: 400;
    font-style: normal;
}

body {
    font-family: "MaPolice", sans-serif;
}
```

AJOUT DES BIBLIOTHÈQUES & ASSETS

- **Bootstrap Icons :**
  - **Bootstrap** met à disposition de nombreuses icônes que vous pouvez consulter sur la page : https://icons.getbootstrap.com/



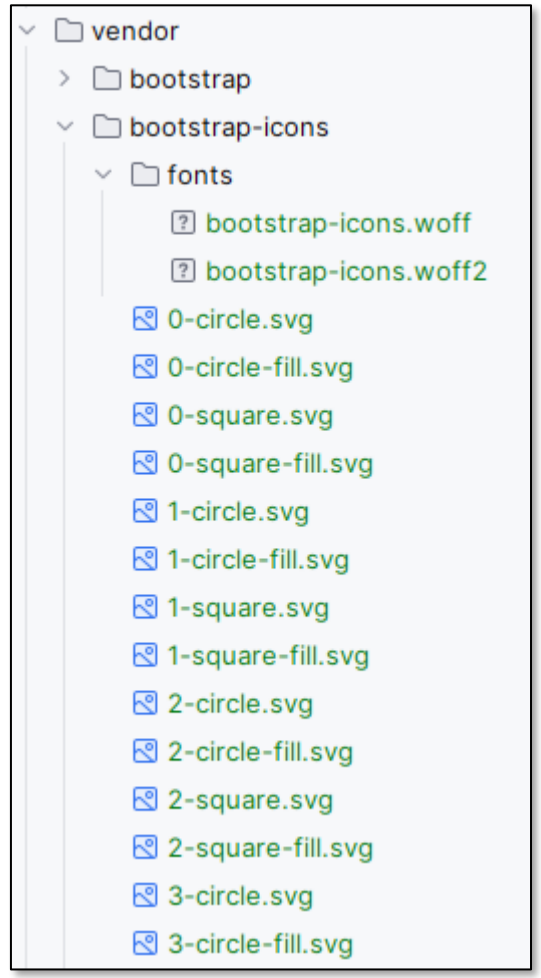
AJOUT DES BIBLIOTHÈQUES & ASSETS

- **Bootstrap Icons :**
  - On peut récupérer l'ensemble des fichiers de la bibliothèque à partir du dépôt officiel sur **Github** :
    - https://github.com/twbs/icons/releases/download/v1.13.2/bootstrap-icons-1.13.2.zip
  - Télécharger et décompresser l'archive avec un clic droit sur le fichier ou en exécutant la commande : **`unzip bootstrap-icons-1.13.2.zip`**
  - Dans le dossier **`bootstrap-icons-1.13.2`** récupérer le fichier : **`bootstrap-icons.min.css`** et tous les fichiers **`.svg`**
  - Récupérer également le dossier : **`bootstrap-icons-1.13.2/fonts`** qui contient les 2 fichiers : **`bootstrap-icons.woff`** et **`bootstrap-icons.woff2`**

AJOUT DES BIBLIOTHÈQUES & ASSETS

- **Bootstrap Icons :**

  - Copier le dossier `fonts`, le fichier `bootstrap-icons-min.css` et l'ensemble des fichiers `.svg` afin d'obtenir la même arborescence que l'image ci-contre.

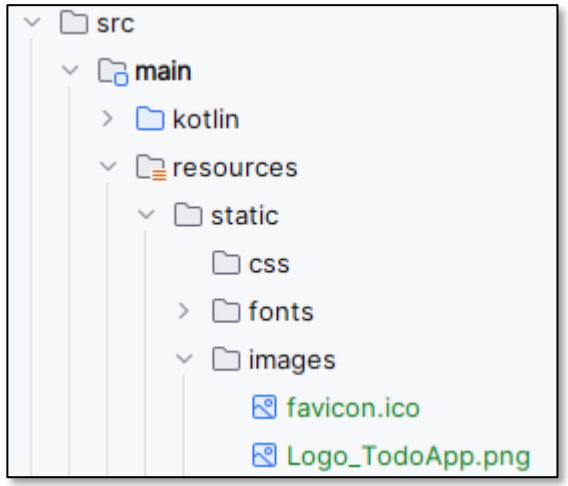


AJOUT DES BIBLIOTHÈQUES & ASSETS

- **Bootstrap Icons :**
  - Enfin nous vous fournissons le logo de notre application avec 2 fichiers :
    - `Logo_TodoApp.png`
    - `favicon.ico`

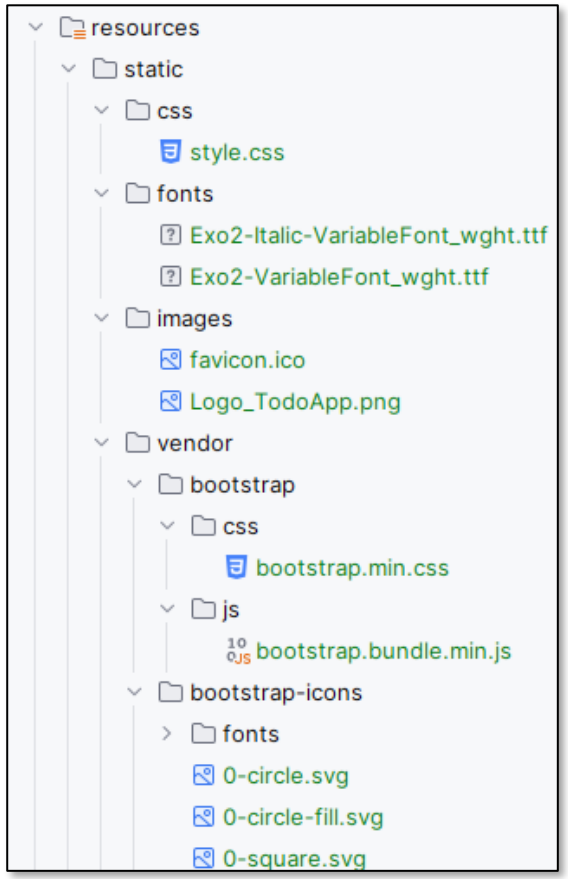


  - Copier ces 2 fichiers dans le dossier `/images`



AJOUT DES BIBLIOTHÈQUES & ASSETS

- **Bilan :**
  - Votre arborescence doit être la suivante :



CONFIGURATION DE L'ACCÈS À LA BD

CONFIGURATION DE L'ACCÈS À LA BD

- **Rappels sur votre SGBD :**
  - Vous disposez d'une machine virtuelle exécutant un serveur **MariaDB**
    - L'**IP** de votre VM est au format : **`172.17.70.X`**
    - Le **login** d'accès à **MariaDB** : **`sio`**
    - Le mot de passe d'accès à **MariaDB** : **`sio22`**

CONFIGURATION DE L'ACCÈS À LA BD

- **Information du SGBD dans le fichier `application.properties` :**
  - Ouvrir le fichier `application.properties` et le compléter comme ci-dessous en remplaçant le dernier octet de l'adresse IP par celui de votre VM :

```
spring.application.name=TodoApp

# Connexion MariaDB :
spring.datasource.url=jdbc:mariadb://172.17.70.X:3306/todoapp?createDatabaseIfNotExist=true
spring.datasource.username=sio
spring.datasource.password=sio22
spring.datasource.driver-class-name=org.mariadb.jdbc.Driver


# Gestion du schema par Hibernate :
spring.jpa.hibernate.ddl-auto=update
```

CLASSES & PERSISTENCE EN BD

CLASSES & PERSISTENCE EN BD

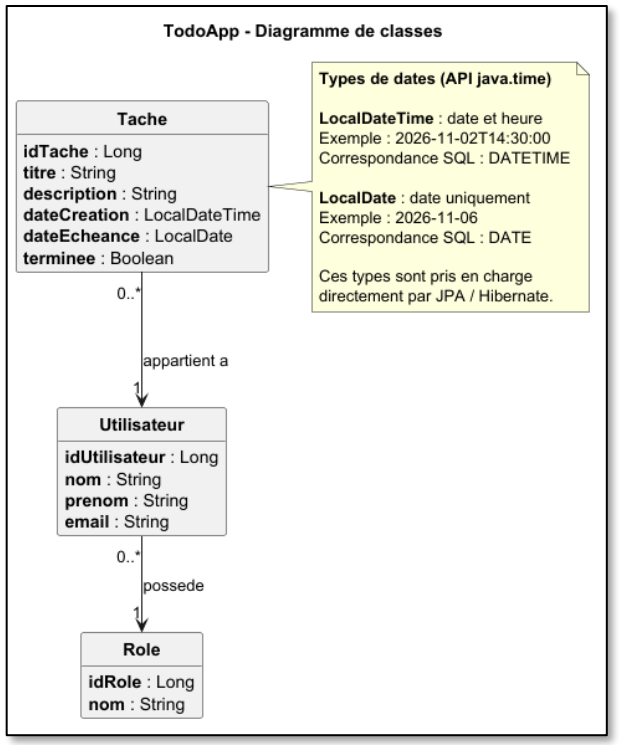
- **TodoApp - Une application "code-first" :**

  - 2 approches existent pour développer la base de données d'une application web :
    - L'approche historique dite **"database-first"** : Dans cas le schéma relationnel est construit indépendamment et en amont de l'application. L'application devra respecter le schéma relationnel de la base de données.

    - L'approche dite **"code-first"** : C'est le code de l'application (grâce à l'**ORM**) qui va construire un schéma relationnel dans le **SGBD**. Dans cette approche se sont le classes dites métiers (généralement désignées entités) qui imposent les tables et relations de la base de données.

CLASSES & PERSISTENCE EN BD

- **Modélisation Conceptuelle des Données :**
  - La **MCD** (**M**odélisation **C**onceptuel des **D**onnées) consiste à réaliser un diagramme de classes du modèle de données de notre application.
  - Nous vous proposons le diagramme de classes suivant :

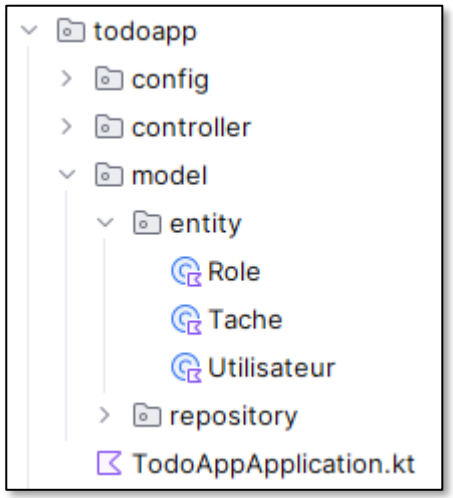


  - **<u>Remarque :</u>**
    - Durant le **MCD** nous représentons que les classes dont les données seront persistées (enregistrées) dans la base de données.
    - On désigne ces classes "entités".

CLASSES & PERSISTENCE EN BD

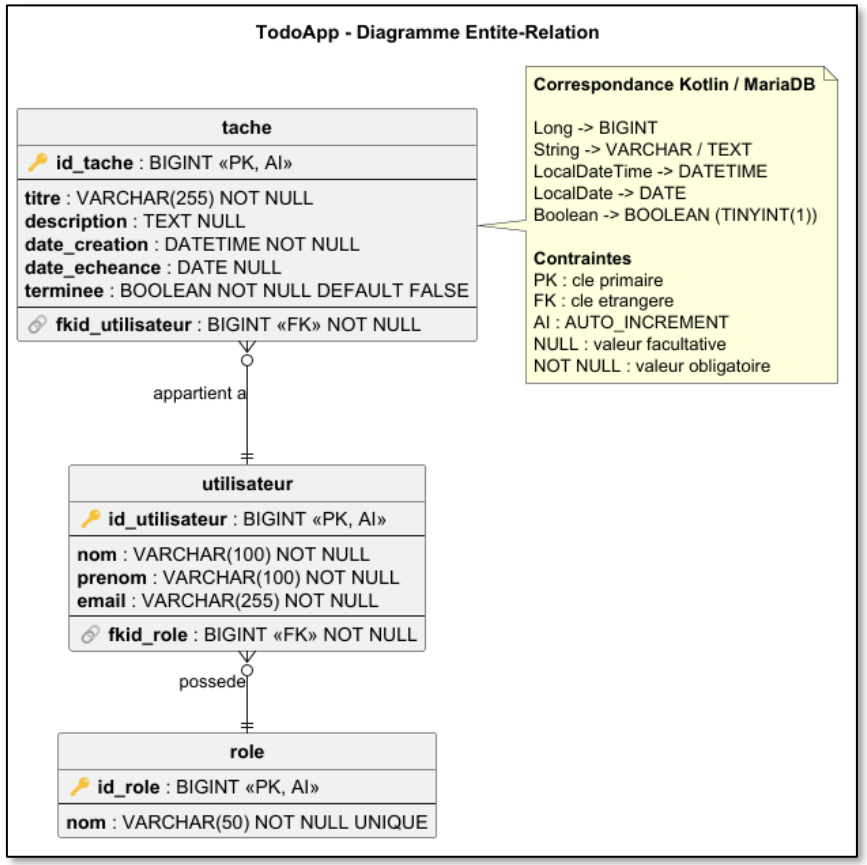
- **Création des fichiers des classes entités :**

  - Dans le package `model/entity` rajouter les 3 fichiers de classes suivants :
    - `Role.kt`
    - `Tache.kt`
    - `Utilisateur.kt`



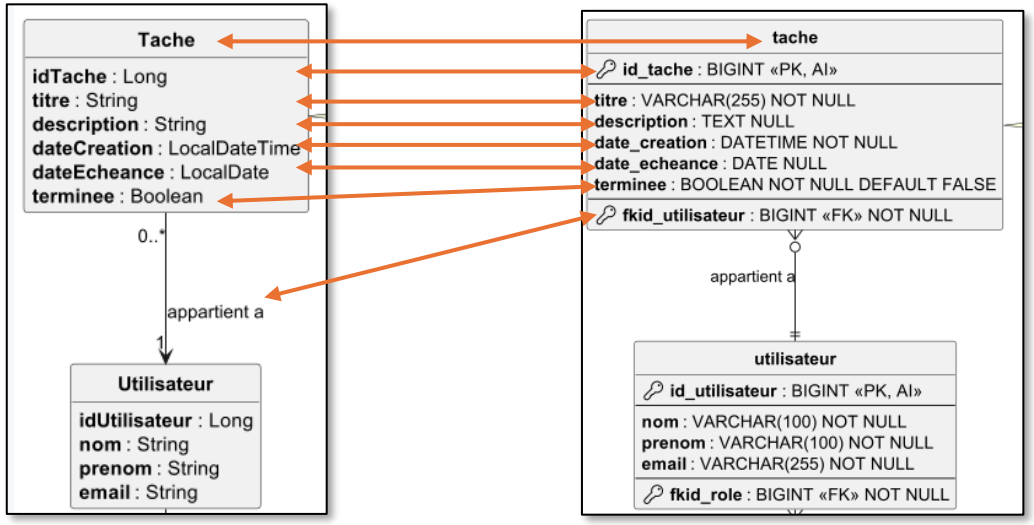
CLASSES & PERSISTENCE EN BD

- **Modélisation Logique des Données :**
  - L'**ORM JPA-Hibernate** va traduire les classes entités en tables **SQL**.
  - Notre application est simple par le fait qu'elle ne comporte que 3 entités/tables, mais surtout que nous n'avons pas d'association **many-to-many**. Cependant nous devons anticiper les types **SQL** adéquats.
  - Le diagramme **Entité-Relation** nous permet justement de représenter les types avec les tables.



CLASSES & PERSISTENCE EN BD

- **Implémentation des classes entités – l'entité `Tache` :**
  - Ouvrir le fichier `Tache.kt`.
  - L'analyse du diagramme de classes permet et/ou du diagramme E-R d'identifier 6 attributs :
    - **`idTache`**; **`titre`**; **`description`**; **`dateCreation`**; **`dateEcheance`**; **`terminee`**
  - Et nous avons un 7ème attribut que nous nommerons **`utilisateur`** qui correspond à l'association **many-to-one** des tâches d'un utilisateur.
  - Notre classe **`Tache`** devra notamment définir le **mapping** entre ses attributs et ceux de la table SQL correspondante.



Le diagramme de classe pour le "point de vue application Kotlin"

Le diagramme E-R pour le "point de vue SGBD"

CLASSES & PERSISTENCE EN BD

- **Implémentation des classes entités – l'entité `Tache` :**
  - Dans le fichier **`Tache.kt.`**
  - Commençons à procéder aux imports comme ci-dessous :

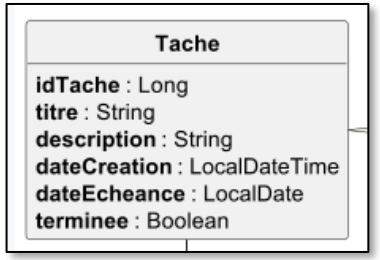
```
package org.ldv.todoapp.model.entity

import jakarta.persistence.*
import java.time.LocalDate
import java.time.LocalDateTime
```

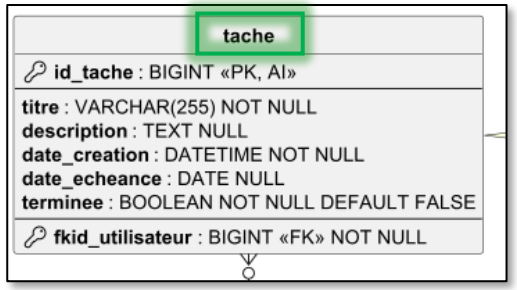
  - L'import **`jakarta.persistance.*`** permet de récupérer l'ORM JPA-Hibernate.
    En pratique nous utiliserons des annotations dédiées pour indiquer à Spring Boot nos spécifications (`@Entity`; `@Table`; `@Id`; `@Column`; ...) .
  - Comme nous avons besoin de manipuler des dates, on importe **`LocalDate`** et **`LocalDateTime`** :
    - **`LocaleDate` :** Génère des objets représentant des dates **sans l'heure** (jours, mois et année)
    - **`LocalDateTime` :** Génère des objets représentant des dates **avec l'heure**.

CLASSES & PERSISTENCE EN BD

- **Implémentation des classes entités – l'entité `Tache` :**
  - Ensuite nous devons rajouter 2 annotations pour l'identifiant de la classe :
    - `@Entity` : Permet de spécifier à **Spring Boot** que nous implémentons une classe/entité persistée en base de données.
    - `@Table(name = "tache")` : Permet de spécifier la table SQL associé à la classe.



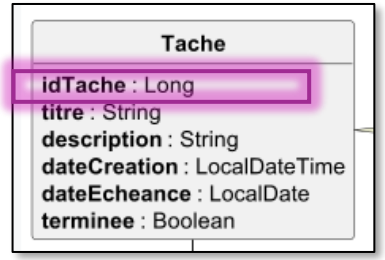

```
@Entity
@Table(name = "tache")
   class Tache(
   // A venir ...
)
```

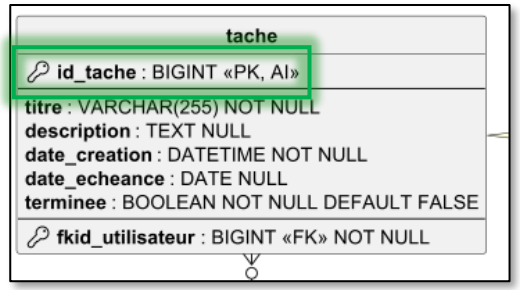


CLASSES & PERSISTENCE EN BD

- **Implémentation des classes entités – l'entité `Tache` :**
  - Cas de la clé primaire `idTache`/`id_tache` :
    - La clé primaire comporte 2 annotations spécifiques :
      - `@Id` : qui permet d'indiquer que l'attribut qui le suit est la clé primaire;
      - `@GeneratedValue(strategy = GenerationType.IDENTITY)` : Qui permet avec son argument de spécifier que notre la clé primaire sera générée par la base de donnes :
    - Et l'annotation `@Column(name = "id_tache")` qui permet de faire le mapping avec le nom de la table SQL.

```
@Entity
@Table(name = "tache")
 open class Tache(
     @Id
     @GeneratedValue(strategy = GenerationType.IDENTITY)
     @Column(name = "id_tache")
     open var idTache: Long? = null,
)
```



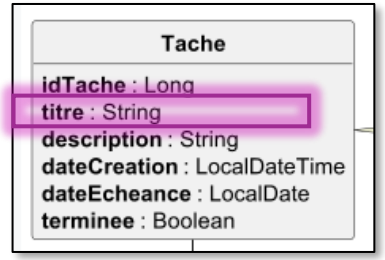


REMARQUES :

- L'attribut étant une **clé primaire** sera naturellement **non nul** dans MariaDB. Par contre du côté de Spring Boot nous sommes obligés d'initialiser l'attribut `idTache` avec **`null`** en attendant de récupérer la valeur de MariaDB.
- Les attributs sont déclarés avec **`var`** pour permettre les mises à jour avec la base de données.

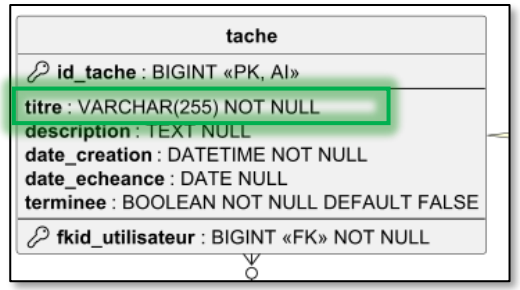
CLASSES & PERSISTENCE EN BD

- **Implémentation des classes entités – l'entité `Tache` :**
  - Attribut **`titre`**/**`titre`** :
    - Côté classe l'attribut **`titre`** est un **`String`** initialisé avec une chaîne vide.
    - Côté SQL l'attribut **`titre`** sera déclaré automatiquement en **VARCHAR** par Spring Boot. On précise avec l'annotation **`@Column`** qu'il est **non nul** et qu'il aura une longueur maxi de **255 caractères**, conformément à la description du diagramme Entité-Relation.



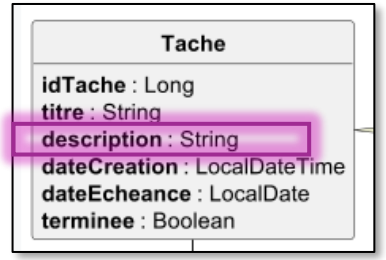

```
open class Tache(
    // ...
    @Column(name = "id_tache")
    open var idTache: Long? = null,

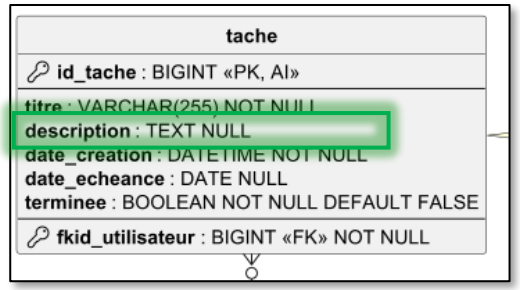
    @Column(name = "titre", nullable = false, length = 255)
     open var titre: String = "",
    // ...
)
```



CLASSES & PERSISTENCE EN BD

- **Implémentation des classes entités – l'entité `Tache` :**
  - Attribut **`description`**/**`descripiton`** :
    - Côté classe l'attribut **`description`** est un **`String?`** initialisé nul.
    - Côté SQL l'attribut **`description`** sera déclaré automatiquement en **TEXT** par Spring Boot. On précise avec l'annotation **`@Column`** qu'il est **nullable** et on précise son type SQL **TEXT** avec le paramètre **`columnDefinition = "TEXT"`**.





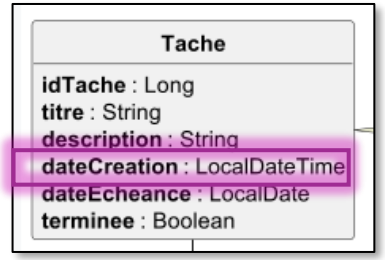

```
open class Tache(
    // ...
    @Column(name = "titre", nullable = false, length = 255)
     open var titre: String = "",

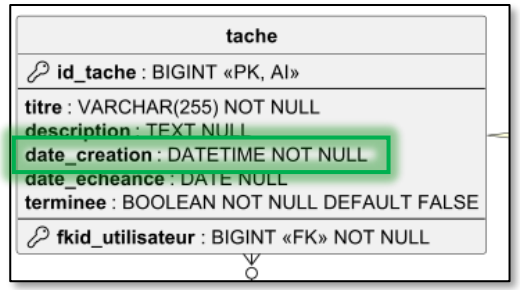
    @Column(name = "description", columnDefinition = "TEXT", nullable = true)
    open var description: String? = null,

    // ...
)
```

CLASSES & PERSISTENCE EN BD

- **Implémentation des classes entités – l'entité `Tache` :**
  - Attribut **`dateCreation`**/**`date_creation`** :
    - Côté classe l'attribut **`dateCreation`** est un **`LocalDateTime`**. Il est initialisé avec la date retournée par la méthode **`now()`**.
    - Côté SQL l'attribut **`date_creation`** est déclaré non nullable. Spring Boot le déclarera automatiquement avec le type SQL **`DATETIME`**.





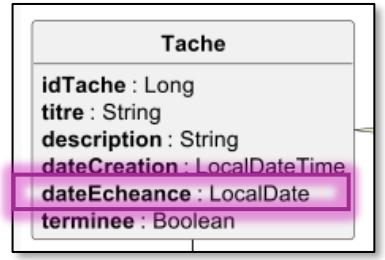

```
open class Tache(
    // ...
    @Column(name = "description", columnDefinition = "TEXT", nullable = true)
    open var description: String? = null,

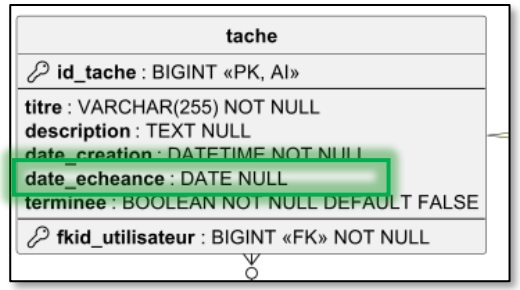
    @Column(name = "date_creation", nullable = false)
    open var dateCreation: LocalDateTime = LocalDateTime.now(),

    // ...
)
```

CLASSES & PERSISTENCE EN BD

- **Implémentation des classes entités – l'entité `Tache` :**
  - Attribut **`dateEcheance`**/**`date_echeance`** :
    - Côté classe l'attribut **`dateEcheanche`** est un **`LocalDate`**. Il est initialisé avec la date retournée par la méthode **`now()`**.
    - Côté SQL l'attribut **`date_echeance`** est déclaré nullable. Spring Boot le déclarera automatiquement avec le type SQL **`DATE`**.





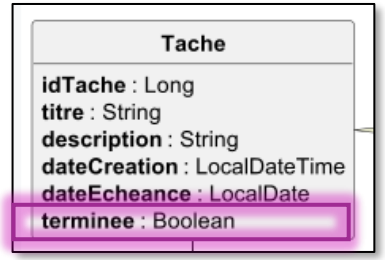

```
open class Tache(
    // ...
    @Column(name = "date_creation", nullable = false)
    open var dateCreation: LocalDateTime = LocalDateTime.now(),

    @Column(name = "date_echeance", nullable = true)
    open var dateEcheance: LocalDate? = null,

    // ...
)
```

CLASSES & PERSISTENCE EN BD

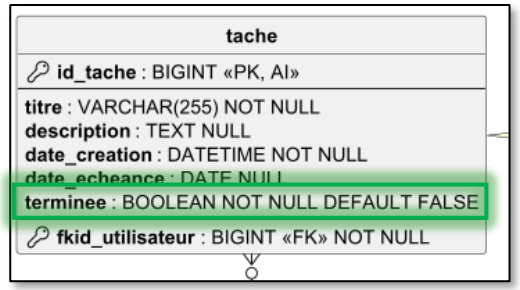
- **Implémentation des classes entités – l'entité `Tache` :**
  - Attribut **`terminee`**/**`terminee`** :
    - Côté classe l'attribut **`terminee`** est un **`Boolean`**. Il est initialisé à **`false`**.
    - Côté SQL l'attribut **`terminee`** est déclaré non nullable.




```
open class Tache(
    // ...
   @Column(name = "date_echeance", nullable = true)
   open var dateEcheance: LocalDate? = null,

   @Column(name = "terminee", nullable = false)
   open var terminee: Boolean = false,

    // ...
)
```

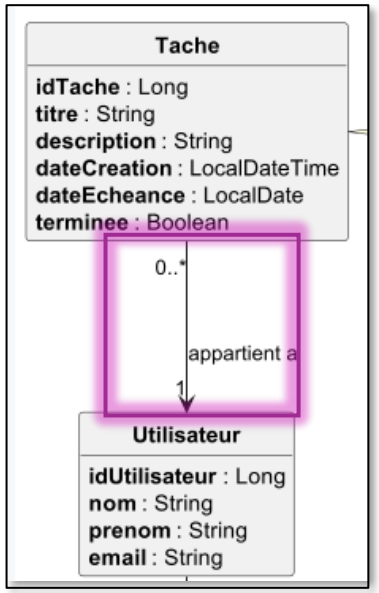


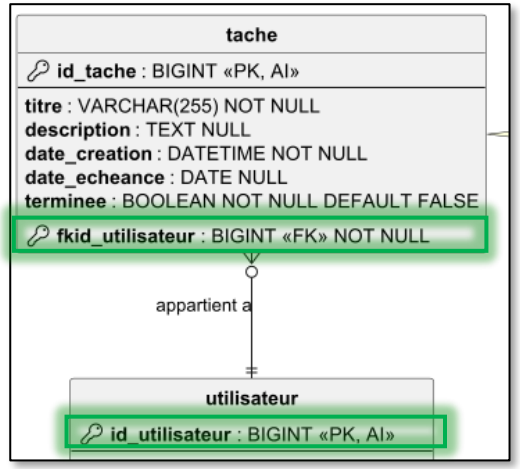
CLASSES & PERSISTENCE EN BD

- **Implémentation des classes entités – l'entité `Tache` :**
  - Attribut `utilisateur`/`fkid_utilisateur` & association `ManyToOne` :
    - On déclare l'association avec l'annotation `@ManyToOne` le paramètre **`fetch`** est optionnel, mais **IntelliJ** proposera **`FetchType.LAZY`** en auto-complétion. Le mode LAZY pour faire simple minimisera les échanges avec la BD au strict nécessaire.
    - Côté classe l'attribut `utilisateur` référence un objet de type **`Utilisateur?`**. Il est initialisé à **`null`** (la dépendance sera injectée ultérieurement).
    - Côté SQL, ici on n'utilise pas un simple `@Column`, mais `@JoinColumn` qui permet de préciser l'attribut de clé étrangère `fkid_utilisateur` (déclaré non nullable). le paramètre **referencedColumnName** permet de préciser l'identifiant de la clé primaire `id_utilisateur` dans la table utilisateur. La table utilisateur n'a pas besoin d'être précisée car elle est implicitement connue grâce au contexte du code.

```
open class Tache(
    // ...
  @Column(name = "terminee", nullable = false)
  open var terminee: Boolean = false,

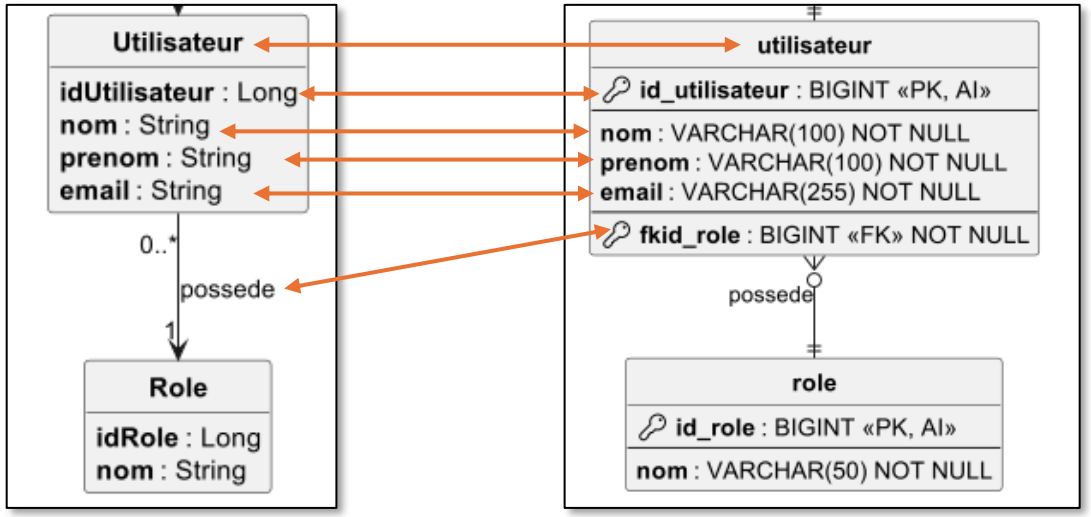
   @ManyToOne(fetch = FetchType.LAZY)
   @JoinColumn( name = "fkid_utilisateur", referencedColumnName = "id_utilisateur", nullable = false)
    open var utilisateur: Utilisateur? = null
)
```





CLASSES & PERSISTENCE EN BD

- **Implémentation des classes entités – l'entité Utilisateur :**
  - Ouvrir le fichier `Utilisateur.kt`.
  - L'analyse du diagramme de classes permet et/ou du diagramme E-R d'identifier 4 attributs :
    - **`idUtilisateur`**; **`nom`**; **`prenom`**; **`email`**
  - Et nous avons un 5ème attribut que nous nommerons **`role`** qui correspond à l'association **many-to-one** des utilisateurs d'un rôle.
  - Notre classe **`Utilisateur`** devra notamment définir le **mapping** entre ses attributs et ceux de la table SQL correspondante.



Le diagramme de classe pour le "point de vue application Kotlin"

Le diagramme E-R pour le "point de vue SGBD"

CLASSES & PERSISTENCE EN BD

- **Implémentation des classes entités – l'entité `Utilisateur` :**
  - Dans le fichier **`Utilisateur.kt.`**
  - Commençons à procéder aux imports comme ci-dessous qui se limite ici **`jakarta.persistence.*`** :

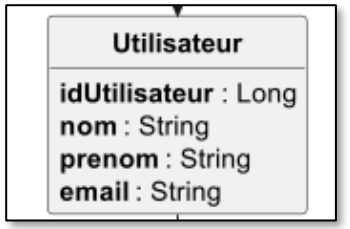
```
package org.ldv.todoapp.model.entity

import jakarta.persistence.*
```

CLASSES & PERSISTENCE EN BD

- **Implémentation des classes entités – l'entité `Utilisateur` :**
  - Ajoutons les 2 annotations pour l'identifiant de la classe :
    - `@Entity` : Permet de spécifier à **Spring Boot** que nous implémentons une classe/entité persistée en base de données.
    - `@Table(name = "utilisateur")` : Permet de spécifier la table SQL associé à la classe.

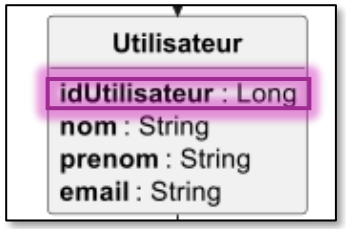
```
@Entity
@Table(name = "utilisateur")
open class Utilisateur(
  // …
)
```

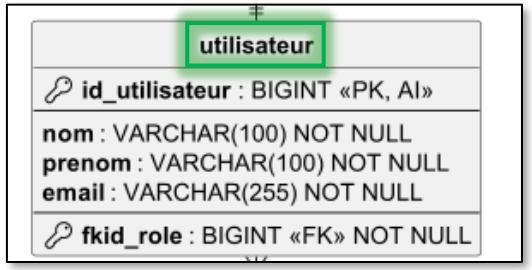




CLASSES & PERSISTENCE EN BD

- **Implémentation des classes entités – l'entité `Utilisateur` :**
  - Cas de la clé primaire `idTache`/`id_tache` :
    - La clé primaire comporte 2 annotations spécifiques :
      - `@Id` : Indique l'attribut qui est associé à la clé primaire;
      - `@GeneratedValue(strategy = GenerationType.IDENTITY)` : Spécifie que notre la clé primaire sera générée par la base de donnes :
    - `@Column(name = "id_utilisateur")` réalise le mapping avec le nom de la table SQL.






```
@Entity
@Table(name = "utilisateur")
open class Utilisateur(
  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  @Column(name = "id_utilisateur")
  open var idUtilisateur: Long? = null,

  // ...
)
```

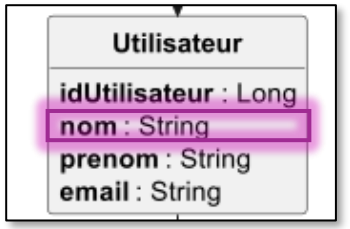
CLASSES & PERSISTENCE EN BD

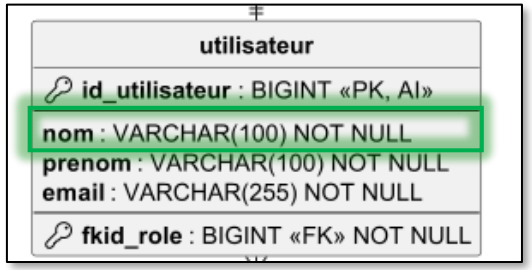
- **Implémentation des classes entités – l'entité `Utilisateur` :**
  - Attribut `nom`/`nom` :
    - Côté classe l'attribut `nom` est un **`String`** initialisé avec une chaîne vide.
    - Côté SQL l'attribut `nom` sera déclaré automatiquement en **VARCHAR** par Spring Boot.
      On précise avec l'annotation `@Column` qu'il est **non nul** et qu'il aura une longueur maxi de **100 caractères.**

```
open class Utilisateur(
  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  @Column(name = "id_utilisateur")
  open var idUtilisateur: Long? = null,

  @Column(name = "nom", nullable = false, length = 100)
  open var nom: String = "",

  // ...
)
```





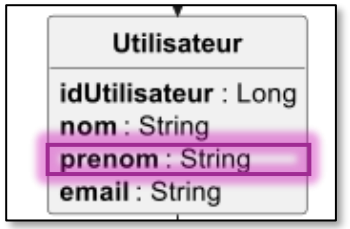
CLASSES & PERSISTENCE EN BD

- **Implémentation des classes entités – l'entité `Utilisateur` :**
  - Attribut **`prenom`**/**`prenom`** :
    - Côté classe l'attribut **`prenom`** est un **`String`** initialisé avec une chaîne vide.
    - Côté SQL l'attribut **`prenom`** sera déclaré automatiquement en **VARCHAR** par Spring Boot. On précise avec l'annotation **`@Column`** qu'il est **non nul** et qu'il aura une longueur maxi de **100 caractères.**

```
open class Utilisateur(
  // ...
  @Column(name = "nom", nullable = false, length = 100)
  open var nom: String = "",

  @Column(name = "prenom", nullable = false, length = 100)
  open var prenom: String = "",


  // ...
)
```





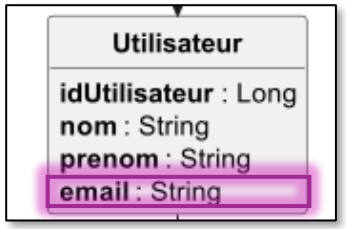
CLASSES & PERSISTENCE EN BD

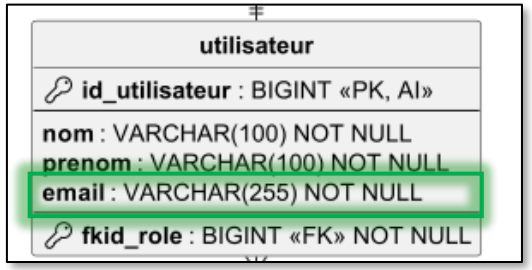
- **Implémentation des classes entités – l'entité `Utilisateur` :**
  - Attribut **`email`**/**`email`** :
    - Côté classe l'attribut **`email`** est un **`String`** initialisé avec une chaîne vide.
    - Côté SQL l'attribut **`email`** sera déclaré automatiquement en **VARCHAR** par Spring Boot. On précise avec l'annotation **`@Column`** qu'il est **non nul** et qu'il aura une longueur maxi de **255 caractères.**

```
open class Utilisateur(
  // ...
  @Column(name = "prenom", nullable = false, length = 100)
  open var prenom: String = "",

  @Column(name = "email", nullable = false, unique = true, length = 255)
  open var email: String = "",

  // ...
)
```



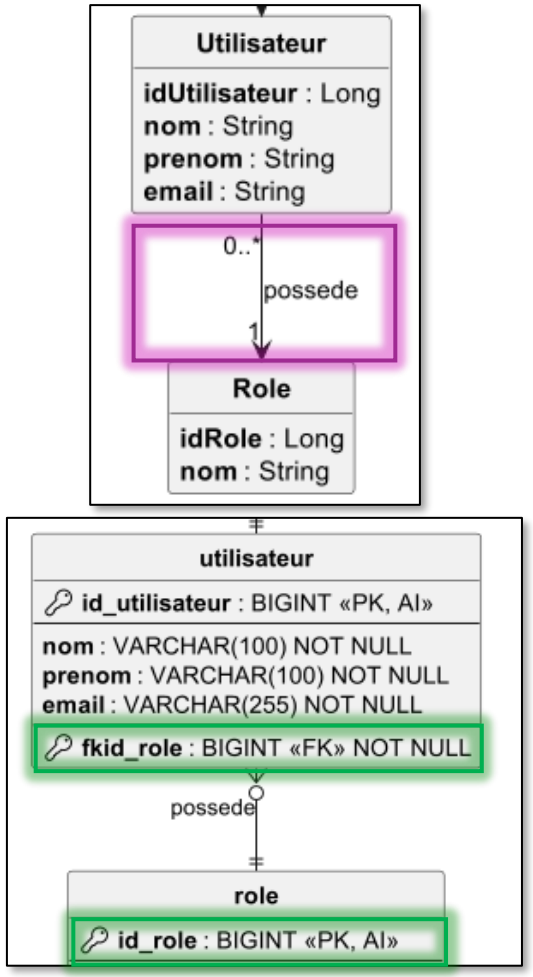


CLASSES & PERSISTENCE EN BD

- **Implémentation des classes entités – l'entité `Utilisateur` :**
  - Attribut **`role`**/**`fkid_role`** & association `ManyToOne` :
    - **`@ManyToOne`** avec **`FetchType.LAZY`** pour déclarer l'association.
    - Côté classe l'attribut **`role`** référence un objet de type **`Role?`**.
      Il est initialisé à **`null`** (dépendance injectée ultérieurement).
    - Côté SQL, **`@JoinColumn`** précise l'attribut pour la clé étrangère **`fkid_rôle`**
      (déclaré non nullable). **referencedColumnName** précise l'identifiant de la clé primaire **`id_role`** dans la table **`role`**.

```
open class Utilisateur(
  // ...
  @Column(name = "email", nullable = false, unique = true, length = 255)
  open var email: String = "",

  @ManyToOne(fetch = FetchType.LAZY)
  @JoinColumn(name = "fkid_role", referencedColumnName = "id_role", nullable = false)
  open var role: Role? = null
)
```



CLASSES & PERSISTENCE EN BD

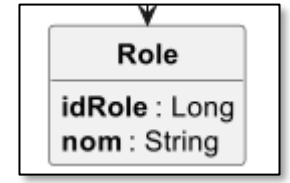
- **Implémentation des classes entités – l'entité `Role` :**
  - Ouvrir le fichier **`Role.kt`**.
  - Ici la classe est simple, nous vous fournissons directement le code sans analyse particulière à l'exception du paramètre **`unique = true`** de l'annotation **`@Column`** qui spécifie une contrainte pour imposer que chaque rôle possède un nom unique.

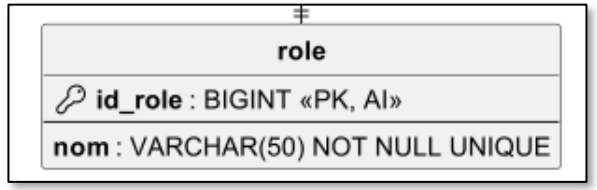
```kotlin
package org.ldv.todoapp.model.entity

import jakarta.persistence.*

@Entity
@Table(name = "role")
open class Role(
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id_role")
    open var idRole: Long? = null,

    @Column(name = "nom", nullable = false, unique = true, length = 50)
    open var nom: String = ""
)
```





CLASSES & PERSISTENCE EN BD

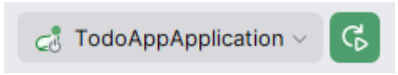
- **Test fonctionnel & validation :**
  - Il est temps maintenant de vérifier que notre application réalise les tâches suivantes à sa première exécution :
    - Création d'une base de données **todoapp** sur votre serveur MariaDB avec l'instruction donnée dans le fichier **`application.properties`**

```
# Connexion MariaDB :
spring.datasource.url=jdbc:mariadb://localhost:3306/todoapp?createDatabaseIfNotExist=true
```

    - Construction des tables et de leurs relations par JPA-Hibernate
    - La construction du schéma relationnel à partir du code de l'application est rendue possible par l'option renseigne également dans le fichier **`application.properties`**

```
# Gestion du schema par Hibernate
spring.jpa.hibernate.ddl-auto=update
```

CLASSES & PERSISTENCE EN BD

- **Test fonctionnel & validation :**
  - Exécuter votre application : 

  - Si tout se passe bien, vous application doit tourner en boucle.

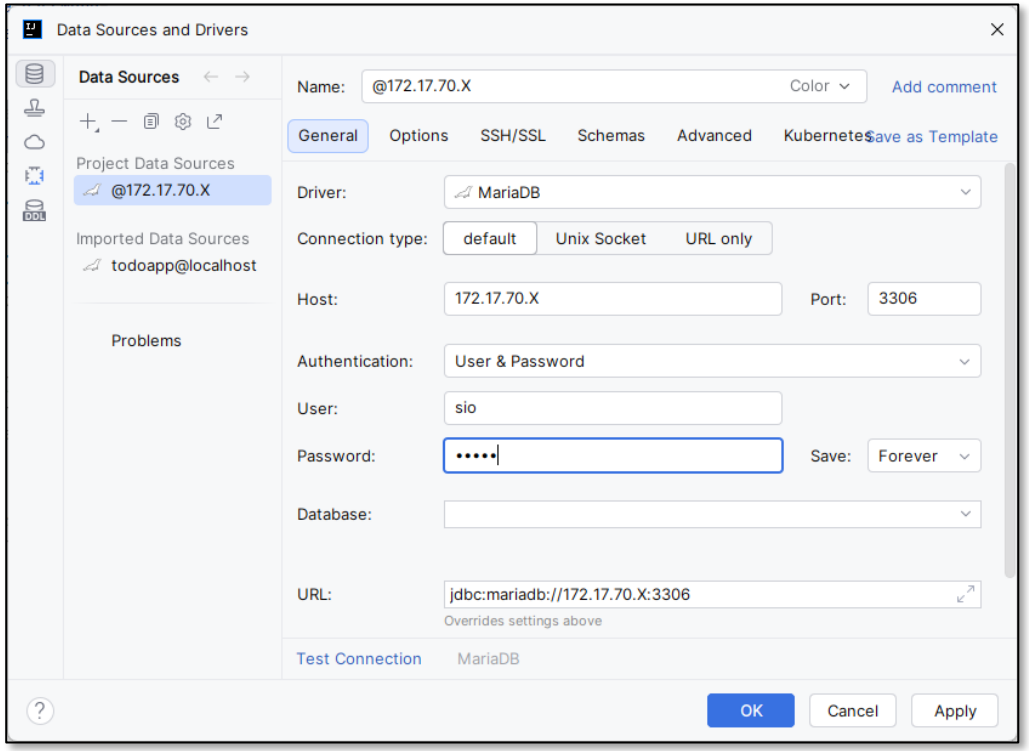
```
 .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/

 :: Spring Boot ::                (v4.1.1)

…
2026-11-01T09:56:22.833+01:00  INFO 13880 --- [TodoApp] [  restartedMain] org.hibernate.orm.connections.pooling    : HHH10001005: Database info:
                        Database JDBC URL [jdbc:mariadb://localhost/todoapp?user=root&createDatabaseIfNotExist=true]
                        Database driver: MariaDB Connector/J
                        Database dialect: MariaDBDialect
                        Database version: 11.1.2
                        Default catalog/schema: todoapp/undefined
                        Autocommit mode: undefined/unknown
                        Isolation level: REPEATABLE_READ [default REPEATABLE_READ]
                        JDBC fetch size: none
                        Pool: DataSourceConnectionProvider
                        Minimum pool size: undefined/unknown
                        Maximum pool size: undefined/unknown
2026-11-01T09:56:24.082+01:00  INFO 13880 --- [TodoApp] [  restartedMain] org.hibernate.orm.core                  : HHH000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to enable JTA platform integration)
2026-11-01T09:56:24.303+01:00  INFO 13880 --- [TodoApp] [  restartedMain] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
2026-11-01T09:56:24.943+01:00  INFO 13880 --- [TodoApp] [  restartedMain] o.s.d.j.r.query.QueryEnhancerFactories   : Hibernate is in classpath; If applicable, HQL parser will be used.
2026-11-01T09:56:26.440+01:00  WARN 13880 --- [TodoApp] [  restartedMain] JpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed during view rendering. Explicitly configure spring.jpa.open-in-view to disable this warning
2026-11-01T09:56:27.198+01:00  INFO 13880 --- [TodoApp] [  restartedMain] o.s.boot.tomcat.TomcatWebServer          : Tomcat started on port 8080 (http) with context path '/'
2026-11-01T09:56:27.205+01:00  INFO 13880 --- [TodoApp] [  restartedMain] org.ldv.todoapp.TodoAppApplicationKt     : Started TodoAppApplicationKt in 7.806 seconds (process running for 9.177)
```

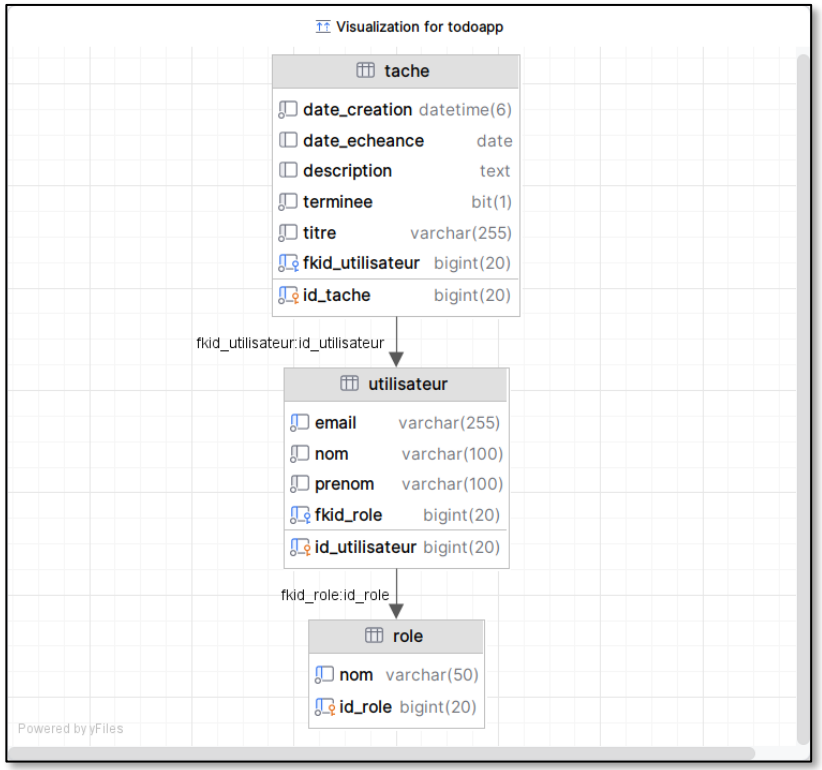
CLASSES & PERSISTENCE EN BD

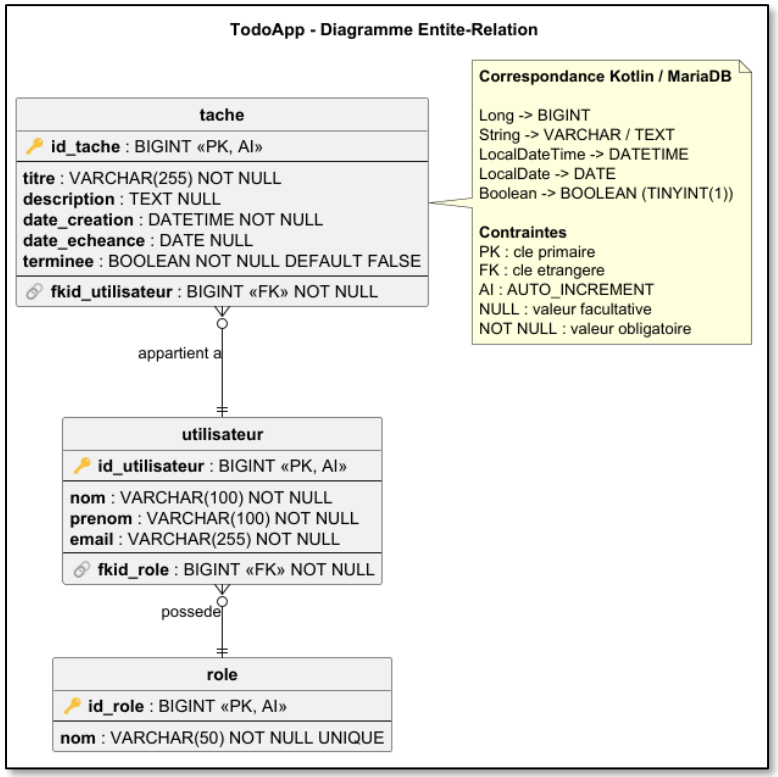
- **Test fonctionnel & validation :**
  - Pour vérifier la création du schéma relation dans MariaDB et pour faciliter la suite du développement, nous allons utiliser l'outil **database** d'IntelliJ en créant un **datasource** MariaDB.



CLASSES & PERSISTENCE EN BD

- **Test fonctionnel & validation :**
  - Une fois votre **datasource** connecté à votre **BD**, nous vous invitons à ouvrir le diagramme.
  - Nous pouvons le comparer à notre intention initiale avec le diagramme **E-R**.



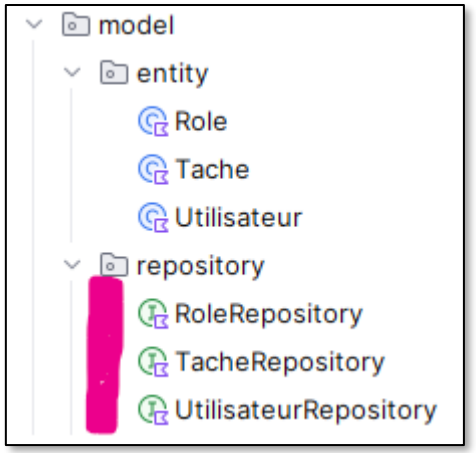


CLASSES & PERSISTENCE EN BD

- **Mise en place de la couche DAO/Repository :**
  - Nous avons mis en place le modèle de données en définissant nos classes entités.
  - Le schéma relationnel a été également été défini à la volée en utilisant et exploitant les annotations JPA-Hibernate : `@Entity`; `@Table`; `@Id`; etc.
  - Cependant nos classes entités de disposent pas de méthodes permettant de réaliser les opérations/requêtes de **C**réation, **L**ecture (**R**ead), d'**U**pdate et de **S**uppression (**D**elete).
  - C'est la couche **Respository** que l'on nomme également **DAO** pour (**D**ata **A**ccess **O**bject) qui assumera cette responsabilité à partir de classes qui permettrons d'instancier des objets dédiés à l'accès aux données des classes entités/métiers vers leurs tables en base de données.
  - **REMARQUE :** En Pratique avec **Spring Boot** et **JPA-Hibernate** nous utiliserons des **Interfaces** au lieu de classes, cela permettra à Spring Boot de récupérer ces interfaces pour implémenter elle-même les principales méthodes d'accès grâe à JPA-Hibernate !

CLASSES & PERSISTENCE EN BD

- **Mise en place de la couche DAO/Repository :**
  - **REGLE GENERALE :** Chaque classe de la couche entité est accompagnée d'une interface afin d'implémenter l'accès des données en base de données.
  - **A FAIRE :** Créer les 3 interfaces dans le package **`model/repository`** :
    - `RoleRepository.kt`
    - `TacheRepository.kt`
    - `UtilisateurRepository.kt`



CLASSES & PERSISTENCE EN BD

- **Mise en place de la couche DAO/Repository :**
  - Définition de minimale de l'interface **RoleRepository** :
    - Import des dépendances :
      - L'interface doit permettre de créer des instances d'objets de la classe **`Role`**, on va donc importer la classe **`Role`**.
      - On va importer pour chaque interface du repository la dépendance **`JpaRepository`** qui est une interface fournie par **JPA** et qui va permettre d'hériter de méthodes qui seront implémentées par **Hibernate**.
      - Ajouter les imports comme ci-dessous :

```
package org.ldv.todoapp.model.repository

import org.ldv.todoapp.model.entity.Role
import org.springframework.data.jpa.repository.JpaRepository

interface RoleRepository {
}
```

CLASSES & PERSISTENCE EN BD

- **Mise en place de la couche DAO/Repository :**
  - Définition de minimale de l'interface **RoleRepository** :
    - Définition du type d'objet retourné et du type de la clé primaire.
      - **JpaRepository** s'attend à ce qu'on définisse les 2 types, la syntaxe est la suivante :
        - **`JpaRespository<TypeObjetRetourne, TypeClePrimaire>`**
      - Appliqué l'interface **`RoleRepository`** cela donne : **`JpaRespository<Role, Long>`**
      - Il ne reste plus qu'à faire hériter l'interface **`RoleRespository`** de l'interface **`JpaRespository<Role, Long>`** :

```
package org.ldv.todoapp.model.repository

import org.ldv.todoapp.model.entity.Role
import org.springframework.data.jpa.repository.JpaRepository

interface RoleRepository : JpaRepository<Role, Long> {
}
```

CLASSES & PERSISTENCE EN BD

- **Mise en place de la couche DAO/Repository :**
  - Avec l'héritage de **`JpaRepository<TypeObj, TypePK>`** notre simple interface RoleRepository dispose maintenant de nombreuses méthodes d'accès ! Et cela sans écrire une seule requête SQL !
  - Voici les principales :
    - Méthodes d'accès en **L**ecture (**R**ead) :
      - **`findById(id)`** : Cherche et retourne un objet à partir de sa clé primaire.
      - **`findAll()`** : Retourne tous les objets de la table sous la forme d'une liste
      - **`findAllById(ids)`** : Retour dans une liste tous les objets dont les clés primaires étaient consignées dans la liste **`ids`**.
      - **`existsById(id)`** : Retourne un booléen indiquant la présence de l'entité avec la clé primaire **`id`**.
      - **`count()`** : Retourne le nombre total d'enregistrement dans la table.

CLASSES & PERSISTENCE EN BD

- **Mise en place de la couche DAO/Repository :**
  - Suite de la liste des principales méthodes d'accès :
    - Méthodes d'accès en de **C**réation et de **M**odification (**C**reate et **U**pdate) :
      - **`save(entity)`** : Sauvegarde l'objet, si la clé primaire de l'objet existe déjà, l'enregistrement est mis à jour sinon un nouvel enregistrement est rajouté dans la table. La méthode retourne l'objet crée ou modifié.
      - **`saveAll(entities)`** : Sauvegarde toutes les entités contenus la liste **`entities`**.

    - Méthode de **S**uppression (**D**elete) :
      - **`deleteById(id)`** : Supprime l'enregistre de l'entité possédant id comme clé primaire.
      - **`delete(entity)`** : Supprime l'enregistrement de l'objet **`entity`** passé en argument.
      - **`deleteAllById(ids)`** : Supprime tous enregistrements des entités dont la clé primaire est contenu dans la liste **`ids`**.
      - **`deleteAll(entities)`** : Supprimer toutes les entités contenues dans la liste **`entities`**.
      - **`deleteAll()`** : Supprimer tous les enregistements de la table !

CLASSES & PERSISTENCE EN BD

- **Mise en place de la couche DAO/Repository :**
  - Il nous reste maintenant plus qu'à réalise l'implémentation minimale des interfaces `UtilisateurRepository` et `TacheRepository` :

```
package org.ldv.todoapp.model.repository

import org.ldv.todoapp.model.entity.Utilisateur
import org.springframework.data.jpa.repository.JpaRepository

interface UtilisateurRepository : JpaRepository<Utilisateur, Long> {
}
```

```
package org.ldv.todoapp.model.repository

import org.ldv.todoapp.model.entity.Tache
import org.springframework.data.jpa.repository.JpaRepository

interface TacheRepository : JpaRepository<Tache, Long> {
}
```

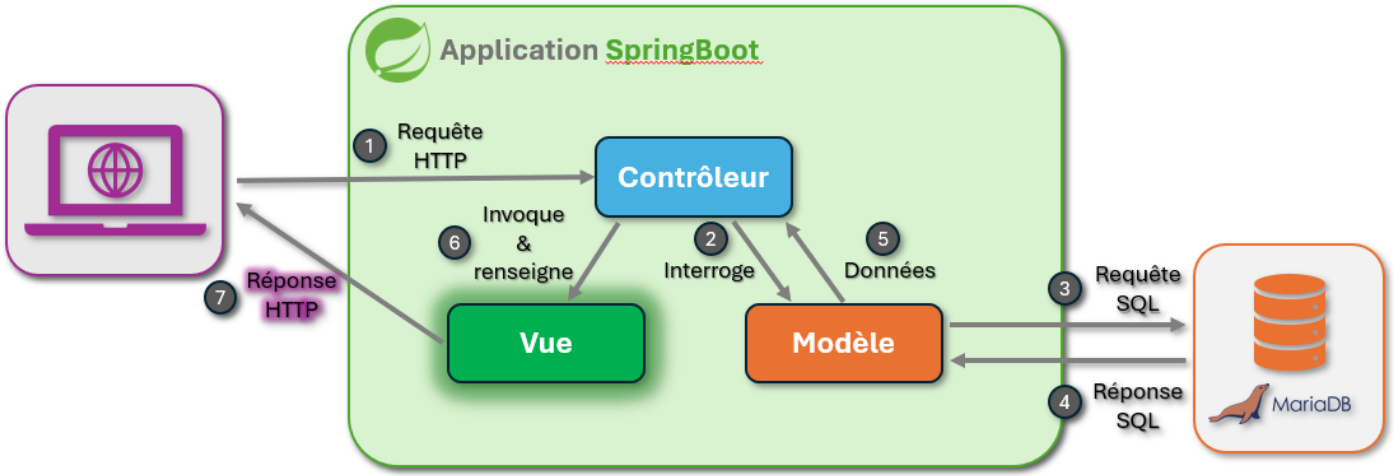
CLASSES & PERSISTENCE EN BD

- **Bilan du DAO/Repository :**
  - Nous avons maintenant à des accès méthode **CRUD** sur nos 3 entités : **Role**, **Utilisateur** et **Tache**.
  - Nous aurons cependant besoin de méthodes d'accès plus complexes pour récupérer par exemple l'ensemble des tâches associées à un utilisateur.
  - Mais nous verrons cela plus tard dans le TP.
  - Il est maintenant temps de s'occuper de la couche **Vue** de l'application avec un template complet.

MISE EN PLACE DU TEMPLATE

MISE EN PLACE DU TEMPLATE

- **Rappel sur la Vue du modèle MVC :**
  - La **Vue** aura pour mission d'élaborer une réponse à partir des données fournies par le contrôleur.
  - La réponse sera au standard **HTTP** afin de transmettre en général une page HTML.



MISE EN PLACE DU TEMPLATE

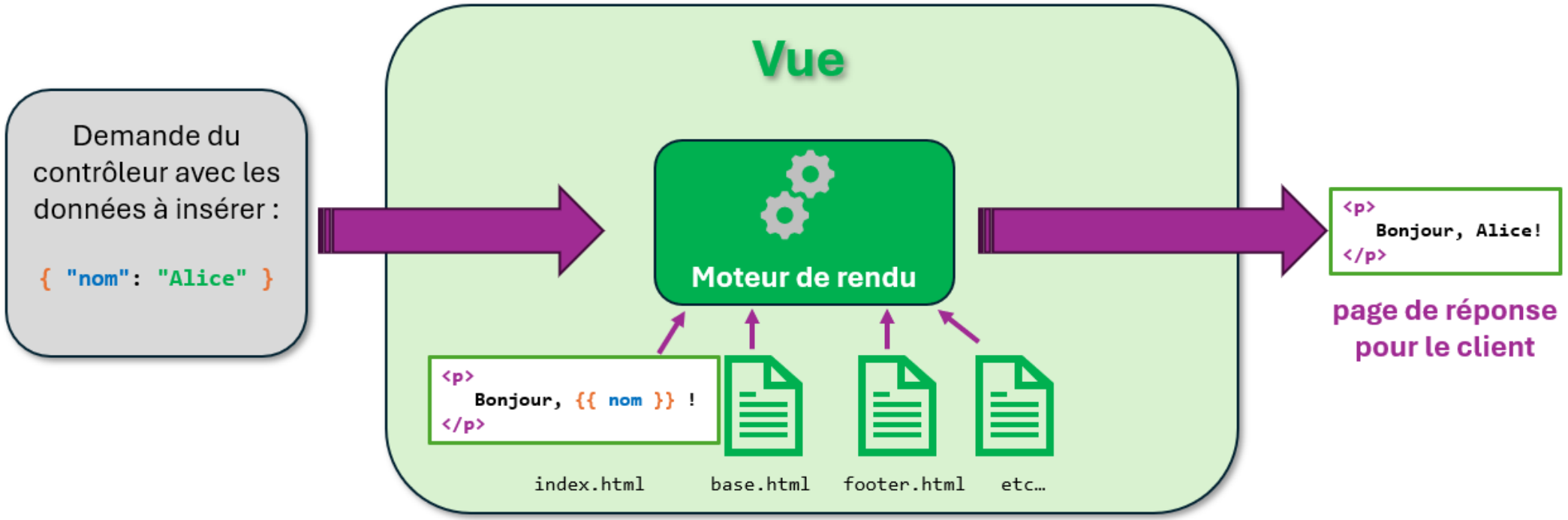
- **Le templating :**
  - Le templating dans notre contexte est la solution technique qui permettra de construire une page Web de réponse en reprenant une page **HTML** enrichie du langage de la solution de templating.
  - Le langage de templating permet l'insertion des données fournies par le **contrôleur**.
  - Le fichier de template est choisi par le contrôleur parmi l'ensemble des fichiers templates rédigés en amont par le développeur.

MISE EN PLACE DU TEMPLATE

- **Fonctionnement :**
  - Le moteur de template repose sur 3 éléments fondamentaux :
    - **Un modèle de vues (template) :**
      → Un ou plusieurs fichiers **HTML** contenant des zones dynamiques à remplacer (variables, boucles, conditions...).
    - **Un contexte (ou modèle de données) :**
      → Une structure (souvent un objet ou une map) contenant les données à injecter dans le template.
    - **Un moteur de rendu :**
      → Le moteur lit le template, remplace les variables par leurs valeurs, évalue les instructions conditionnelles et itératives, puis produit une page **HTML** finale.

MISE EN PLACE DU TEMPLATE

- **Fonctionnement :**
  - Le schéma ci-dessous illustre le processus avec les éléments de code utilisés par chaque partie (contrôleur; template et résultat)



MISE EN PLACE DU TEMPLATE

- **Factoriser et optimiser le code de la vue avec le template :**
  - L'un des avantages des moteurs de template est qu'il est possible de décomposer les fichiers du template en modules que l'on nomme **fragments**.
  - Ainsi des éléments se répétant comme typiquement le ***header*** et le ***footer*** sont placés dans des fichiers séparés pour constituer des **fragments** que nous pouvons inclure dans les autres fichiers du template.
  - Enfin la construction des fichiers dont `accueil.html` prendra le fichier `base.html` pour établir leurs structures.

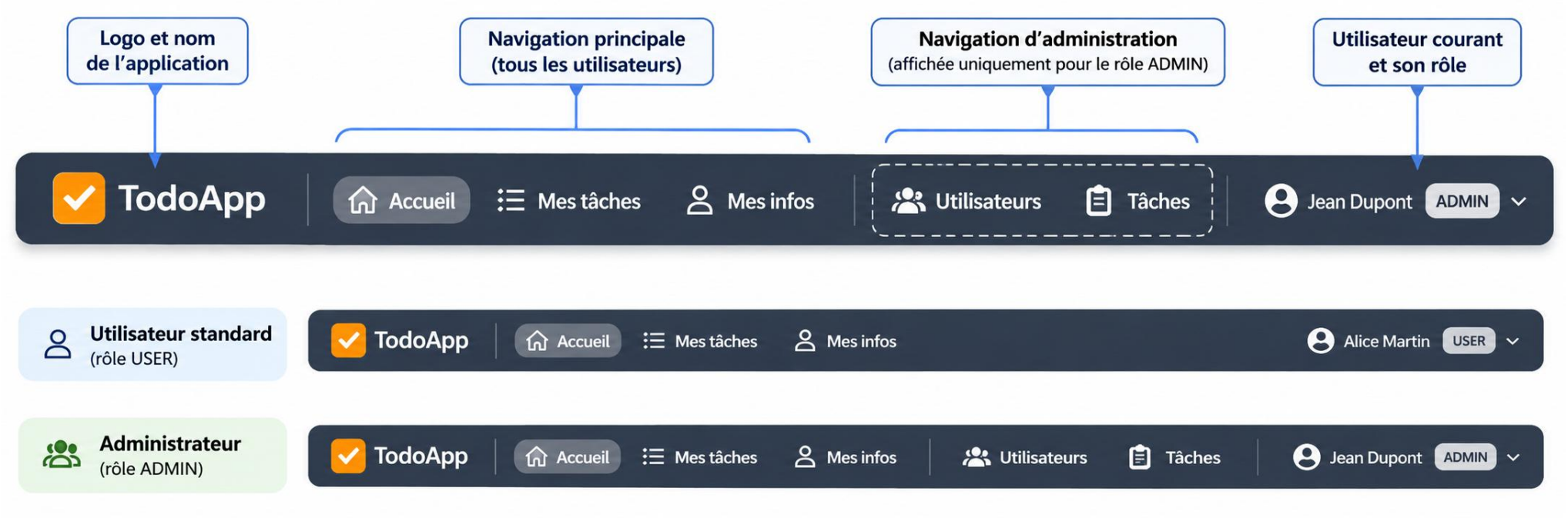
MISE EN PLACE DU TEMPLATE

- **Structure type :**
  - Il est conseillé de suivre l'arborescence suivante pour construire ses templates :

```
src/
└── main/
    └── resources/
        └── templates/
            ├── base.html
            ├── fragments/
            │   ├── header.html
            │   ├── navbar.html
            │   └── footer.html
```

MISE EN PLACE DU TEMPLATE

- **Navigation dans l'application :**
  - Notre application est pensée pour s'articuler autour de 5 pages :
    - Navigation principale : page ***Accueil***; page ***Mes tâches***; page ***Mes Infos***
    - Backoffice/administration : page ***Utilisateur***; page ***Tâches***
  - Le wireframe ci-dessous donne une idée d'une solution pour le header :



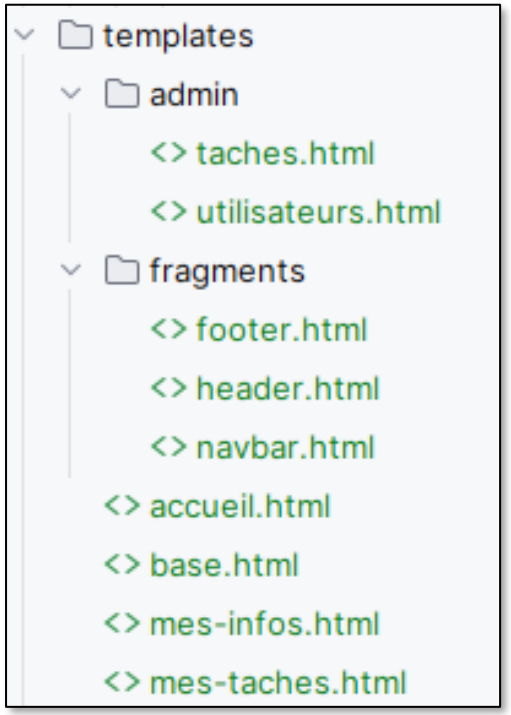
MISE EN PLACE DU TEMPLATE

- **Navigation dans l'application :**
  - Chaque page doit être routée par le **contrôleur**, pour cela on défini des **routes/endpoints** qui définiront la partie de droite des **URL/URI** de l'application.
  - Notre **vue** doit proposer un fichier **template** (html) pour chaque page.

Page	Routes sur routeur/contrôleur	Fichiers template (depuis templates/)	Visibilité
Accueil	`/`	`accueil.html`	Tous
Mes tâches	`/mes-taches`	`mes-taches.html`	Tous
Mes infos	`/mes-infos`	`mes-infos.html`	Tous
Utilisateurs	`/admin/utilisateurs`	`admin/utilisateurs.html`	Administrateur
Tâches	`/admin/taches`	`admin/taches.html`	Administrateur

MISE EN PLACE DU TEMPLATE

- **Structure du template**
  - Dans le dossier `templates` :
    - Rajouter les dossiers : `admin` et `fragments`
    - les fichiers : `accueil.html`; `base.html`; `mes-infos.html`; `mes-taches.html`
  - Dans le dossier `admin` rajouter :
    - `taches.html`
    - `utilisateurs.html`
  - Dans le dossier `fragments` rajouter :
    - `footer.html`
    - `header.html`
    - `navbar.html`



MISE EN PLACE DU TEMPLATE

- **Structure du template :**
  - Le fichier `base.html` sera utilisé par tous les fichiers du template hors fichiers fragments, soit :
    - `accueil.html`
    - `mes-taches.html`
    - `mes-infos.html`
    - `taches.html`
    - `utilisateurs.html`
  - Les 2 fragments `header.html` et `footer.html` seront directement utilisés par le fichier `base.html`
  - Le fichier `header.html` utilisera le fragment `navbar.html`.

MISE EN PLACE DU TEMPLATE

- **Construction du ficher `base.html`**

  - Le fichier **`base.html`** collecte le code commun à toutes les pages, typiquement le contenu des balises **`<head>…</head>`** ainsi que **`<header>…</header>`** et **`<footer>…</footer>`**

  - Il récupère le contenu à afficher pour une page donnée pour l'insérer à l'intérieur de la balise **`<main>…</main>`**.

  - Ouvrir le fichier **`base.html`**, puis saisir son squelette (voir page suivante pour le code)

MISE EN PLACE DU TEMPLATE

- **Construction du ficher `base.html`**
  - Code à copier dans le fichier **`base.html`** :

Explication du code à la page suivante

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org" lang="fr"
      th:fragment="base(content, titre, activePage)">
<head>

</head>

<body class="d-flex flex-column min-vh-100">

  <!-- header -->
  <header th:replace="~{fragments/header :: header(${activePage})}"></header>

  <!-- Contenu principal -->
  <main class="container flex-grow-1">
      <!-- Contenu spécifique des pages -->
      <div th:replace="${content}"></div>
  </main>

  <!-- Footer -->
  <footer th:replace="~{fragments/footer :: footer}"></footer>

</body>
</html>
```

MISE EN PLACE DU TEMPLATE

- **Description du code :**
  - Notre fichier est en fait un fragment L'attribut **`th:fragment`** permet de le définir comme fragment.
  - la valeur associée permettant de définir :
    - Son identifiant : **`base`**
    - Ses paramètres :
      - **`content`** : Récupère le contenu qui sera affiché.
      - **`titre`** : Récupère le titre qui sera affiché dans l'onglet du navigateur.
      - **`activePage`** : Récupère un string correspondant à la page affichée, ce qui permettra de mettre en surbrillance l'entrée dans la navbar.

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org" lang="fr"
      th:fragment="base(content, titre, activePage)">
<head>

</head>

<body class="d-flex flex-column min-vh-100">

  <!-- header -->
  <header th:replace="~{fragments/header :: header(${activePage})}"></header>

  <!-- Contenu principal -->
  <main class="container flex-grow-1">
      <!-- Contenu spécifique des pages -->
      <div th:replace="${content}"></div>
  </main>

  <!-- Footer -->
  <footer th:replace="~{fragments/footer :: footer}"></footer>

</body>
</html>
```

MISE EN PLACE DU TEMPLATE

- **Description du code :**
  - Nous pouvons repérer les 3 attributs : `th:replace` qui permettent de spécifier les points d'insertion du contenu.
  - Pour le **header** et le **footer** nous avons une syntaxe commune pour la valeur :

`~{chemin_fichier/fichier :: ident(arg1, arg2,…)}`

chemin du fichier html contenant le fragment — Identifiant — Argument(s) si existant(s)

  - Pour le contenu principal `${content}` est la valeur (ici du contenu **HTML**) qui est passée via le paramètre `content` avec `base(content,…)`.

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org" lang="fr"
      th:fragment="base(content, titre, activePage)">
<head>

</head>

<body class="d-flex flex-column min-vh-100">

  <!-- header -->
  <header th:replace="~{fragments/header :: header(${activePage})}"></header>

  <!-- Contenu principal -->
  <main class="container flex-grow-1">
      <!-- Contenu spécifique des pages -->
      <div th:replace="${content}"></div>
  </main>

  <!-- Footer -->
  <footer th:replace="~{fragments/footer :: footer}"></footer>

</body>
</html>
```

MISE EN PLACE DU TEMPLATE

- **Construction du ficher `base.html`**
  - Nous vous donnons ci-dessous le code à insérer dans la balise **`<head>…</head>`**.
  - C'est ici que nous allons définir l'emplacement de : *Bootstrap*; *Bootstrap Icons*; *police*; fichier de style *css* etc.

```
<head>
    <meta charset="UTF-8">
     <title th:text="${titre} ?: 'TodoApp'">TodoApp</title>

    <!-- Favicon -->
    <link rel="icon" type="image/x-icon" th:href="@{/images/favicon.ico}">

    <!-- Bootstrap + Bootstrap Icons -->
    <link rel="stylesheet" th:href="@{/vendor/bootstrap/css/bootstrap.min.css}">
    <link rel="stylesheet" th:href="@{/vendor/bootstrap-icons/bootstrap-icons.min.css}">

    <!-- Feuille de style de l'application -->
    <link rel="stylesheet" th:href="@{/css/style.css}">
</head>
```

MISE EN PLACE DU TEMPLATE

- **Description du code :**
  - L'attribut **`th:text`** qui permet d'insérer du texte qui ici sera la valeur de la variable **`titre`**. Si titre n'a pas de valeur alors c'est la valeur par défaut **`'TodoApp'`** qui sera affichée.

```
<head>
    <meta charset="UTF-8">
     <title th:text="${titre} ?: 'TodoApp'">TodoApp</title>

    <!-- Favicon -->
    <link rel="icon" type="image/x-icon" th:href="@{/images/favicon.ico}">

    <!-- Bootstrap + Bootstrap Icons -->
    <link rel="stylesheet" th:href="@{/vendor/bootstrap/css/bootstrap.min.css}">
    <link rel="stylesheet" th:href="@{/vendor/bootstrap-icons/bootstrap-icons.min.css}">

    <!-- Feuille de style de l'application -->
    <link rel="stylesheet" th:href="@{/css/style.css}">
</head>
```

MISE EN PLACE DU TEMPLATE

- **Description du code :**
  - L'attribut **`th:ref`** permet de définir les chemins de nos éléments statiques.

```
<head>
    <meta charset="UTF-8">
     <title th:text="${titre} ?: 'TodoApp'">TodoApp</title>

    <!-- Favicon -->
    <link rel="icon" type="image/x-icon" th:href="@{/images/favicon.ico}">

    <!-- Bootstrap + Bootstrap Icons -->
    <link rel="stylesheet" th:href="@{/vendor/bootstrap/css/bootstrap.min.css}">
    <link rel="stylesheet" th:href="@{/vendor/bootstrap-icons/bootstrap-icons.min.css}">

    <!-- Feuille de style de l'application -->
    <link rel="stylesheet" th:href="@{/css/style.css}">
</head>
```

MISE EN PLACE DU TEMPLATE

- **Construction du ficher `base.html`**
  - Il nous reste à déclarer en bas du fichier la localisation du fichier **`bootstrap.bundle.min.js`**.
  - Pour cela on ajout une balise **`<script>…</script>`** juste avant la balise fermante **`</body>`**.

```
<!-- JS Bootstrap -->
<script th:src="@{/vendor/bootstrap/js/bootstrap.bundle.min.js}"></script>
</body>
</html>
```

MISE EN PLACE DU TEMPLATE

- **Construction du ficher `footer.html`**
  - Ouvrir le fichier **`footer.html`** et copier le code ci-dessous :

```
<div xmlns:th="http://www.thymeleaf.org">
<footer th:fragment="footer"
        class="bg-light border-top py-3 mt-auto">
    <div class="container text-center">
        <p class="mb-0 text-muted small">
            &copy; 2026 TodoApp - Application pédagogique BTS SIO SLAM
        </p>
    </div>
</footer>
</div>
```

  - **Remarque :**
    - `<div xmlns:th="http://www.thymeleaf.org">` a pour unique utilité d'activer l'autocomplétion d'IntelliJ pour Thymeleaf.

MISE EN PLACE DU TEMPLATE

- **Construction du ficher `header.html`**
  - Ouvrir le fichier **`header.html`** et copier le code ci-dessous :

```
<div xmlns:th="http://www.thymeleaf.org">
<header th:fragment="header(activePage)"
        class="bg-light border-bottom">
    <div class="container d-flex align-items-center py-2">
        <!-- Logo de l'application -->
        <a th:href="@{/}" class="me-4">
            <img th:src="@{/images/favicon.ico}"
                 alt="TodoApp"
                 width="40"
                 height="40">
        </a>
        <!-- Barre de navigation -->
        <div th:replace="~{fragments/navbar :: navbar(${activePage})}"></div>
    </div>
</header>
</div>
```

MISE EN PLACE DU TEMPLATE

- **Construction du ficher `header.html`**
  - Ouvrir le fichier `navbar.html` et copier le code ci-dessous :

```
<div xmlns:th="http://www.thymeleaf.org">
<nav th:fragment="navbar(activePage)" class="navbar navbar-expand-lg p-0">
    <div class="container-fluid p-0">

        <!-- Bouton de navigation pour les petits écrans -->
        <button class="navbar-toggler" type="button"
                data-bs-toggle="collapse"
                data-bs-target="#navbarTodoApp"
                aria-controls="navbarTodoApp"
                aria-expanded="false"
                aria-label="Afficher la navigation">
            <span class="navbar-toggler-icon"></span>
        </button>

        <!-- Liens de navigation -->
        <div class="collapse navbar-collapse" id="navbarTodoApp">
            <ul class="navbar-nav">

                <!-- Accueil -->
                <li class="nav-item">
                    <a class="nav-link"
                       th:href="@{/}"
                       th:classappend="${activePage} == 'accueil' ? ' active' : ''">
                        <i class="bi bi-house-door"></i>
                        Accueil
                    </a>
                </li>

                <!-- Mes tâches -->
                <li class="nav-item">
                    <a class="nav-link"
                       th:href="@{/mes-taches}"
                       th:classappend="${activePage} == 'mes-taches' ? ' active' : ''">
                        <i class="bi bi-list-check"></i>
                        Mes tâches
                    </a>
                </li>

                <!-- Mes informations -->
                <li class="nav-item">
                    <a class="nav-link"
                       th:href="@{/mes-infos}"
                       th:classappend="${activePage} == 'mes-infos' ? ' active' : ''">
                        <i class="bi bi-person-circle"></i>
                        Mes infos
                    </a>
                </li>

                <!-- Groupe Administration -->
                <li class="nav-item dropdown">
                    <a class="nav-link dropdown-toggle"
                       href="#"
                       th:classappend="${activePage != null and activePage.startsWith('admin/')} ? 'active' : ''"
                       id="navbarAdmin"
                       role="button"
                       data-bs-toggle="dropdown"
                       aria-expanded="false">
                        <i class="bi bi-gear"></i>
                        Administration
                    </a>
                    <!-- Menu déroulant -->
                    <ul class="dropdown-menu"
                        aria-labelledby="navbarAdmin">
                        <li> <!-- Gestion des tâches -->
                            <a class="dropdown-item"
                               th:href="@{/admin/taches}"
                               th:classappend="${activePage} == 'admin/taches' ? ' active' : ''">
                                <i class="bi bi-list-task"></i>
                                Gestion des tâches
                            </a>
                        </li>
                        <li> <!-- Gestion des utilisateurs -->
                            <a class="dropdown-item"
                               th:href="@{/admin/utilisateurs}"
                               th:classappend="${activePage} == 'admin/utilisateurs' ? ' active' : ''">
                                <i class="bi bi-people"></i>
                                Gestion des utilisateurs
                            </a>
                        </li>
                    </ul>
                </li>
            </ul>
        </div>
    </div>
</nav>
</div>
```

MISE EN PLACE DU TEMPLATE

- **Construction du ficher `header.html`**
  - Ouvrir le fichier **`accueil.html`** et copier le code ci-dessous :

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org"
      th:replace="~{base :: base(~{::content}, 'TodoApp-Accueil', 'accueil')}">

<div th:fragment="content">
    <h1>Bienvenue sur TodoApp</h1>
    <p>Application de gestion des tâches.</p>
</div>

</html>
```

MISE EN PLACE DU TEMPLATE

- **Bilan provisoire :**

  - Nous avons traité les principaux fichiers du template.
  - Les fichiers restants seront complétés après avoir vérifier que le contenu de la page d'accueil s'affiche bien.
  - Afin de procéder à cet essai sur la page d'accueil, nous allons commencer à construire le contrôleur.

MISE EN PLACE DU CONTRÔLEUR

MISE EN PLACE DU CONTRÔLEUR

- **Rôle du contrôleur :**

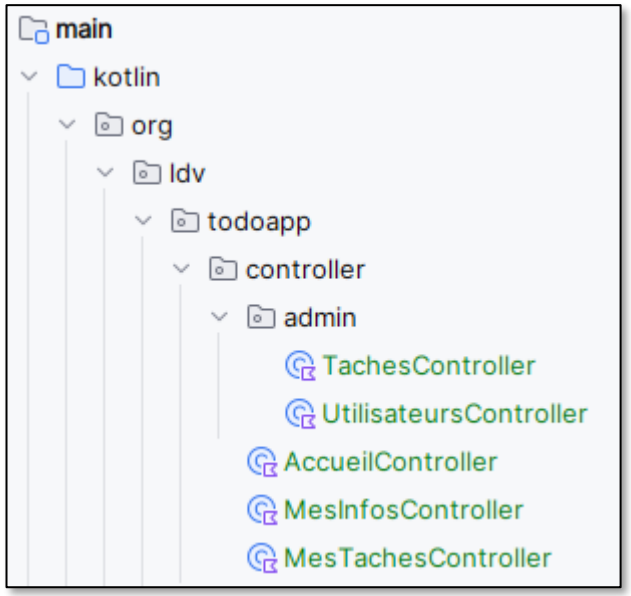
  - Pour accéder à nos pages nous avons besoin de définir l'ensemble des routes d'accès avec le contrôleur de l'application.

  - Afin de gagner en clarté, nous allons créer une classe contrôleur pour chacune des 5 pages de notre application **TodoApp**. Ce qui correspondra à 5 fichiers :
    - `AccueilController.kt`
    - `MesInfosController.kt`
    - `MesTachesController.kt`
    - `TachesController.kt`
    - `UtilisateursController.kt`

MISE EN PLACE DU CONTRÔLEUR

- **Création des fichiers :**

  - Créer un package **`admin`** dans le package **`controller`**.
  - Créer les 5 fichiers de classe **Kotlin** (**`.kt`**) en respectant l'arborescence ci-dessous :



MISE EN PLACE DU CONTRÔLEUR

- **Contenu du fichier `AccueilController.kt`**

  - Ouvrir le fichier **`AccueilController.kt`** et copier le code fourni :

```
package org.ldv.todoapp.controller

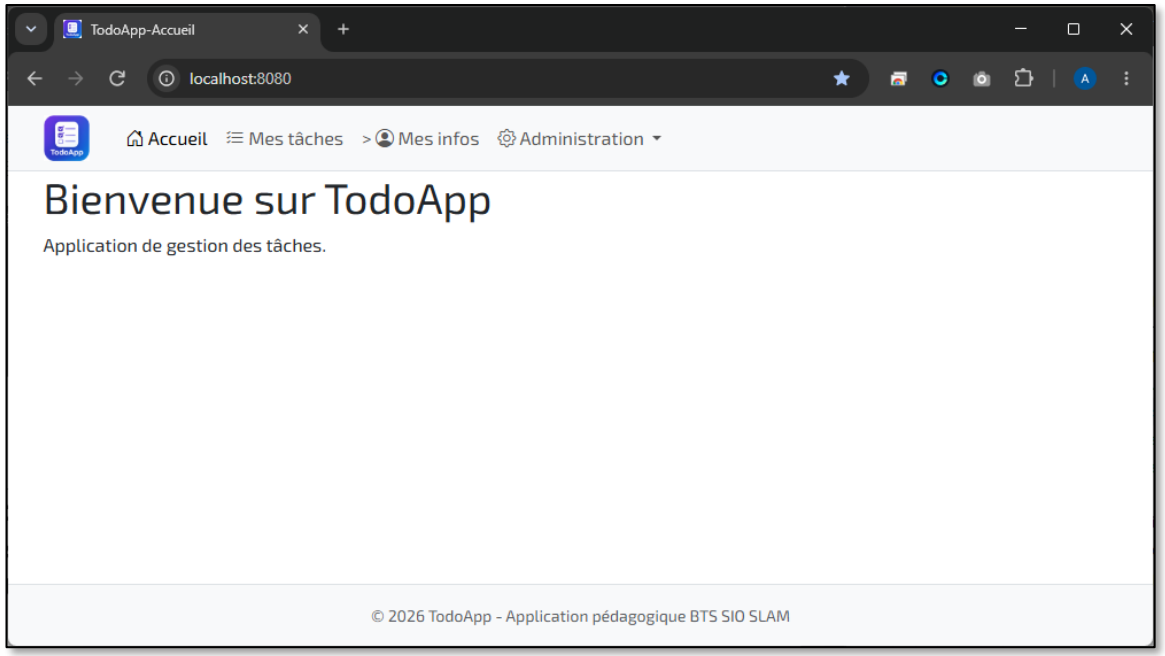
import org.springframework.stereotype.Controller
import org.springframework.web.bind.annotation.GetMapping

@Controller
class AccueilController {

     @GetMapping("/")
     fun accueil(): String {
        return "accueil"
     }
}
```

MISE EN PLACE DU CONTRÔLEUR

- **1er test fonctionnel :**
  - Notre application est maintenant prête pour afficher la page d'accueil.
  - Exécuter le projet et ouvrir tester l'URL/URI : http://localhost:8080/
  - Vous devriez obtenir l'affichage suivant :



TRAVAIL À FAIRE

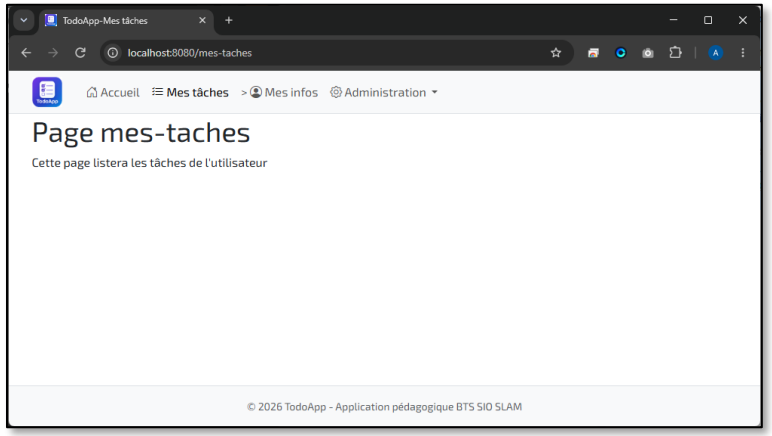
TRAVAIL À FAIRE

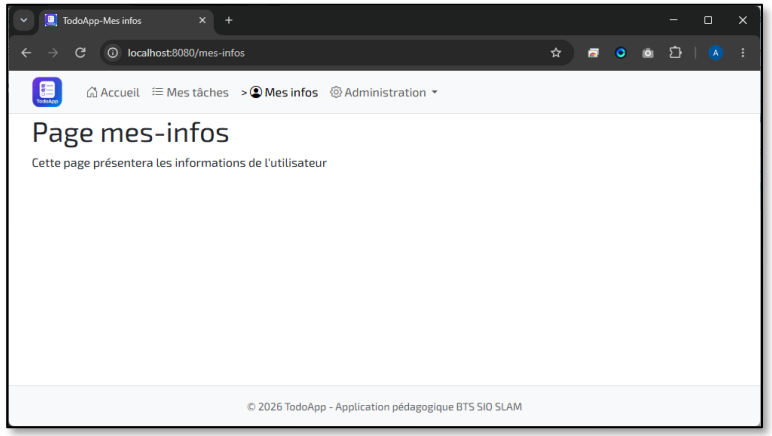
- **On vous demande de compléter les fichiers et templates :**
  - `MesInfosController.kt` et le template `mes-infos.html`
  - `MesTachesController` et le template `mes-taches.html`
  - `TachesController` et le template `admin/taches.html`
  - `UtilisateurController` et le template `admin/utilisateur.html`

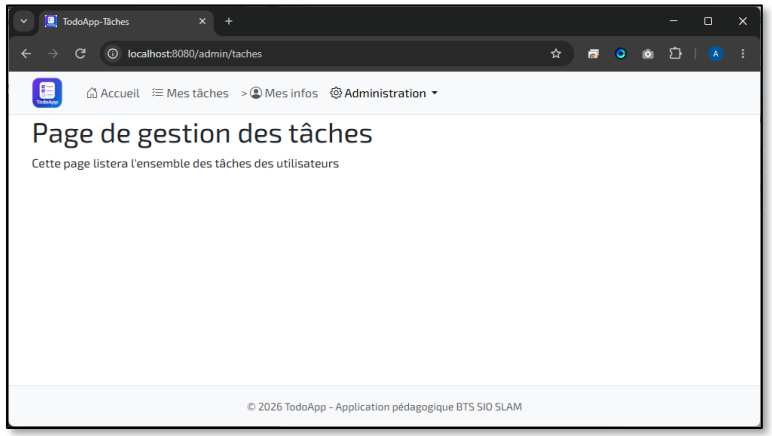
Consulter la page suivante pour voir le rendu attendu ->>

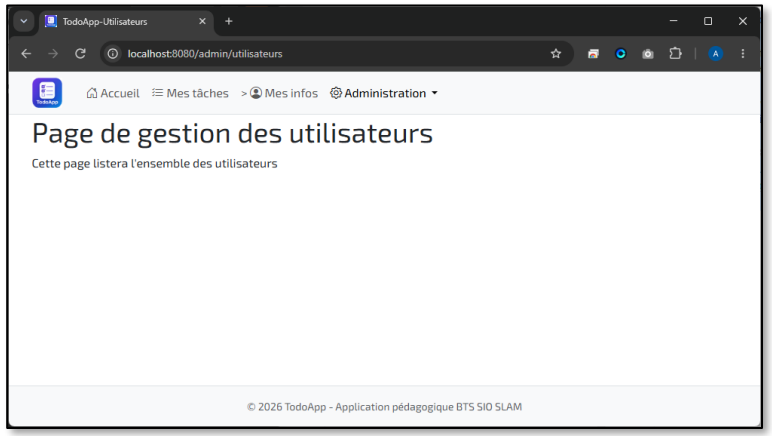
TRAVAIL À FAIRE

- **Une fois le travail terminé, vous devriez obtenir les rendus suivants :**









AJOUT DE MÉTHODES AU REPOSITORY

AJOUT DE MÉTHODES AU REPOSITORY

- **Pourquoi ajouter de nouvelles méthodes aux repository ?**
  - Nous disposons actuellement des méthodes standards de base livrées par **JPA/Hibernate** : **`findById(id)`**; **`findAll()`**; **`deleteById(id)`**; **`save(entity)`**; etc.
  - Cependant **`UtilisateurRepository`** devra couvrir également les méthodes suivantes :
    - Trouver un utilisateur à partir de son email.
    - Récupérer un utilisateur par sa clé primaire (id) avec son rôle : En effet par défaut la méthode **`findById(id)`** retournera un utilisateur potentiellement sans son **Role**. Et la méthode lève une exception si id n'existe pas dans la base de données.
    - Récupérer tous les utilisateurs avec leur rôle.

AJOUT DE MÉTHODES AU REPOSITORY

- **Pourquoi ajouter de nouvelles méthodes aux repository ?**
  - Nous disposons actuellement des méthodes standards de base livrées par **JPA/Hibernate** : **`findById(id)`**; **`findAll()`**; **`deleteById(id)`**; **`save(entity)`**; etc.
  - Cependant **`UtilisateurRepository`** devra couvrir également les méthodes suivantes :
    - Trouver un utilisateur à partir de son email.
    - Récupérer un utilisateur par sa clé primaire (id) avec son rôle : En effet par défaut la méthode **`findById(id)`** retournera un utilisateur potentiellement sans son **Role**. Et la méthode lève une exception si id n'existe pas dans la base de données.
    - Récupérer tous les utilisateurs avec leur rôle.
  - **A FAIRE :** Ouvrir le fichier : **`UtilisateurRepository.kt`**

```
1   package org.ldv.todoapp.model.repository
2
3   import org.ldv.todoapp.model.entity.Utilisateur
4   import org.springframework.data.jpa.repository.JpaRepository
5
6   interface UtilisateurRepository : JpaRepository<Utilisateur, Long> {
7   }
```

AJOUT DE MÉTHODES AU REPOSITORY

- **Ajout d'une méthode `findByEmail(email)` :**
  - Il suffit tout simplement d'écrire comme ci-dessous l'entête de la méthode, aucune implémentation n'est nécessaire pour le corps de la méthode !

```
interface UtilisateurRepository : JpaRepository<Utilisateur, Long> {
    fun findByEmail(email: String): Utilisateur?
}
```

  - En fait **Spring Boot** et **JPA-Hibernate** appliquent un mécanisme appelé : *Query Creation from Method Names* (Création de Requête à partir de l'Identifiant de la Méthode).

AJOUT DE MÉTHODES AU REPOSITORY

- **Quelques règles pour le *Query Creation from Method Names* :**
  - **1/ Structure syntaxique globale :**
    - Le nom d'une méthode se compose de 2 parties principales séparées par le mot-clé **By** :
    - **Exemple : `find`** *(Verbe)* + **`Utilisateur`** *(Optionnel)* + **By** + **`Email`** *(Critère)*.

  - **2. Les préfixes autorisés (**Les Verbes**) :**
    - Pour démarrer la méthode, vous devez utiliser l'un des verbes reconnus par **Spring Boot** :
      - **Lecture** : `find`...`By`, `read`...`By`, `get`...`By`, `query`...`By`, `stream`...`By`.
      - **Compte** : `count`...`By` (retourne un Long).
      - **Existence** : `exists`...`By` (retourne un Boolean).
      - **Suppression** : `delete`...`By` ou `remove`...`By`.

  - **3. Les règles syntaxiques pour les critères (**après le **By) :**
    - Le nom après le **By** doit correspondre exactement au nom de la propriété définie dans la classe.

  - **4. Les opérateurs logiques (**Combinaisons**) :**
    - Vous pouvez combiner plusieurs critères en utilisant les mots-clés **And** et **Or**
    - **Exemple : `findUtilisateurByNomAndPrenom(nom: String, prenom: String)`**

AJOUT DE MÉTHODES AU REPOSITORY

- **Ajout d'une méthode `findAllAvecRole()` :**
  - Nous voulons récupérer tous les utilisateurs avec leur rôle. Dans ce cas nous allons devoir spécifier la requête associée avec le langage **JQPL** (*Java **P**ersistance **Q**uerry **L**anguage*).
  - C'est un langage proche du SQL mais qui pourra justement être converti en SQL pure ou dans un autre langage de requête selon la SGBD utilisé.
  - **A FAIRE :** Rajouter les 2 imports pour **`Query`** et **`Param`** :

```
import org.ldv.todoapp.model.entity.Utilisateur
import org.springframework.data.jpa.repository.JpaRepository
import org.springframework.data.jpa.repository.Query
import org.springframework.data.repository.query.Param
```

AJOUT DE MÉTHODES AU REPOSITORY

- **Ajout d'une méthode `findAllAvecRole()` :**
  - **A FAIRE :** Rajouter la méthode suivante à **`UtilisateurRepository`** :

```
interface UtilisateurRepository : JpaRepository<Utilisateur, Long> {
    fun findByEmail(email: String): Utilisateur?

    @Query("""
        SELECT u
        FROM Utilisateur u
        JOIN FETCH u.role
        ORDER BY u.nom, u.prenom
        """)
    fun findAllAvecRole(): List<Utilisateur>
}
```

AJOUT DE MÉTHODES AU REPOSITORY

- **Ajout d'une méthode `findByIdAvecRole()` :**
  - **A FAIRE :** Rajouter la méthode suivante à **`UtilisateurRepository`** :

```
interface UtilisateurRepository : JpaRepository<Utilisateur, Long> {
   //…

   @Query("""
      SELECT u
      FROM Utilisateur u
      JOIN FETCH u.role
      WHERE u.idUtilisateur = :id
       """)
   fun findByIdAvecRole(@Param("id") id: Long): Utilisateur?
}
```

AJOUT DE MÉTHODES AU REPOSITORY

- **Ajout des méthodes pour TacheRepository :**
  - Nous aurons besoin de retrouver une tâche avec son utilisateur et même tous les utilisateurs avec leurs tâches pour le backoffice de gestion des tâches.
  - **A FAIRE :** Compléter les méthodes manquantes à **`TacheRepository`** :

```
package org.ldv.todoapp.model.repository

import org.ldv.todoapp.model.entity.Tache
import org.springframework.data.jpa.repository.JpaRepository
import org.springframework.data.jpa.repository.Query

 interface TacheRepository : JpaRepository<Tache, Long> {

   fun findByUtilisateur_IdUtilisateur(idUtilisateur: Long): List<Tache>

   @Query("""
      SELECT t
      FROM Tache t
      JOIN FETCH t.utilisateur
      ORDER BY t.dateCreation DESC
      """)
   fun findAllAvecUtilisateur(): List<Tache>
}
```

AJOUT DE MÉTHODES AU REPOSITORY

- **Ajout des méthodes pour RoleRepository :**
  - Ici nous avons juste besoin de rajouter une méthode **`findByNom(nom: String)`**
  - **A FAIRE :** Compléter **`RoleRepository`** :

```
package org.ldv.todoapp.model.repository

import org.ldv.todoapp.model.entity.Role
import org.springframework.data.jpa.repository.JpaRepository

interface RoleRepository : JpaRepository<Role, Long> {
   fun findByNom(nom: String): Role?
}
```

EXPLOITATION DU MODÈLE DE DONNÉES

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Bilans des acquis et tâches à venir :**
  - Nous avons mis en place toute la structure pour accueillir la présentation des données :
    - La structure complète du templating
    - Les contrôleurs d'accès aux pages
  - Pour la suite de ce TP nous allons exploiter des données déjà inscrites dans la base de données et les présenter dans les différentes vues.
  - Cependant durant ce TP nous ne mettrons pas en place les formulaires et méthodes **HTTP POST** permettant de modifier ou d'ajouter des données. Cela fera l'objet spécifique d'un second TP.

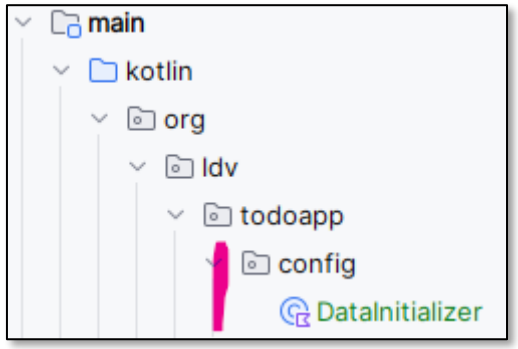
EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Ajout de données pour les tests fonctionnels :**
  - **Problématique :**
    - Notre base de données est actuellement vide. Nous devons pourtant valoriser nos tables pour pouvoir tester le rendu des vues et le bon fonctionnement des contrôleurs.
  - **Solutions :**
    - Rajouter les données avec des requêtes **SQL** placées dans le ficher `application.properties`. C'est une solution satisfaisante mais contraignante car dans ce cas nous devons rédiger des requêtes sur une ligne unique.
    - Fournir une classe appelée **`DataInitializer`** qui va instancier des objets des classes persistées de notre application : **Role**; **Utilisateur** et **Tache** et les sauvegarder dans la base de données avec la méthode **`saveAll()`** fournie par **JPA-Hibernate**. Cette méthode permet de rajouter de la logique si nécessaire pour conditionner la création des données de test, comme par exemple ne pas rajouter les nouvelles données si elle existe déjà.

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Création du DataInitializer :**
  - Nous allons utiliser un **DataInitializer** pour créer :
    - 2 objets **Role** : *"ROLE_USER"* et *"ROLE_ADMIN"*
    - 4 objets **Utilisateur** : *Alice Martin*; *Bob Durand*; *Claire Bernard*; *TodoApp Admin*
    - 8 objets **Taches** avec une répartition de 2 tâches par utilisateur.

  - Nous allons placer notre classe **`DataInitializer`** dans un package **`config`**.
    - A faire : Créer le package **`config`** et le fichier de classe **`DataInitializer`**.



EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Création du DataInitializer :**
  - Avant de commencer à implémenter notre **DataInitializer**, nous allons analyser sa structure et fonctionnement.
  - Notre classe héritera de l'interface **`CommandLineRunner`**. L'interface **`CommandeLineRunner`** est une interface proposée par **Spring Boot** et qui permet d'exécuter automatiquement du code à chaque démarrage de l'application.
  - Toute classe implémentant cette interface doit redéfinir la méthode **`run()`**, méthode qui sera justement appelée à chaque démarrage de l'application.
  - La classe sera précédée par l'annotation `@Component` qui indiquera que c'est **Spring Boot** qui aura pour charge d'instancier un objet **`DataInitializer`** !

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Création du DataInitializer :**
  - Voici le code de base illustré

Fera instancier la classe par **Spring Boot**

Injection des dépendances par le constructeur. **Spring** fournira lui-même les objets du **repository**

Héritage & implémentation de l'interface **CommandLineRunner**

Redéfinition de la méthode **run** qui sera automatiquement appelée par **Spring**

```
@Component  new *
open class DataInitializer(
    private val roleRepository: RoleRepository,
    private val utilisateurRepository: UtilisateurRepository,
    private val tacheRepository: TacheRepository
) : CommandLineRunner {

    override fun run(vararg args: String) {  new *
    // TODO: Implémenter la logique d'instance des objets...
    }
}
```

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Création du DataInitializer :**
  - Nous allons implémenter progressivement notre classe.
    Pour commencer, copier/coller le squelette de la classe dans le fichier `DataInitializer.kt`

```kotlin
package org.ldv.todoapp.config

import org.ldv.todoapp.model.entity.Role
import org.ldv.todoapp.model.entity.Utilisateur
import org.ldv.todoapp.model.entity.Tache
import org.ldv.todoapp.model.repository.RoleRepository
import org.ldv.todoapp.model.repository.UtilisateurRepository
import org.ldv.todoapp.model.repository.TacheRepository
import org.springframework.boot.CommandLineRunner
import org.springframework.stereotype.Component
import java.time.LocalDate

@Component
class DataInitializer(
    private val roleRepository: RoleRepository,
    private val utilisateurRepository: UtilisateurRepository,
    private val tacheRepository: TacheRepository
) : CommandLineRunner {

    override fun run(vararg args: String) {
    // TODO: Implémenter la logique d'instance des objets...
    }
}
```

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Création du DataInitializer :**
  - Nous allons maintenant créer une structure conditionnelle qui interrompt la méthode **`run()`** si notre base de données contient déjà des données.
  - **A FAIRE :** Rajouter le code suivant à la méthode **`run()`**

```
override fun run(vararg args: String) {

    // Initialisation uniquement si les tables sont vides
    if (roleRepository.count() > 0 ||
        utilisateurRepository.count() > 0 ||
        tacheRepository.count() > 0) {
        println("Base de données déjà initialisée.")
        return
    }
```

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Création du DataInitializer :**
  - On poursuit avec la création des 2 rôles **"ROLE_USER"** et **"ROLE_ADMIN"**.
  - Pour ce faire on utilise l'objet **roleRepository** et la méthode **save()** à laquelle on donne un objet **Role** instancié à la volée.
  - Rajouter le code suivant à la suite du précédent :

```
println("Initialisation des données TodoApp...")

// 1. Création des rôles
val roleUser = roleRepository.save(
    Role(nom = "ROLE_USER")
)

val roleAdmin = roleRepository.save(
    Role(nom = "ROLE_ADMIN")
)
```

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Création du DataInitializer :**
  - On poursuit la même logique pour créer les utilisateurs.
  - Rajoute toujours à la suite le code ci-dessous qui ajoute l'utilisateur **`alice`**

```
// 2. Création des utilisateurs
val alice = utilisateurRepository.save(
    Utilisateur(
        nom = "Martin",
        prenom = "Alice",
        email = "alice@example.com",
        role = roleUser
    )
)
```

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Création du DataInitializer :**
  - On poursuit avec l'utilisateur **bob** :

```
val bob = utilisateurRepository.save(
    Utilisateur(
        nom = "Durand",
        prenom = "Bob",
        email = "bob@example.com",
        role = roleUser
    )
)
```

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Création du DataInitializer :**
  - Puis l'utilisateur **`claire`** :

```
val claire = utilisateurRepository.save(
    Utilisateur(
        nom = "Bernard",
        prenom = "Claire",
        email = "claire@example.com",
        role = roleUser
    )
)
```

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Création du DataInitializer :**
  - Et enfin l'administrateur **`admin`** :

```
val admin = utilisateurRepository.save(
    Utilisateur(
        nom = "Admin",
        prenom = "TodoApp",
        email = "admin@example.com",
        role = roleAdmin
    )
)
```

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Création du DataInitializer :**
  - Passons à la création des tâches.
  - Ici on va instancier tous les objets dans une liste taches, puis nous utilisons la méthode **`saveAll()`** de l'objet **`tacheRepository`** pour tous les sauvegarder !
  - Copier/coller la structure à la suite de la création des utilisateurs. Nous allons ensuite rajouter les différentes tâches dans la liste taches.

```
// 3. Création des tâches
val aujourdHui = LocalDate.now()

val taches = listOf(
    // Prochaine étape, on rajoute les objets Tache.
)

tacheRepository.saveAll(taches)
```

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Création du DataInitializer :**
  - Voici le code à rajouter dans la liste taches pour les tâches d'**Alice** :

```
Tache(
    titre = "Préparer le TP Spring Boot",
    description = "Rédiger le support de cours MVC.",
    dateEcheance = aujourdHui.plusDays(7),
    utilisateur = alice
),
Tache(
    titre = "Installer IntelliJ IDEA",
    description = "Configurer l'environnement Kotlin.",
    dateEcheance = aujourdHui.minusDays(2),
    terminee = true,
    utilisateur = alice
),
```

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Création du DataInitializer :**
  - Puis à la suite, rajouter le code des tâches pour **Bob** :

```
Tache(
    titre = "Réviser les requêtes SQL",
    dateEcheance = aujourdHui.plusDays(3),
    utilisateur = bob
),
Tache(
    titre = "Terminer le projet JDBC",
    description = "Finaliser les méthodes DAO.",
    dateEcheance = aujourdHui.minusDays(5),
    utilisateur = bob
),
```

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Création du DataInitializer :**
  - Puis à la suite, rajouter le code des tâches pour **Claire** :

```
Tache(
    titre = "Étudier les annotations JPA",
    dateEcheance = aujourdHui.plusDays(10),
    utilisateur = claire
),
Tache(
    titre = "Lire la documentation Thymeleaf",
    terminee = true,
    utilisateur = claire
),
```

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Création du DataInitializer :**
  - Et le code des tâches pour **admin** :

```
Tache(
    titre = "Vérifier les comptes utilisateurs",
    dateEcheance = aujourdHui.plusDays(1),
    utilisateur = admin
),
Tache(
    titre = "Contrôler les tâches enregistrées",
    utilisateur = admin
)
```

EXPLOITATIONS DU MODÈLE DE DONNÉES

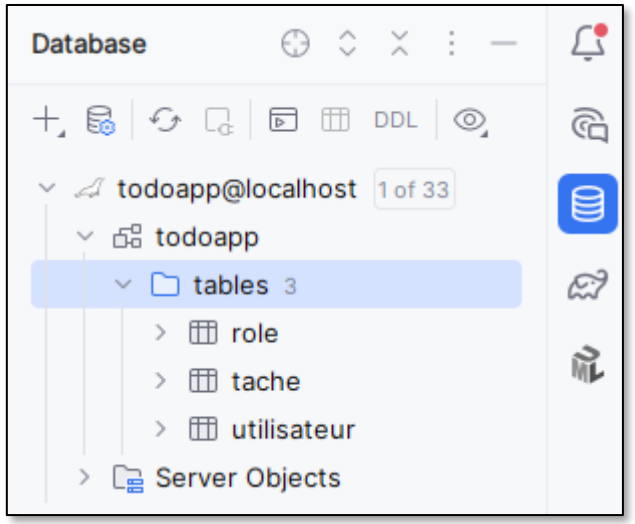
- **Création du DataInitializer :**
  - Pour finir après la ligne **`tacheRepository.saveAll(taches)`**, nous vous invitons à rajouter les 4 **`println()`** qui permettrons d'avoir un log dans la console à l'issue du processus.

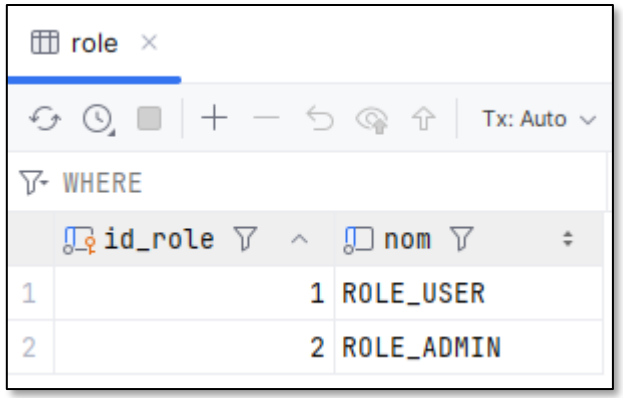
```
tacheRepository.saveAll(taches)

println("Initialisation terminée :")
println("- 2 rôles")
println("- 4 utilisateurs")
println("- ${taches.size} tâches")
```

EXPLOITATIONS DU MODÈLE DE DONNÉES

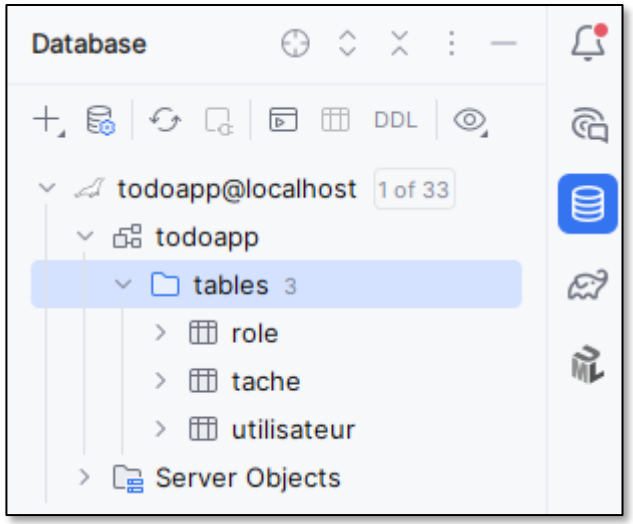
- **Test du DataInitializer :**
  - Nous allons maintenant voir si cela fonctionne. Pour ce faire exécuter votre projet !
  - S'il tourne sans lever d'erreurs, alors ouvrir l'outil **database** d'**IntelliJ** et afficher le contenu des tables :

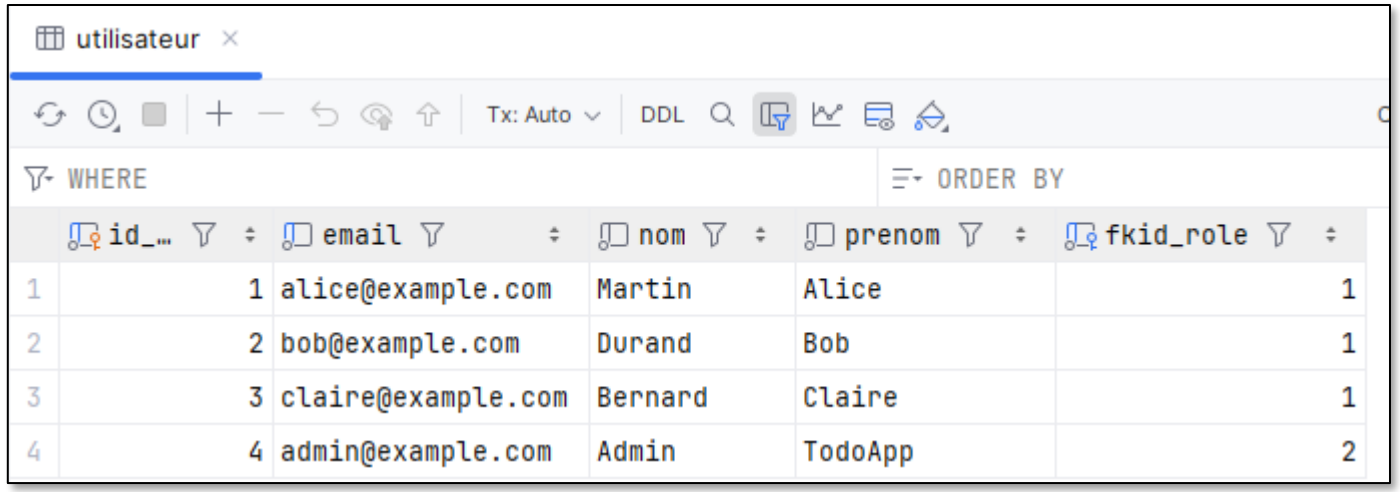




EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Test du DataInitializer :**
  - Nous allons maintenant voir si cela fonctionne. Pour ce faire exécuter votre projet !
  - Si il tourne sans lever d'erreurs, alors ouvrir l'outil **database** d'**IntelliJ** et afficher le contenu des tables :





	id_…	email	nom	prenom	fkid_role
1	1	alice@example.com	Martin	Alice	1
2	2	bob@example.com	Durand	Bob	1
3	3	claire@example.com	Bernard	Claire	1
4	4	admin@example.com	Admin	TodoApp	2

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Test du DataInitializer :**
  - Nous allons maintenant voir si cela fonctionne. Pour ce faire exécuter votre projet !
  - Si il tourne sans lever d'erreurs, alors ouvrir l'outil **database** d'**IntelliJ** et afficher le contenu des tables :

tache

Tx: Auto | DDL

WHERE | ORDER BY

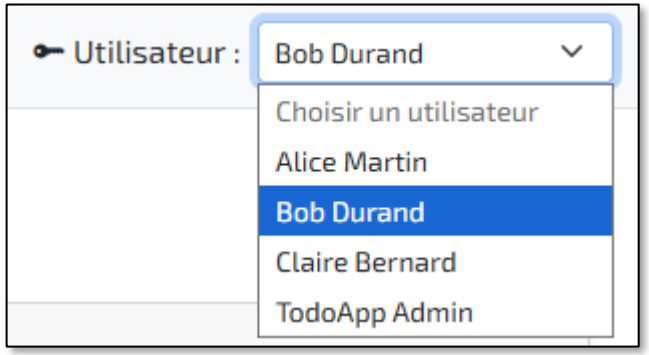
	id…	date_creation	date_echeance	description	terminee	titre	fkid_utilisateur
1	1	2026-10-26 22:52:14.463884	2026-11-02	Rédiger le support de cour…	false	Préparer le TP Spring Boot	1
2	2	2026-10-26 22:52:14.463884	2026-10-24	Configurer l'environnement…	true	Installer IntelliJ IDEA	1
3	3	2026-10-26 22:52:14.463884	2026-10-29	<null>	false	Réviser les requêtes SQL	2
4	4	2026-10-26 22:52:14.463884	2026-10-21	Finaliser les méthodes DAO.	false	Terminer le projet JDBC	2
5	5	2026-10-26 22:52:14.463884	2026-11-05	<null>	false	Étudier les annotations JPA	3
6	6	2026-10-26 22:52:14.463884	<null>	<null>	true	Lire la documentation Thym…	3
7	7	2026-10-26 22:52:14.463884	2026-10-27	<null>	false	Vérifier les comptes utili…	4
8	8	2026-10-26 22:52:14.463884	<null>	<null>	false	Contrôler les tâches enreg…	4

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Choix des utilisateurs dans l'application :**
  - **Bilan :**
    - Nous disposons maintenant de données dans notre base de données qui vont nous permettre de développer et tester simultanément nos **vues** et **contrôleurs**.
  - **Besoin immédiat :**
    - Pour mener nos développement et tests fonctionnels, nous avons besoin de pouvoir basculer entre les 4 utilisateurs : *Alice*, *Bob*, *Claire* et *Admin*.

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Choix des utilisateurs dans l'application :**
  - **Problématique :**
    - Cela nécessite de mettre en place un système d'authentification or à ce stade du développement cela compliquerait le code et son débogage !
  - **Solution proposée :**
    - Nous ne mettrons pas en place les solutions authentification, cependant nous allons ajouter une liste déroulante à droite de notre header qui nous permettra de basculer d'un utilisateur à un autre sans procéder à une authentification.



EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Architecture pour la sélection & mémorisation de l'utilisateur :**
  - Pour mettre en place notre stratégie nous devons anticiper quelques fonctionnalités techniques :
    - **1/** Récupérer la liste des utilisateurs de la base de données et la communiquer aux vues avec un contrôleur.
    - **2/** Récupérer le choix de l'utilisateur avec un contrôleur et l'enregistrer dans **HttpSession**.
    - **4/** Mémoriser l'utilisateur sélectionné et le transmettre à chaque requête **GET**.

  - On va confier la fonction technique 1, 2 et 4 à un seul contrôleur que l'on nommera **`UserSessionAdvice`**.

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Architecture pour la sélection & mémorisation de l'utilisateur :**
  - Le contrôleur **`UserSessionAdvice`** possèdera un comportement particulier grâce à l'annotation **@ControllerAdvice**.
  - En effet, cette annotation permet à **Spring Boot** d'appliquer des traitements communs aux différents contrôleurs de notre application. Associée à l'annotation **@ModelAttribute**, elle permet d'exécuter automatiquement une méthode avant les méthodes des contrôleurs concernés, afin d'enrichir le modèle de données transmis aux vues.

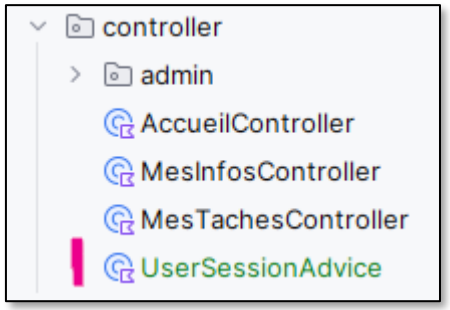
EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Architecture pour la sélection & mémorisation de l'utilisateur :**
  - Nous profitons de ce mécanisme pour mettre à disposition des vues **Thymeleaf** deux informations :
    - **L'utilisateur actif**, dont l'identifiant est mémorisé dans la **session HTTP** (**HttpSession**).
    - **La liste des utilisateurs** présents dans la base de données, nécessaire pour alimenter notre liste déroulante de sélection.

  - Ces informations sont ainsi accessibles dans les différentes vues **Thymeleaf** sans avoir à les transmettre explicitement depuis chaque contrôleur.

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Architecture pour la sélection & mémorisation de l'utilisateur :**
  - **A FAIRE :**
    - Créer le fichier **`UserSessionAdvice.kt`** dans le package **`controller`**



    - Ouvrir le fichier **`UserSessionAdvice.kt`** et coller la structure du code :

```
package org.ldv.todoapp.controller
import org.ldv.todoapp.model.repository.UtilisateurRepository
import org.springframework.web.bind.annotation.ControllerAdvice

@ControllerAdvice
class UserSessionAdvice(
   private val utilisateurRepository: UtilisateurRepository
) {
   // ...
}
```

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Architecture pour la sélection & mémorisation de l'utilisateur :**
  - Comme convenu, notre classe est précédée de l'annotation `@ControllerAdvice`.
  - On va faire instancier un objet de la classe **`UtilisateurRepository`** afin de pouvoir récupérer la liste des utilisateurs présent dans la base de données.

```
package org.ldv.todoapp.controller
import org.ldv.todoapp.model.repository.UtilisateurRepository
import org.springframework.web.bind.annotation.ControllerAdvice

@ControllerAdvice
class UserSessionAdvice(
    private val utilisateurRepository: UtilisateurRepository
) {
    // ...
}
```

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Architecture pour la sélection & mémorisation de l'utilisateur :**
  - Compléter le code avec la méthode **`enrichirModeleUtilisateurs`**.
  - Attention, il faudra importer **`HttpSession`** avec :
    - import jakarta.servlet.http.HttpSession

```
package org.ldv.todoapp.controller
import jakarta.servlet.http.HttpSession
import org.ldv.todoapp.model.repository.UtilisateurRepository
import org.springframework.ui.Model
import org.springframework.web.bind.annotation.ControllerAdvice
import org.springframework.web.bind.annotation.ModelAttribute

@ControllerAdvice
class UserSessionAdvice(
   private val utilisateurRepository: UtilisateurRepository
) {
   @ModelAttribute
   fun enrichirModeleUtilisateurs(
      model: Model,
      session: HttpSession
   ) {
      // Récupération de la liste des utilisateurs …
      // Récupération de l'utilisateur actif …
      // Transmission des deux informations au Model …
   }
}
```

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Architecture pour la sélection & mémorisation de l'utilisateur :**
  - A propos de **Model** :
    - **Model** est une interface fournie par **Spring Boot** qui permet à un **contrôleur** de transmettre des données à une **vue**, notamment à un template **Thymeleaf**.
    - Elle joue le rôle de conteneur de données, sous la forme d'associations *nom / valeur*, que le moteur de template pourra exploiter pour générer dynamiquement le contenu **HTML**.

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **A propos de httpSession et de la gestion des sessions HTTP :**
  - Le protocole **HTTP** est **sans état** (*stateless*) :
    - Le protocole **HTTP** est fondamentalement sans état : chaque requête **HTTP** est indépendante des précédentes.
    - Par défaut, un serveur HTTP **ne conserve pas automatiquement le contexte des échanges précédents** avec un navigateur.
    - Par exemple, lorsqu'un utilisateur consulte successivement plusieurs pages d'une application, le serveur doit disposer d'un mécanisme supplémentaire pour reconnaître qu'il s'agit du même utilisateur.
    - C'est précisément le rôle des **sessions HTTP**.

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **A propos de httpSession et de la gestion des sessions HTTP :**
  - Le principe d'une **session HTTP** :
    - Une **session HTTP** permet au serveur de conserver des informations associées à un navigateur entre plusieurs requêtes.
    - Dans une application **Spring Boot**, l'interface **`HttpSession`**, fournie par **Jakarta Servlet**, permet d'accéder à ces informations.
    - Lorsqu'une session est créée, le serveur lui associe un identifiant unique.
    - Cet identifiant est généralement transmis au navigateur au moyen d'un cookie nommé **`JSESSIONID`**.
    - Le navigateur conserve ce cookie et le renvoie automatiquement au serveur lors des requêtes suivantes vers le site concerné.

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **A propos de httpSession et de la gestion des sessions HTTP :**
  - Le principe d'une **session HTTP :**
    - Le schéma suivant représente les données et échange dans le cadre d'une session HTTP :



  - **A retenir :**
    - **HTTP** est un protocole **sans état**, mais une application Web peut maintenir un état utilisateur grâce à un mécanisme de **session**, associant un identifiant conservé par le navigateur à des données stockées côté serveur.

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Les différentes solutions de stockage des données Web**
  - La problématique de stockage est une problématique récurrente avec plusieurs solutions :

Mécanisme	Emplacement	Durée de conservation habituelle	Utilisation
Model Spring MVC	Serveur, pendant le traitement de la requête	Une requête	Transmettre des données à Thymeleaf
HttpSession	Serveur (par défaut, en mémoire)	Jusqu'à expiration ou invalidation	Conserver l'utilisateur actif
Cookies	Navigateur	Session du navigateur ou date d'expiration	Identifiant de session, préférences
sessionStorage	Navigateur	Durée de vie de l'onglet	Données temporaires côté client
localStorage	Navigateur	Persistant jusqu'à suppression	Préférences d'affichage, paramètres locaux

  - Deux distinctions sont essentielles :
    - **HttpSession** est un mécanisme côté serveur, même si un cookie côté navigateur permet généralement d'identifier la session.
    - **sessionStorage** et **localStorage** sont des mécanismes côté navigateur, accessibles notamment par **JavaScript**. Leurs données ne sont pas automatiquement transmises au serveur à chaque requête HTTP.

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Architecture pour la sélection & mémorisation de l'utilisateur :**
  - Reprenons maintenant le code de notre contrôleur.
  - Implémentons le code qui nous permet en 2 étapes :
    - Récupérer la liste des utilisateurs avec la méthode **`findAll()`** de JPA/Hibernate
    - L'ajout la liste des utilisateurs au **model** transmis aux vues **Thymeleaf** :

```
@ModelAttribute
  fun enrichirModeleUtilisateurs(
      model: Model,
      session: HttpSession
  ) {
      // Récupération de la liste des utilisateurs
      val utilisateurs = utilisateurRepository.findAll()

      // Transmission de la liste au modèle
      model.addAttribute("utilisateurs", utilisateurs)
  }
}
```

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Architecture pour la sélection & mémorisation de l'utilisateur :**
  - Pour finir : Récupération l'**id** utilisateur dans **HttpSession**
  - Recherche de l'**utilisateur** par son **id** dans la base de données
  - Ajout de l'utilisateur actif dans le **modèle** transmis aux vues **Thymeleaf**.

```
// Récupération de l'identifiant de l'utilisateur actif
val utilisateurId = session.getAttribute("utilisateurId") as? Long

// Recherche de l'utilisateur actif dans la BD
val utilisateurActif: Utilisateur? = utilisateurId?.let {
      utilisateurRepository.findByIdAvecRole(it)
   }

// Transmission de l'utilisateur actif au modèle
model.addAttribute("utilisateurActif", utilisateurActif)
```

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Architecture pour la sélection & mémorisation de l'utilisateur :**
  - Il nous reste à mémoriser l'information de l'utilisateur dans **HttpSession**.
  - Mais avant cela nos allons déjà tester si le code que nous avons implémenté est opérationnel.
  - Pour ce faire nous allons rajouter une liste déroulante des utilisateurs à droite de l'entête.
  - Ouvrir le fichier de template :
    - `/templates/fragments/header.html`

```
<div xmlns:th="http://www.thymeleaf.org">
<header th:fragment="header(activePage)"
      class="bg-light border-bottom">
  <div class="container d-flex align-items-center py-2">
    <!-- Logo de l'application -->
    <a th:href="@{/}" class="me-4">
      <img th:src="@{/images/favicon.ico}"
          alt="TodoApp"
          width="40"
          height="40">
    </a>
    <!-- Barre de navigation -->
    <div th:replace="~{fragments/navbar :: navbar(${activePage})}"></div>
 </div>
</header>
</div>
```

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Architecture pour la sélection & mémorisation de l'utilisateur :**
  - Pour ce faire, se placer après l'insertion de la barre de navigation.
  - Créer un bloc **`<div>…</div>`** qui accueillera notre liste :

```
<!-- Barre de navigation -->
<div th:replace="~{fragments/navbar :: navbar(${activePage})}"></div>

<!-- Liste déroulante des utilisateurs -->
<div class="ms-auto d-flex align-items-center gap-2">

</div>
```

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Architecture pour la sélection & mémorisation de l'utilisateur :**
  - Rajoutons le label et le bloc de balise **`<select>…</select>`**

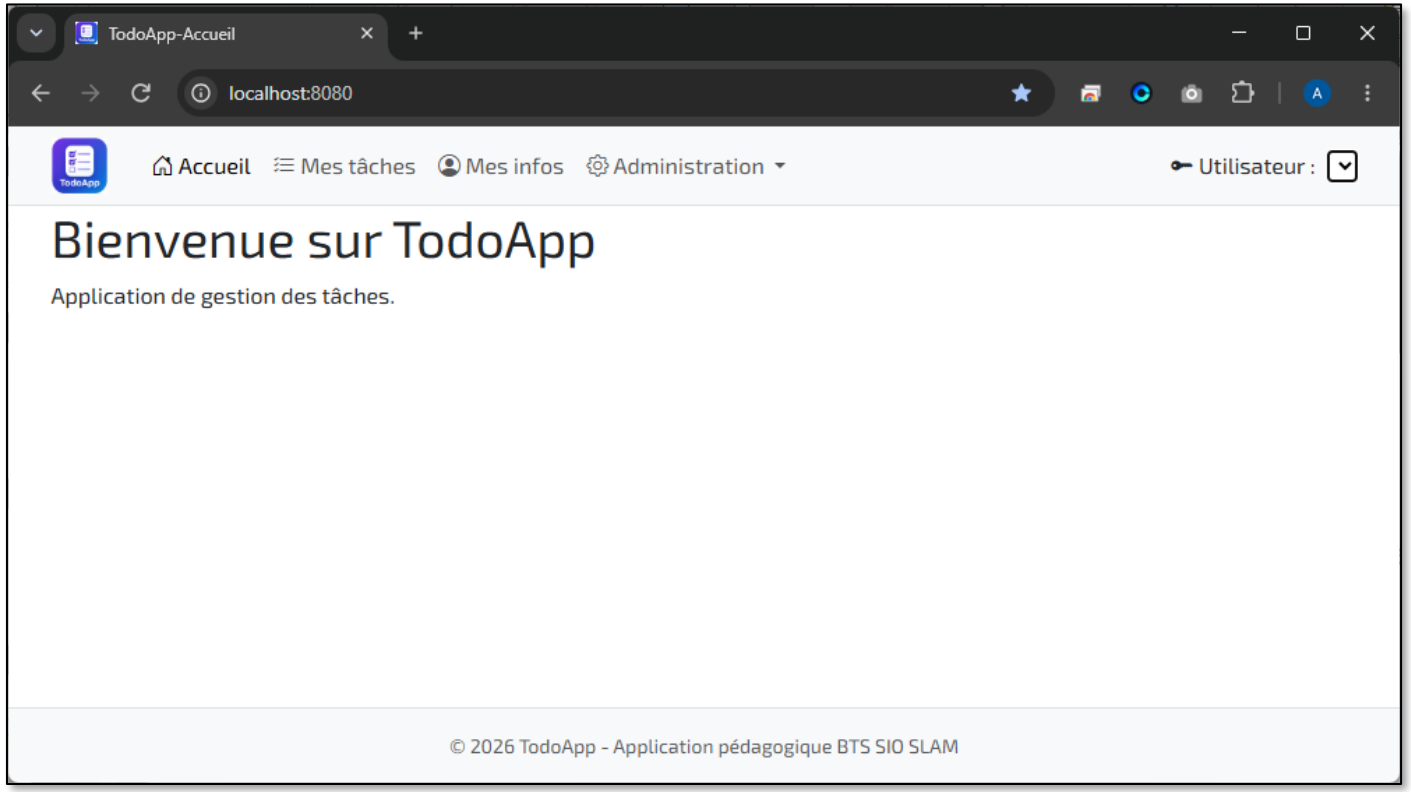
```
<!-- Liste déroulante des utilisateurs -->
<div class="ms-auto d-flex align-items-center gap-2">
   <label for="utilisateurId" class="text-nowrap">
      <i class="bi bi-key-fill"></i>
      Utilisateur :
   </label>

   <select id="utilisateurId">

   </select>
</div>
```

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Architecture pour la sélection & mémorisation de l'utilisateur :**
  - Si vous exécutez l'application, vous observerez la présence du label et de la liste vide.



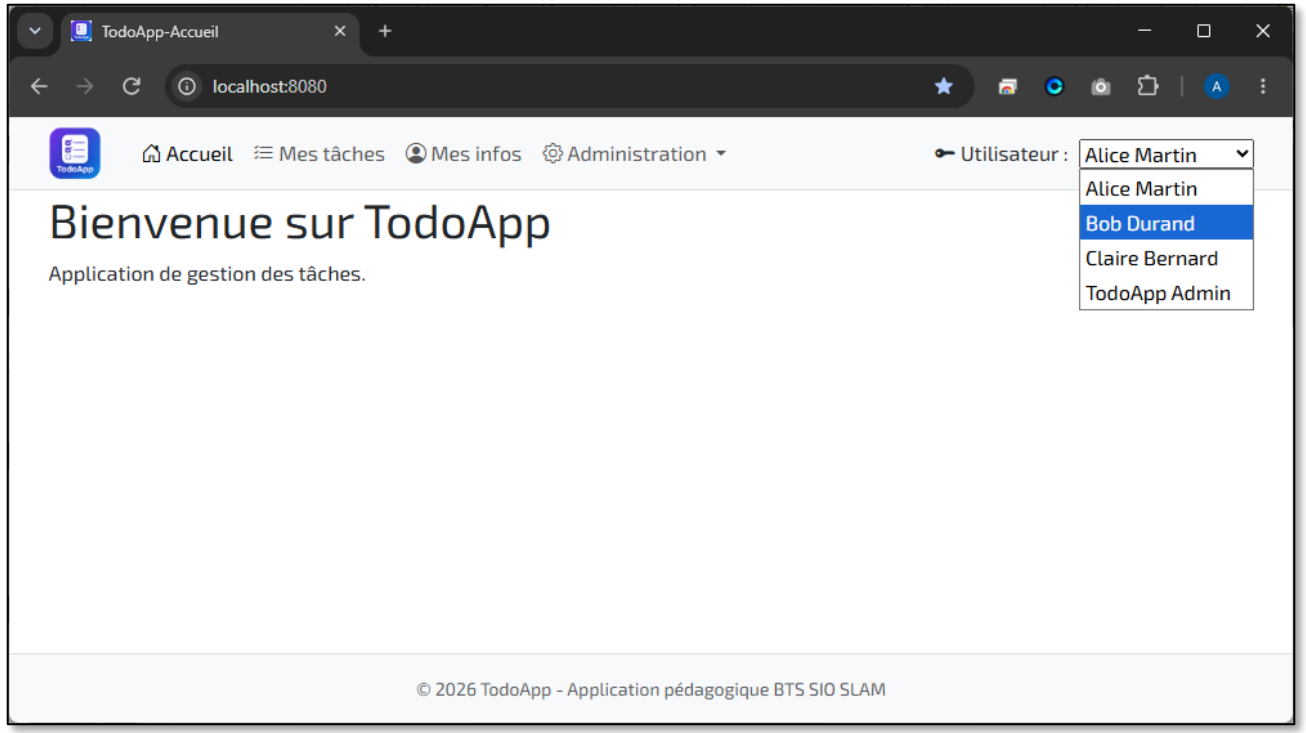
EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Architecture pour la sélection & mémorisation de l'utilisateur :**
  - Pout peupler la liste ***select***, nous allons itérer sur la liste d'utilisateurs transmise au **modèle Thymeleaf**.
  - La boucle d'itération se réalise avec l'instruction :
    - th:each="utilisateur : ${utilisateurs}"
    - utilisateurs est l'itérable et utilisateur la variable d'itération.
  - Appliqué à notre liste select cela donne :

```
<select id="utilisateurId">
   <option th:each="utilisateur : ${utilisateurs}"
         th:value="${utilisateur.idUtilisateur}"
         th:text="${utilisateur.prenom + ' ' + utilisateur.nom}"
         th:selected="${utilisateurActif != null
               and utilisateur.idUtilisateur == utilisateurActif.idUtilisateur}">
   </option>
</select>
```

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Architecture pour la sélection & mémorisation de l'utilisateur :**
  - Si vous retestez votre application, vous pourrez constater l'apparition des utilisateurs dans la liste !



EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Architecture pour la sélection & mémorisation de l'utilisateur :**
  - Nous allons ensuite rajouter l'option sélectionnée :

```
<!-- Liste déroulante des utilisateurs -->
<div class="ms-auto d-flex align-items-center gap-2">
  <label for="utilisateurId" class="text-nowrap">
    <i class="bi bi-key-fill"></i>
    Utilisateur :
  </label>

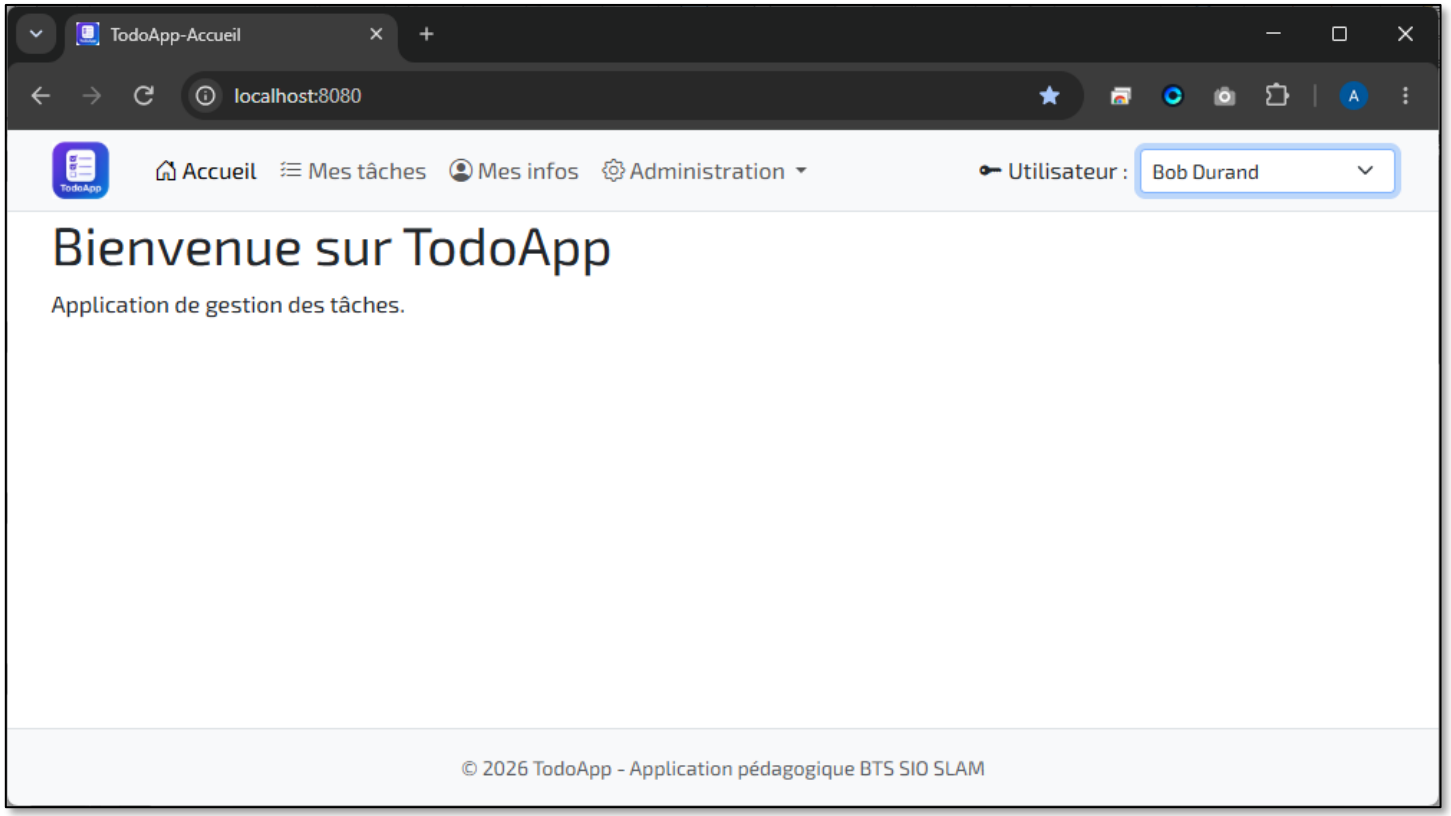
  <select id="utilisateurId"
          name="utilisateurId"
          class="form-select form-select-sm">

    <option value="" disabled
            th:selected="${utilisateurActif == null}">
      Choisir un utilisateur
    </option>

    <option th:each="utilisateur : ${utilisateurs}"
            th:value="${utilisateur.idUtilisateur}"
            th:text="${utilisateur.prenom + ' ' + utilisateur.nom}"
            th:selected="${utilisateurActif != null
                  and utilisateur.idUtilisateur == utilisateurActif.idUtilisateur}">
    </option>
  </select>
</div>
```

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Architecture pour la sélection & mémorisation de l'utilisateur :**
  - Refaite un test de votre application :



EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Architecture pour la sélection & mémorisation de l'utilisateur :**
  - Il nous reste à insérer notre `<select>…</select>` dans un formulaire pour envoyer avec une méthode **HTTP POST** notre sélection.

```
<!-- Barre de navigation -->
<div th:replace="~{fragments/navbar :: navbar(${activePage})}"></div>

<!-- Formulaire de sélection de l'utilisateur -->
<form th:action="@{/session/utilisateur}"
    method="post"
    class="ms-auto d-flex align-items-center gap-2">
  <!-- Label associé à la liste des utilisateurs -->
  <label for="utilisateurId" class="text-nowrap">
    <i class="bi bi-key-fill"></i>
    Utilisateur :
  </label>
  <!-- Liste de sélection des utilisateurs -->
  <select id="utilisateurId"
        name="utilisateurId"
        class="form-select form-select-sm"
        onchange="this.form.requestSubmit()">

    <option value="" disabled
          th:selected="${utilisateurActif == null}">
      Choisir un utilisateur
    </option>

    <option th:each="utilisateur : ${utilisateurs}"
          th:value="${utilisateur.idUtilisateur}"
          th:text="${utilisateur.prenom + ' ' + utilisateur.nom}"
          th:selected="${utilisateurActif != null
                and utilisateur.idUtilisateur == utilisateurActif.idUtilisateur}">
    </option>
  </select>
</form>
```

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Architecture pour la sélection & mémorisation de l'utilisateur :**

  - Ce qui est important d'identifier dans le code du formulaire est la route qui est renseignée dans la balise ouvrante `<form>` avec l'attribut `th:action` :

```
<form th:action="@{/session/utilisateur}"
```

  - Le nom du paramètre **`utilisateurId`** fournit pour valeur de l'attribut : `name="utilisateurId"` de la balise `<select>`.

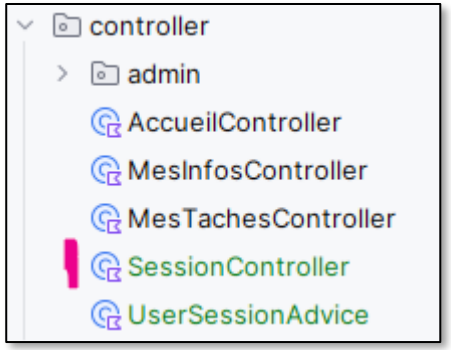
  - Et la valeur qui sera transmise dans la balise `<option>` par l'attribut value : `th:value="${utilisateur.idUtilisateur}"`

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Architecture pour la sélection & mémorisation de l'utilisateur :**

  - Nous allons maintenant rajouter le contrôleur qui va traiter cette réponse **HTTP POST** avec la route /session/utilisateur

  - Créer dans le package contrôleur une nouvelle classe : `SessionController.kt`



EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Architecture pour la sélection & mémorisation de l'utilisateur :**
  - La classe **`SessionController`** est précédée par l'annotation **`@Controller`**.
  - On utilise l'annotation **`@PostMapping`** avec l'endpoint **`"/session/utilisateur"`** pour appeler la méthode **`selectionnerUtilisateur`**.

```
package org.ldv.todoapp.controller
import jakarta.servlet.http.HttpSession
import org.ldv.todoapp.model.repository.UtilisateurRepository
import org.springframework.stereotype.Controller
import org.springframework.web.bind.annotation.PostMapping
import org.springframework.web.bind.annotation.RequestParam

@Controller
class SessionController(
   private val utilisateurRepository: UtilisateurRepository
) {
   @PostMapping("/session/utilisateur")
   fun selectionnerUtilisateur(
      @RequestParam
      utilisateurId: Long,
      session: HttpSession
   ): String {
      // ....
   }
}
```

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Architecture pour la sélection & mémorisation de l'utilisateur :**
  - Le contrôleur dispose de 2 attributs :
    - **`utilisateurId`** précédé par l'annotation **`@RequestParam`**, ce qui permettra d'instancier l'objet avec la valeur associée au paramètre **`utilisateurId`** de la requête **POST**.
    - On déclare un attribut session de type **`HttpSession`** pour pouvoir mémoriser **`utilisateurId`**.

```
package org.ldv.todoapp.controller
import jakarta.servlet.http.HttpSession
import org.ldv.todoapp.model.repository.UtilisateurRepository
import org.springframework.stereotype.Controller
import org.springframework.web.bind.annotation.PostMapping
import org.springframework.web.bind.annotation.RequestParam

@Controller
class SessionController(
   private val utilisateurRepository: UtilisateurRepository
) {
   @PostMapping("/session/utilisateur")
   fun selectionnerUtilisateur(
      @RequestParam
      utilisateurId: Long,
      session: HttpSession
   ): String {
      // ....
   }
}
```

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Architecture pour la sélection & mémorisation de l'utilisateur :**
  - Rajouter le code de la méthode selectionnerUtilisateur :

```
// Vérification de l'existence de l'utilisateur
if (!utilisateurRepository.existsById(utilisateurId)) {
    return "redirect:/"
}
// Enregistrement de l'identifiant dans la session HTTP
session.setAttribute("utilisateurId", utilisateurId)

// Redirection vers la page d'accueil
return "redirect:/"
```

  - Le retour `return "redirect:/"` rappel la page associée au endpoint `"/"`.

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Tests fonctionnels :**

  - Tester votre projet, maintenant vous pouvez sélectionner un utilisateur et constater que votre sélection reste valide pour toutes les pages durant la navigation.

  - De même, la sélection de l'utilisateur reste valide si vous ouvrez un nouvel onglet vers l'application.

  - De même lorsque vous passez d'un onglet à l'autre de l'application **TodoApp**.

EXPLOITATIONS DU MODÈLE DE DONNÉES

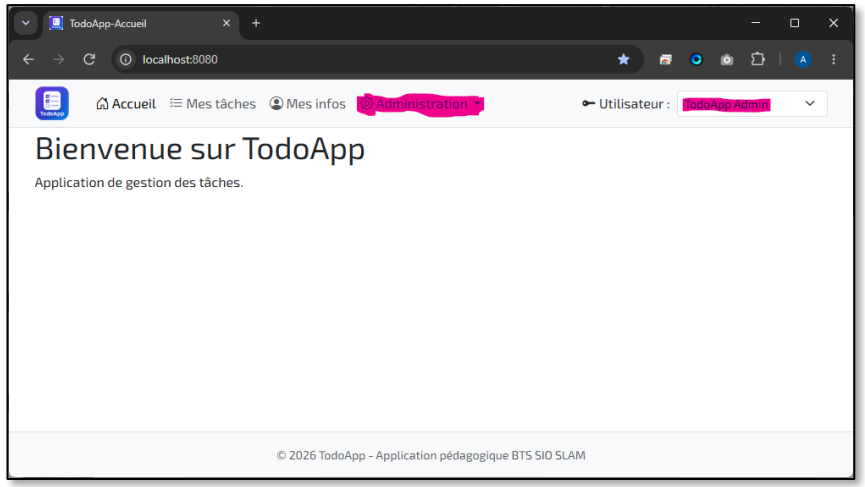
- **Amélioration de la navbar :**
  - On va maintenant profiter de notre de nouvelle information pour conditionner l'affichage du menu d'administration selon le **rôle** de l'utilisateur connecté.
  - Pour ce faire ajouter la condition :

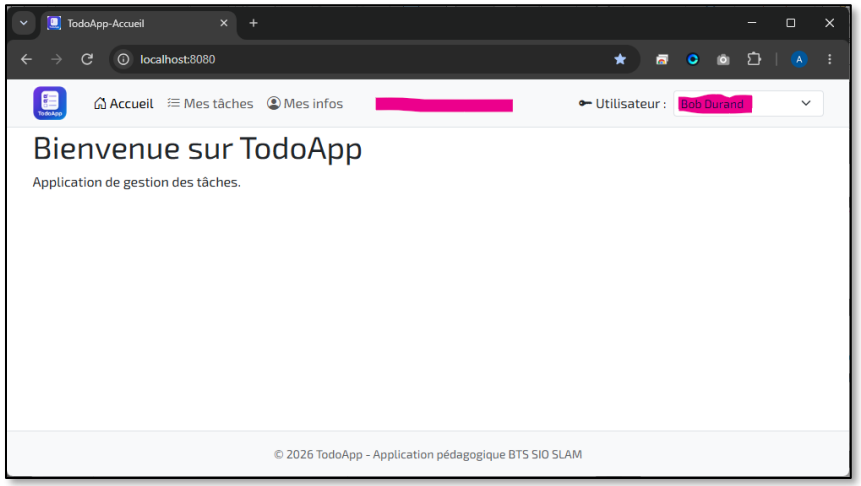
```
th:if="${utilisateurActif != null and utilisateurActif.role.nom == 'ROLE_ADMIN'}">
```

```
<!-- Groupe Administration -->
<li class="nav-item dropdown"
   th:if="${utilisateurActif != null and utilisateurActif.role.nom == 'ROLE_ADMIN'}">
   <a class="nav-link dropdown-toggle"
      href="#"
      th:classappend="${activePage != null and activePage.startsWith('admin/')} ? 'active' : ''"
      id="navbarAdmin"
      role="button"
      data-bs-toggle="dropdown"
      aria-expanded="false">
       <i class="bi bi-gear"></i>
       Administration
   </a>
```

EXPLOITATIONS DU MODÈLE DE DONNÉES

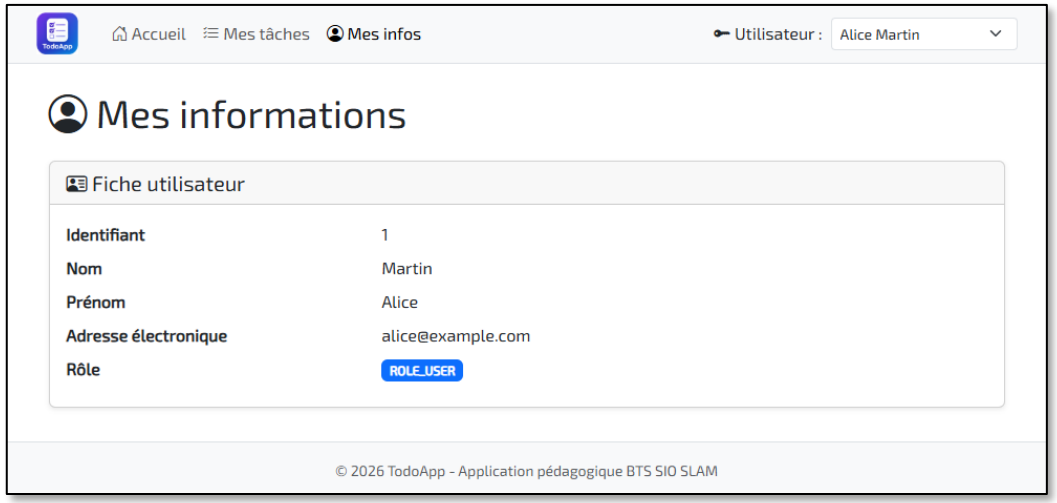
- **Tests fonctionnels :**
  - Exécuter l'application, puis basculer entre le compte admin et les autres afin de constater l'apparition/disparition du menu déroulant Administration.

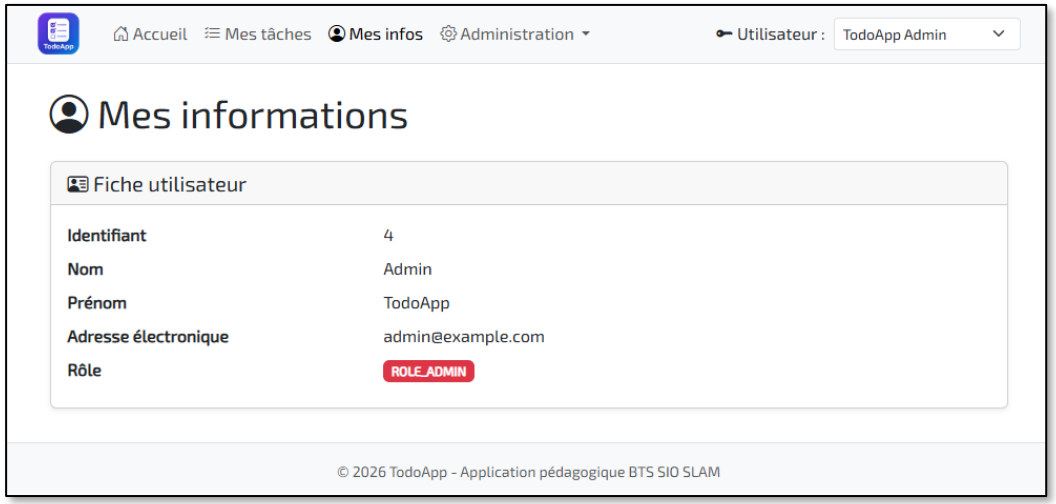




EXPLOITATIONS DU MODÈLE DE DONNÉES

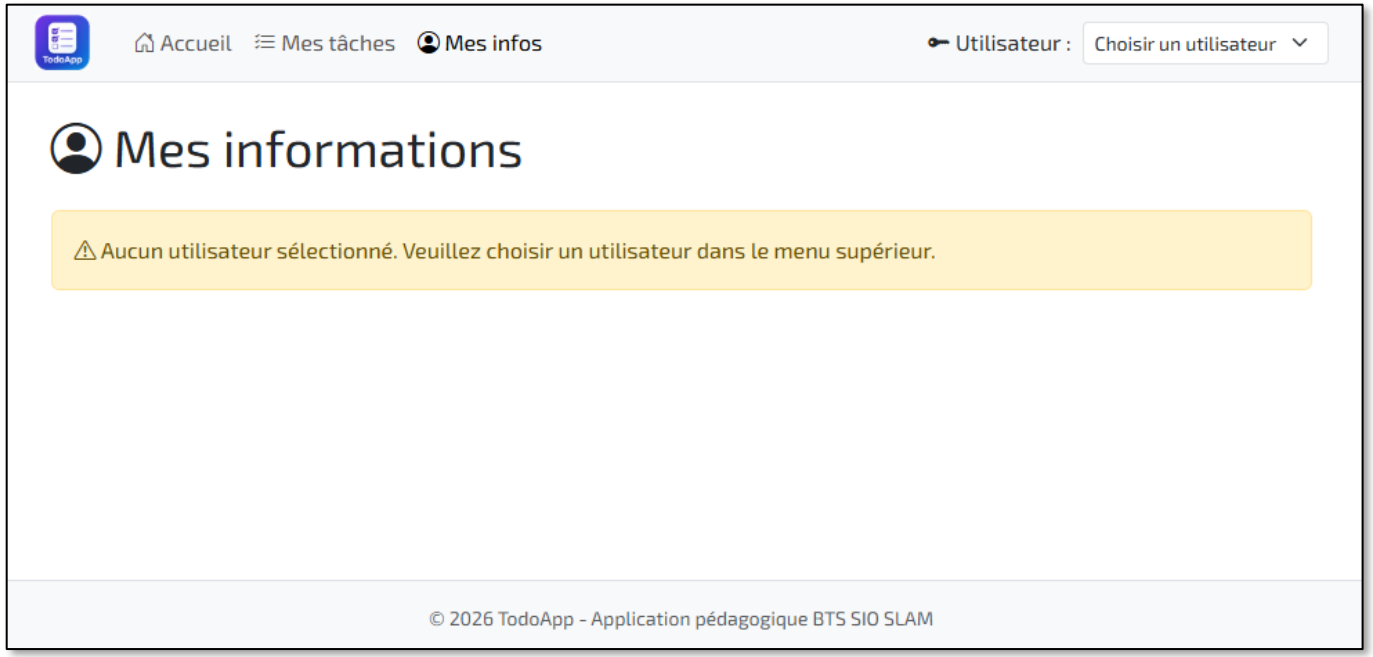
- **Présentation des données dans la vue mes-infos :**
  - L'objectif de cette vue est des présenter de façon synthétique les informations relatives à l'utilisateur actif.
  - Pour cette tâche nous n'avons pas besoin de modifier le contrôleur **`MesInfosController`** car nous disposons déjà de toutes les informations avec le contexte transmis à la **vue**.





EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Présentation des données dans la vue mes-infos :**
  - En cas d'absence d'utilisateur sélectionné, on présentera un message :



EXPLOITATIONS DU MODÈLE DE DONNÉES

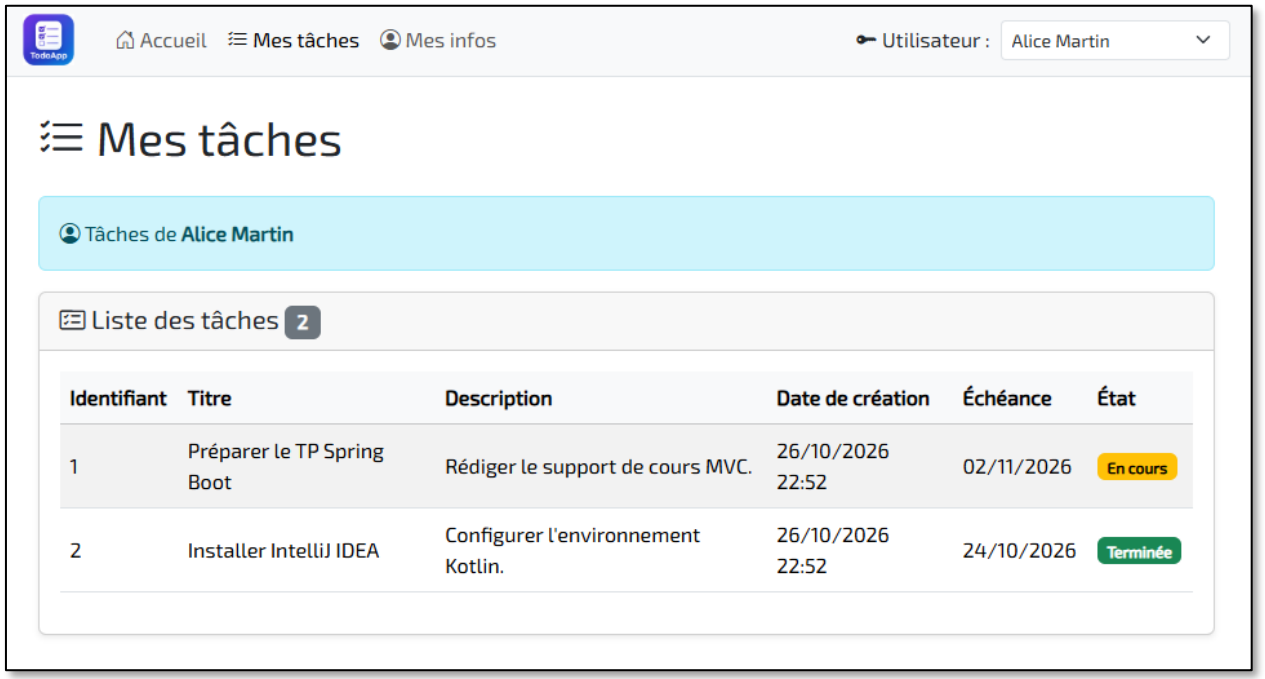
- **Présentation des données dans la vue mes-infos :**
  - On vous donne le code complet du layout de la vue ci-contre :
  - Remplacer les commentaires **`<!- A COMPLETER -->`** par une balise de la forme :
    `th:text="${utilisateurActif.attributObj}"`
  - Une fois terminé, tester votre code.

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org" th:replace="~{base :: base(~{::content}, 'TodoApp-Mes infos', 'mes-infos')}">

<div th:fragment="content" class="container py-4">
  <!-- Titre de la page -->
  <h1 class="mb-4">
    <i class="bi bi-person-circle"></i>
    Mes informations
  </h1>
  <!-- Message si aucun utilisateur n'est sélectionné -->
  <div th:if="${utilisateurActif == null}" class="alert alert-warning" role="alert">
    <i class="bi bi-exclamation-triangle"></i>
    Aucun utilisateur sélectionné.
    Veuillez choisir un utilisateur dans le menu supérieur.
  </div>
  <!-- Fiche de l'utilisateur actif -->
  <div th:if="${utilisateurActif != null}" class="card shadow-sm">
    <!-- En-tête de la fiche -->
    <div class="card-header">
      <h2 class="h5 mb-0">
        <i class="bi bi-person-vcard"></i>
        Fiche utilisateur
      </h2>
    </div>
    <!-- Informations de l'utilisateur -->
    <div class="card-body">
      <dl class="row mb-0">
        <!-- Identifiant -->
        <dt class="col-sm-4">Identifiant</dt>
        <dd class="col-sm-8" <!– A COMPLETER –> ></dd>
        <!-- Nom -->
        <dt class="col-sm-4">Nom</dt>
        <dd class="col-sm-8" <!– A COMPLETER –> ></dd>
        <!-- Prénom -->
        <dt class="col-sm-4">Prénom</dt>
        <dd class="col-sm-8" <!– A COMPLETER –> ></dd>
        <!-- Adresse électronique -->
        <dt class="col-sm-4">Adresse électronique</dt>
        <dd class="col-sm-8" <!– A COMPLETER –> ></dd>
        <!-- Rôle -->
        <dt class="col-sm-4">Rôle</dt>
        <dd class="col-sm-8">
          <span class="badge" th:classappend="${utilisateurActif.role.nom == 'ROLE_ADMIN'} ? 'text-bg-danger' : 'text-bg-primary'">
                                <!– A COMPLETER –> >
          </span>
        </dd>
      </dl>
    </div>
  </div>
</div>
</html>
```

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Présentation des données dans la vue mes-taches :**
  - Notre objectif va être ici plus ambitieux, puisqu'on désire obtenir un tableau qui liste l'ensemble des tâches de l'utilisateur actif.
  - Ci-dessous voici le résultat attendu :



EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Présentation des données dans la vue mes-taches :**
  - Pour atteindre notre objectif nous allons devoir compléter le contrôleur **`MesTachesController`** pour qu'il retourne la liste des tâches pour l'utilisateur actif.
  - Nous avions d'ailleurs défini la méthode **`findByUtilisateur_IdUtilisateur`** dans le **`TacheRepository`** pour anticiper ce besoin.

  - Ensuite nous pourrons exploiter les tâches récupérées pour les présenter dans la **vue**.

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Présentation des données dans la vue mes-taches :**
  - Ouvrir le fichier du contrôleur `MesTachesController.kt`
  - Notre contrôleur devra :
    - Disposer d'une instance de **`TacheRespository`** pour exécuter les méthodes d'accès aux données comme **`findByUtilisateur_IdUtilisateur`**
    - D'une instance à **HttpSession** pour pouvoir récupérer l'**id** de l'utilisateur actif.

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Présentation des données dans la vue mes-taches :**
  - Structure de base du contrôleur :

```
package org.ldv.todoapp.controller
import jakarta.servlet.http.HttpSession
import org.ldv.todoapp.model.repository.TacheRepository
import org.springframework.stereotype.Controller
import org.springframework.ui.Model
import org.springframework.web.bind.annotation.GetMapping

@Controller
class MesTachesController(
   private val tacheRepository: TacheRepository
) {
   @GetMapping("/mes-taches")
   fun mesTaches(model: Model, session: HttpSession): String {
      // Récupération de l'identifiant de l'utilisateur actif …
      // Récupération des tâches de l'utilisateur actif …
      // Transmission de la liste des tâches au modèle …
      // Affichage du template mes-taches.html …
   }
}
```

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Présentation des données dans la vue mes-taches :**
  - Et voici le code des instructions de la méthode **`mesTaches()`** :

```
fun mesTaches(model: Model, session: HttpSession): String {
    // Récupération de l'identifiant de l'utilisateur actif
    val utilisateurId =
        session.getAttribute("utilisateurId") as? Long

    // Récupération des tâches de l'utilisateur actif
    val taches = if (utilisateurId != null) {
        tacheRepository.findByUtilisateur_IdUtilisateur(utilisateurId)
    } else {
        emptyList()
    }

    // Transmission de la liste des tâches au modèle
    model.addAttribute("taches", taches)

    // Affichage du template mes-taches.html
    return "mes-taches"
}
```

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Présentation des données dans la vue mes-taches :**
  - Voici le squelette du layout de la vue (il manque le tableau des tâches) :

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org"
    th:replace="~{base :: base(~{::content}, 'TodoApp-Mes tâches', 'mes-taches')}">

<div th:fragment="content" class="container py-4">
  <!-- Titre de la page -->
  <h1 class="mb-4">
    <i class="bi bi-list-check"></i> Mes tâches
  </h1>

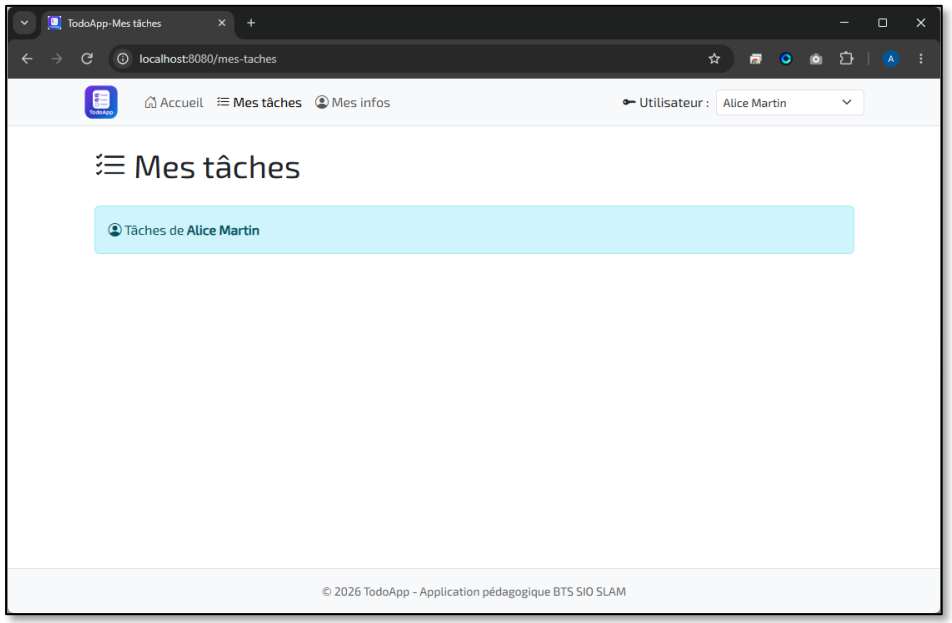
  <!-- Aucun utilisateur sélectionné -->
  <div th:if="${utilisateurActif == null}" class="alert alert-warning" role="alert">
    <i class="bi bi-exclamation-triangle"></i>
    Aucun utilisateur sélectionné. Veuillez choisir un utilisateur dans le menu supérieur.
  </div>

  <!-- Affichage si un utilisateur est sélectionné -->
  <div th:if="${utilisateurActif != null}">

    <!-- Présentation de l'utilisateur -->
    <div class="alert alert-info">
      <i class="bi bi-person-circle"></i> Tâches de
      <strong th:text="${utilisateurActif.prenom + ' ' + utilisateurActif.nom}"></strong>
    </div>
    <!-- Aucune tâche enregistrée -->
    <div th:if="${#lists.isEmpty(taches)}" class="alert alert-secondary">
      <i class="bi bi-info-circle"></i>
      Aucune tâche enregistrée pour cet utilisateur.
    </div>

    <!-- Tableau des tâches -->
    <!-- A VENIR -->

  </div>
</div>
</html>
```



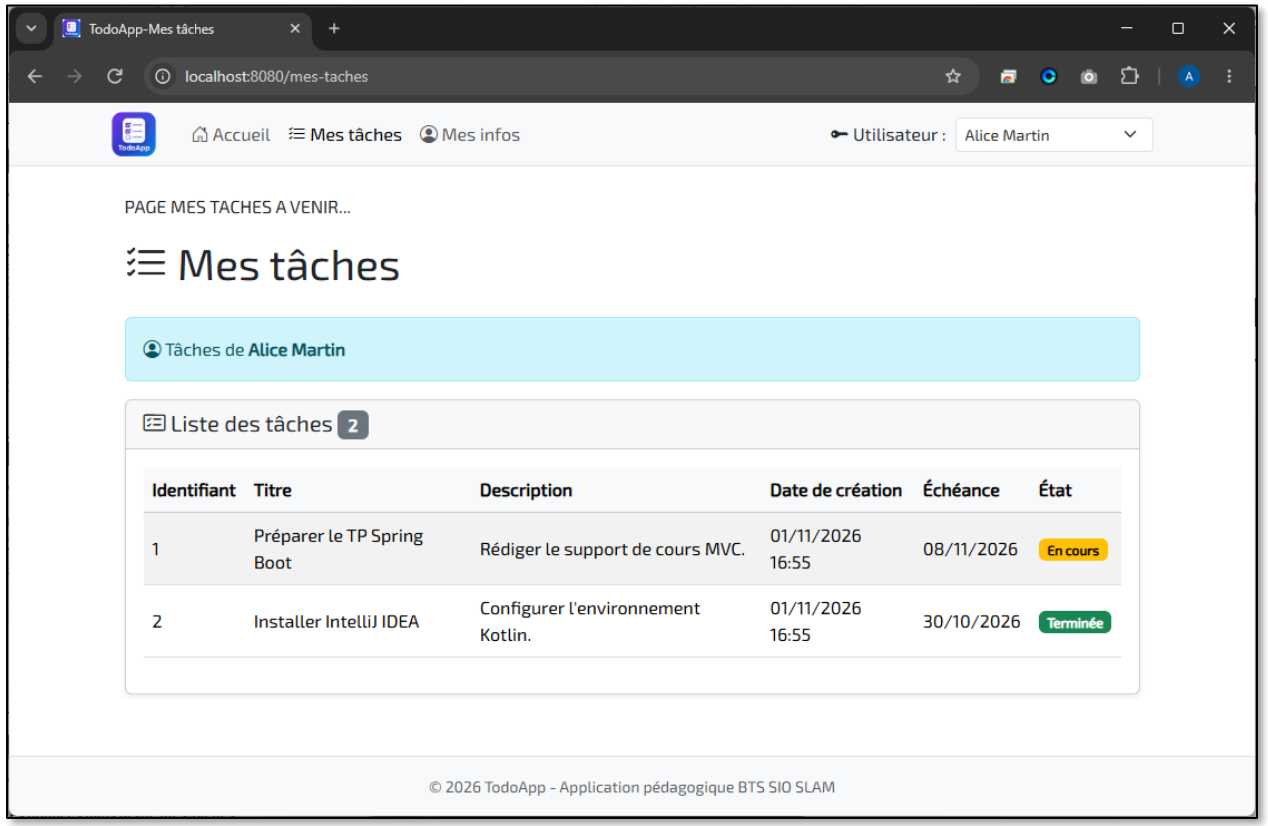
EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Présentation des données dans la vue mes-taches :**
  - Voici le code complet à rajouter pour produire l'affichage du tableau des tâches.

```
<!-- Tableau des tâches -->
<div th:if="${not #lists.isEmpty(taches)}" class="card shadow-sm">
  <div class="card-header">
    <h2 class="h5 mb-0">
      <i class="bi bi-card-checklist"></i> Liste des tâches
      <span class="badge text-bg-secondary" th:text="${#lists.size(taches)}"></span>
    </h2>
  </div>
  <div class="card-body">
    <div class="table-responsive">
      <table class="table table-striped table-hover align-middle">
        <thead class="table-light">
        <tr>
          <th>Identifiant</th> <th>Titre</th><th>Description</th><th>Date de création</th> <th>Échéance</th><th>État</th>
        </tr>
        </thead>
        <tbody>
        <tr th:each="tache : ${taches}">
          <!– Identifiant | Titre| Description -->
          <td th:text="${tache.idTache}"></td><td th:text="${tache.titre}"></td><td th:text="${tache.description != null ? tache.description : '-'}"></td>
          <!-- Date de création -->
          <td th:text="${#temporals.format(tache.dateCreation, 'dd/MM/yyyy HH:mm')}"></td>
          <!-- Date d'échéance -->
          <td th:text="${tache.dateEcheance != null ? #temporals.format(tache.dateEcheance, 'dd/MM/yyyy') : '-'}">
          </td>
          <!-- État de la tâche -->
          <td>
            <span class="badge"
                th:classappend="${tache.terminee} ? 'text-bg-success' : 'text-bg-warning'"
                th:text="${tache.terminee} ? 'Terminée' : 'En cours'">
            </span>
          </td>
        </tr>
        </tbody>
      </table>
    </div>
  </div>
</div>
```

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Présentation des données dans la vue mes-taches :**
  - Si vous avez bien implémenté le code, vous devriez obtenir le résultat suivant :



EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Présentation des données backoffice de la vue taches :**
  - Voici le squelette du template `tache.html` sans la liste des tâches.

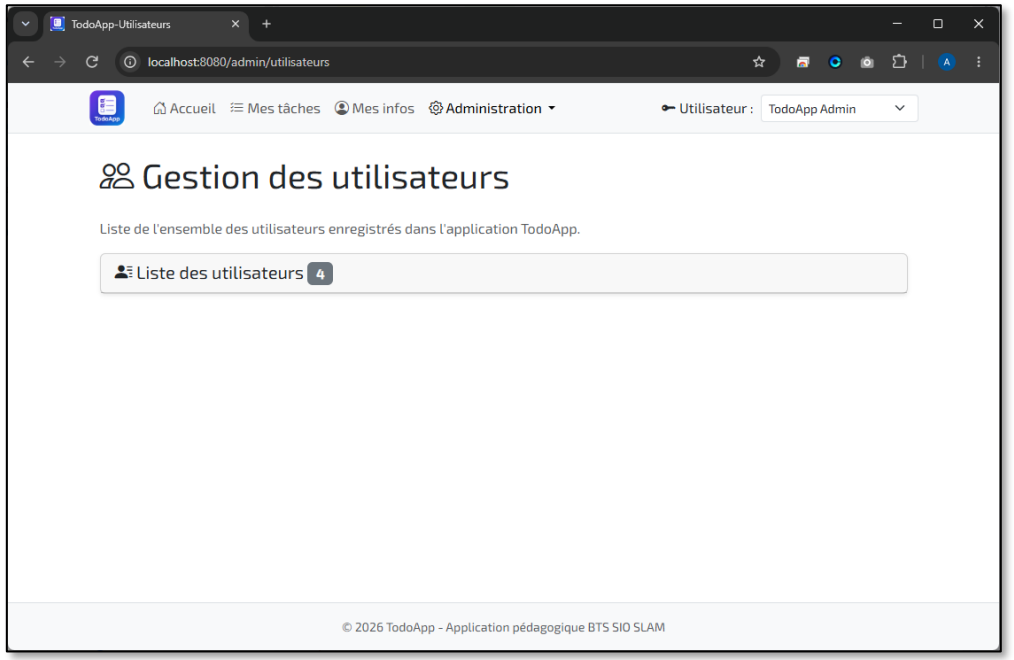
```
<html xmlns:th="http://www.thymeleaf.org"
    th:replace="~{base :: base(~{::content}, 'TodoApp-Utilisateurs', 'admin/utilisateurs')}">

<div th:fragment="content" class="container py-4">
  <!-- Titre de la page -->
  <h1 class="mb-4"><i class="bi bi-people"></i> Gestion des utilisateurs
  </h1>

  <!-- Présentation de la page -->
  <p class="text-muted">
    Liste de l'ensemble des utilisateurs enregistrés dans l'application TodoApp.
  </p>

  <!-- Aucun utilisateur enregistré -->
  <div th:if="${#lists.isEmpty(listeUtilisateurs)}" class="alert alert-info" role="alert">
    <i class="bi bi-info-circle"></i> Aucun utilisateur enregistré.
  </div>
  <!-- Tableau des utilisateurs -->
  <div th:if="${not #lists.isEmpty(listeUtilisateurs)}"
      class="card shadow-sm">

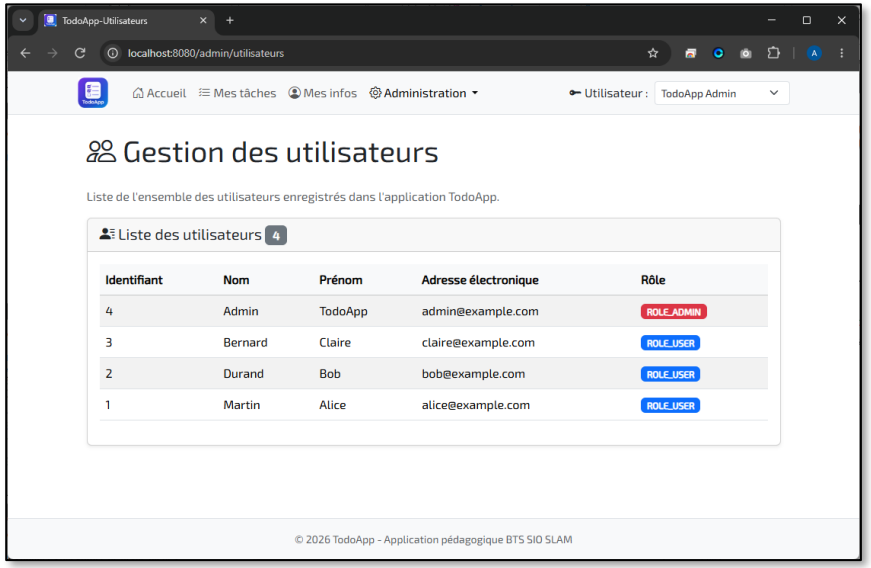
    <!-- En-tête de la carte -->
    <div class="card-header">
      <h2 class="h5 mb-0">
        <i class="bi bi-person-lines-fill"></i> Liste des utilisateurs
        <span class="badge text-bg-secondary"
            th:text="${#lists.size(listeUtilisateurs)}">
        </span>
      </h2>
    </div>
    <!-- Contenu de la carte -->
    <!-- A VENIR -->
  </div>
</div>
</html>
```



EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Présentation des données backoffice de la vue taches :**
  - Et le code à ajouter pour l'implémentation du tableau des utilisateurs :

```
<!-- Contenu de la carte -->
<div class="card-body">
  <div class="table-responsive">
    <table class="table table-striped table-hover align-middle">
      <thead class="table-light">
      <tr><th>Identifiant</th><th>Nom</th><th>Prénom</th><th>Adresse électronique</th><th>Rôle</th>
      </tr>
      </thead>
      <tbody>
      <tr th:each="utilisateur : ${listeUtilisateurs}">
        <!-- Identifiant -->
        <td th:text="${utilisateur.idUtilisateur}"></td>
        <!-- Nom -->
        <td th:text="${utilisateur.nom}"></td>
        <!-- Prénom -->
        <td th:text="${utilisateur.prenom}"></td>
        <!-- Adresse électronique -->
        <td th:text="${utilisateur.email}"></td>
        <!-- Rôle -->
        <td>
          <span class="badge"
              th:classappend="${utilisateur.role.nom == 'ROLE_ADMIN'}
                ? 'text-bg-danger'
                : 'text-bg-primary'"
              th:text="${utilisateur.role.nom}">
          </span>
        </td>
      </tr>
      </tbody>
    </table>
  </div>
</div>
```

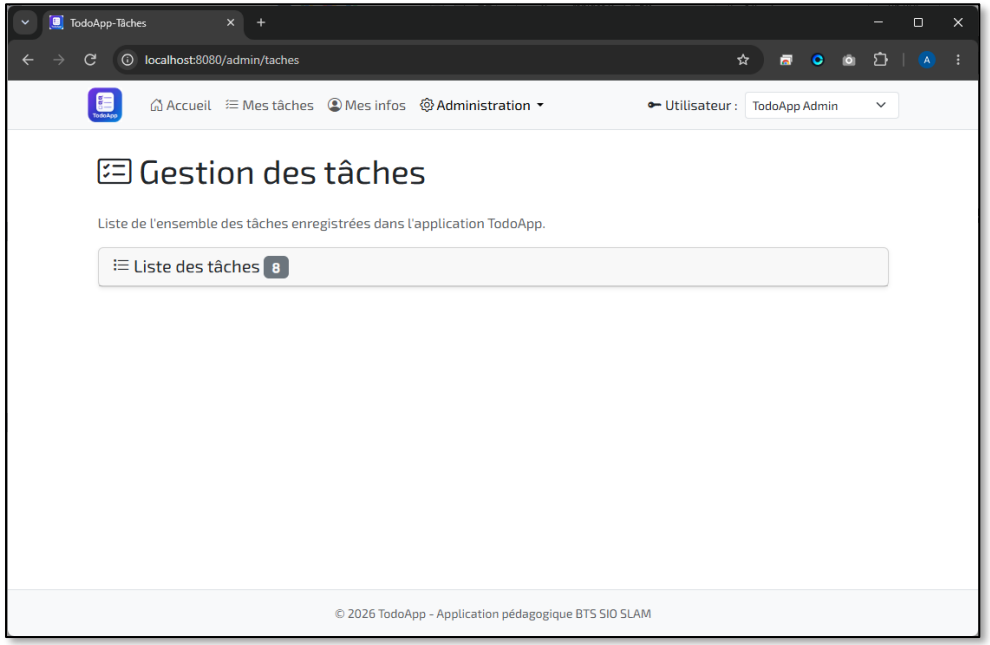


EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Présentation des données backoffice de la vue taches :**
  - Enfin le squelette code du template **`tache.html`** :

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org"
    th:replace="~{base :: base(~{::content}, 'TodoApp-Tâches', 'admin/taches')}">

<div th:fragment="content" class="container py-4">
  <!-- Titre de la page -->
  <h1 class="mb-4">
    <i class="bi bi-card-checklist"></i> Gestion des tâches
  </h1>
  <!-- Présentation de la page -->
  <p class="text-muted">
    Liste de l'ensemble des tâches enregistrées dans l'application TodoApp.
  </p>
  <!-- Aucune tâche enregistrée -->
  <div th:if="${#lists.isEmpty(listeTaches)}" class="alert alert-info" role="alert">
    <i class="bi bi-info-circle"></i> Aucune tâche enregistrée.
  </div>
  <!-- Tableau des tâches -->
  <div th:if="${not #lists.isEmpty(listeTaches)}"
     class="card shadow-sm">
    <!-- En-tête de la carte -->
    <div class="card-header">
      <h2 class="h5 mb-0">
        <i class="bi bi-list-task"></i> Liste des tâches
        <span class="badge text-bg-secondary" th:text="${#lists.size(listeTaches)}"></span>
      </h2>
    </div>
    <!-- Contenu de la carte -->
    <!-- A VENIR -->
  </div>
</div>
</html>
```



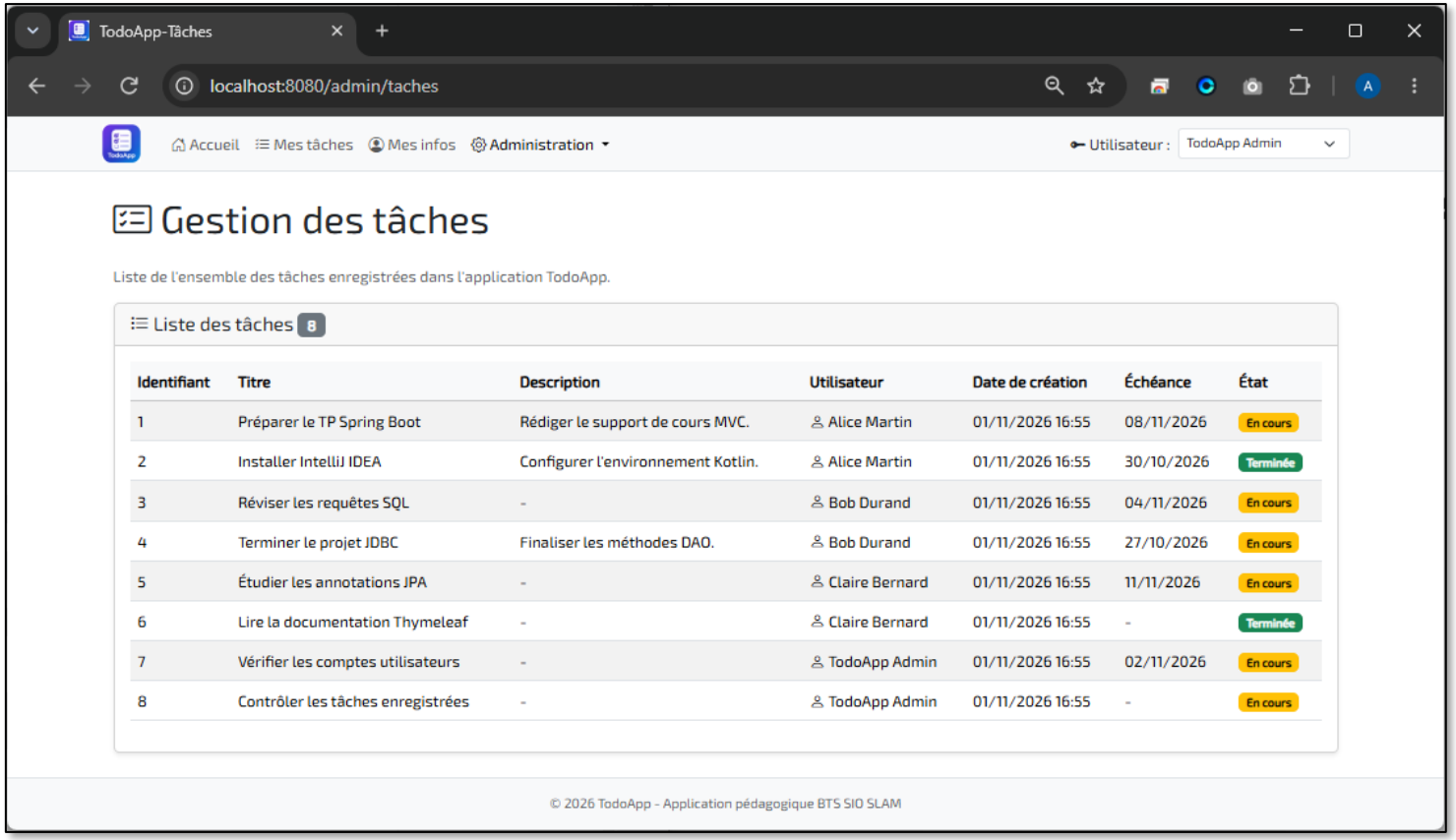
EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Présentation des données backoffice de la vue taches :**
  - Le code de la liste des tâches :

```
<!-- Contenu de la carte -->
<div class="card-body">
  <div class="table-responsive">
    <table class="table table-striped table-hover align-middle">
      <thead class="table-light">
      <tr>
        <th>Identifiant</th><th>Titre</th><th>Description</th><th>Utilisateur</th><th>Date de création</th><th>Échéance</th><th>État</th>
      </tr>
      </thead>
      <tbody>
      <tr th:each="tache : ${listeTaches}">
        <!– Identifiant | Titre | Description -->
        <td th:text="${tache.idTache}"></td><td th:text="${tache.titre}"></td> <td th:text="${tache.description != null  ? tache.description : '-'}"></td>
        <!-- Utilisateur associé -->
        <td>
          <i class="bi bi-person"></i>
          <span th:text="${tache.utilisateur.prenom + ' ' + tache.utilisateur.nom}"></span>
        </td> <!-- Date de création -->
        <td th:text="${#temporals.format(tache.dateCreation, 'dd/MM/yyyy HH:mm')}">
        </td> <!-- Date d'échéance -->
        <td th:text="${tache.dateEcheance != null ? #temporals.format(tache.dateEcheance, 'dd/MM/yyyy') : '-'}">
        </td> <!-- État de la tâche -->
        <td><span class="badge"
              th:classappend="${tache.terminee} ? 'text-bg-success' : 'text-bg-warning'"
              th:text="${tache.terminee} ? 'Terminée' : 'En cours'">
          </span>
        </td>
      </tr>
      </tbody>
    </table>
  </div>
</div>
```

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Présentation des données backoffice de la vue taches :**
  - Vous devez obtenir le résultat suivant :



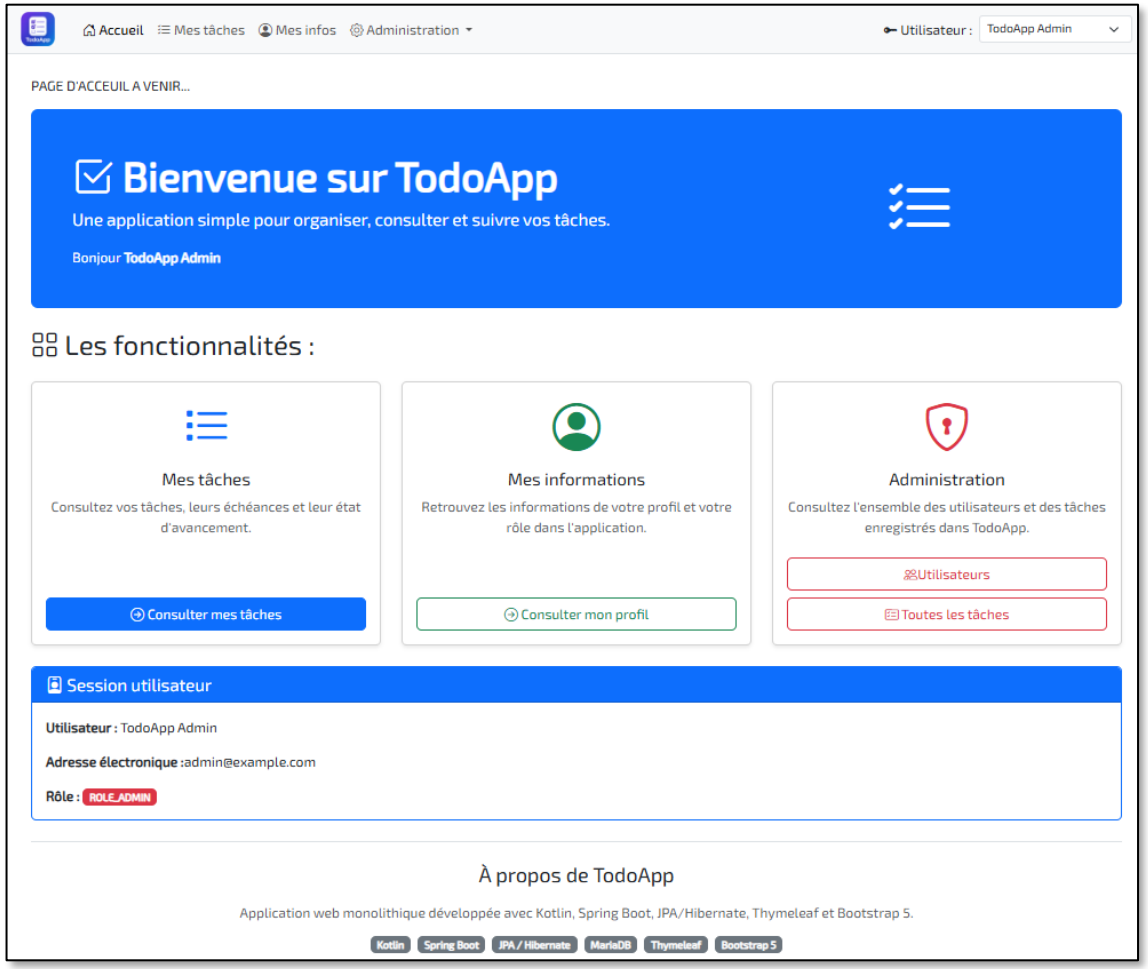
EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Personnalisation de la page d'accueil :**
  - Vous disposez maintenant de toutes les compétences et savoir pour implémenter les pages de présentation des données.
  - **A FAIRE :**
    - Ouvrir le fichier de template `accueil.html`
    - Imaginer une page d'accueil qui présenterait de façon dashboard l'agrégation de données disponibles pour l'utilisateur sélectionné.

    - Un exemple de rendu très complet est proposé à la page suivante.

EXPLOITATIONS DU MODÈLE DE DONNÉES

- **Personnalisation de la page d'accueil :**
  - Exemple de rendu pour la page d'accueil :



Fin